

Pontifícia Universidade Católica de Campinas

Estruturas e Recuperação de Dados II



FILAS



Prof. Me. Darcy H.F.R. Moreira



FILAS

Queue: Baseada no princípio **FIFO (First In, First Out)**.



FILAS

Queue: Baseada no princípio **FIFO (First In, First Out)**.



Características

- **Lineares:** Organizados em **sequência com início e fim**.
- Acesso restrito: **não** é permitido inserir nem remover em **posições intermediárias**.
 - **Inserção:** apenas no **fim**
 - **Remoção:** apenas no **início**
- Pode ser estática ou dinâmica

FILAS

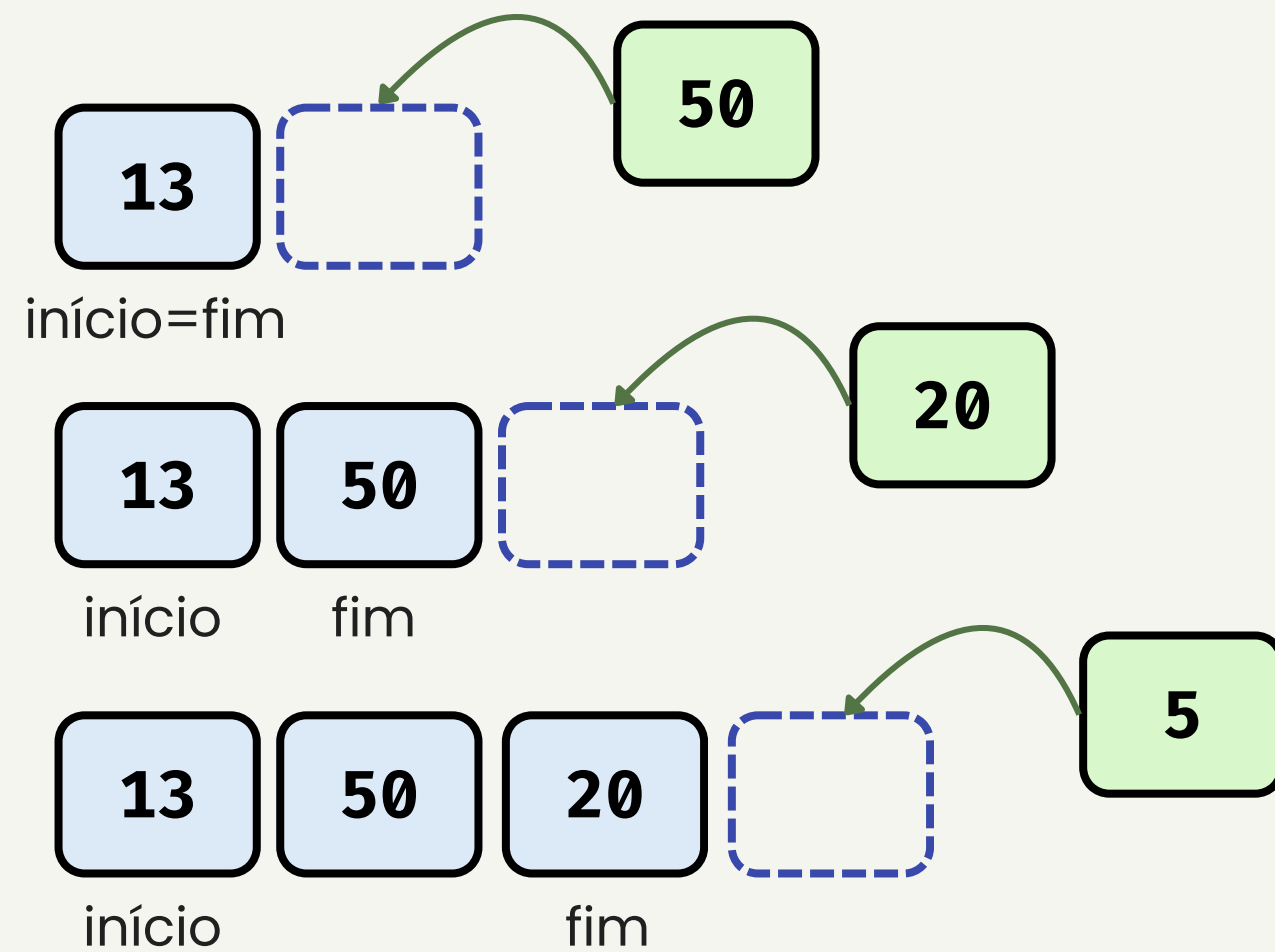
Operações

FILAS

Operações

Enfileirar (Enqueue)

Inserir um novo elemento no final da fila

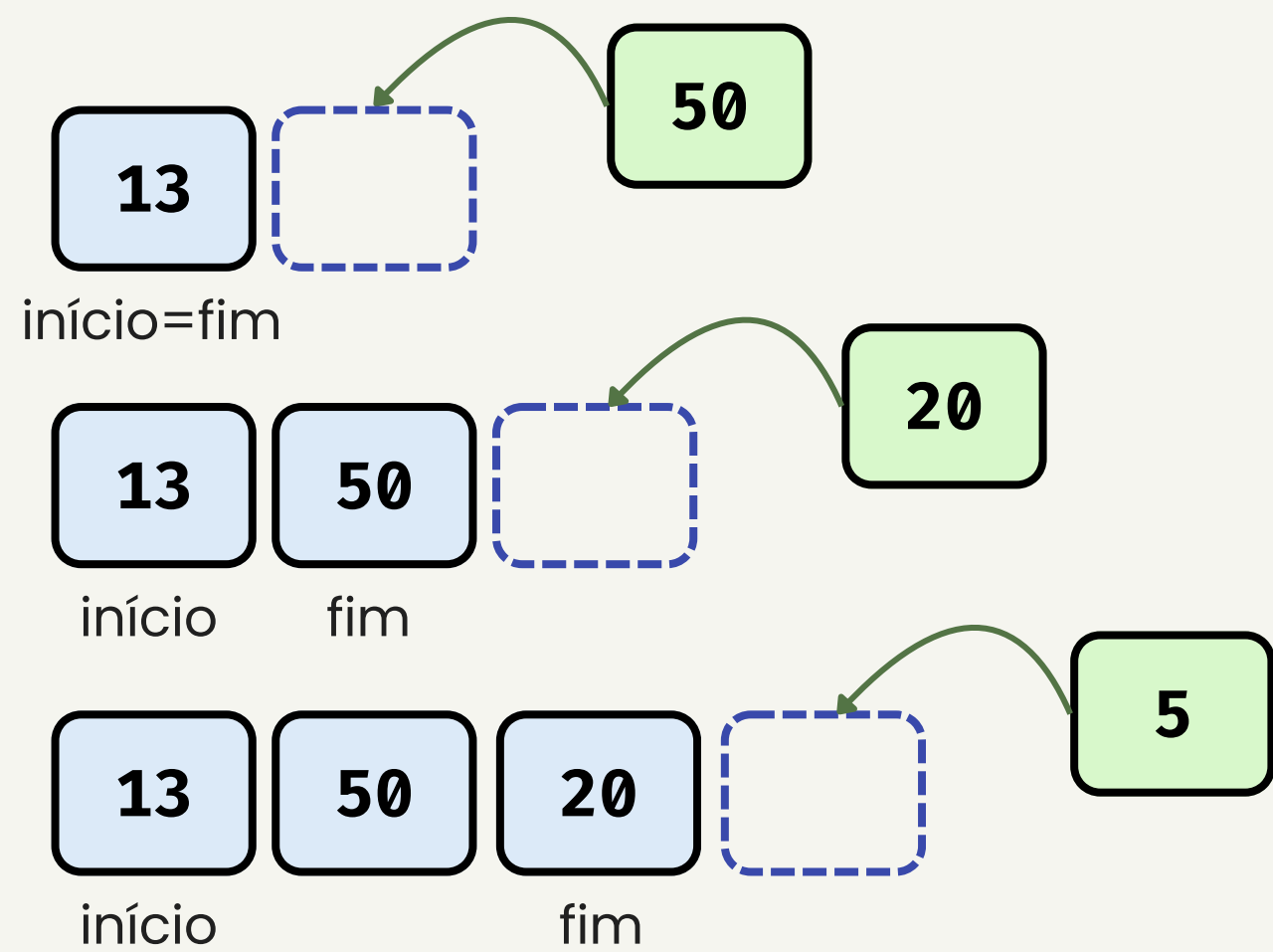


FILAS

Operações

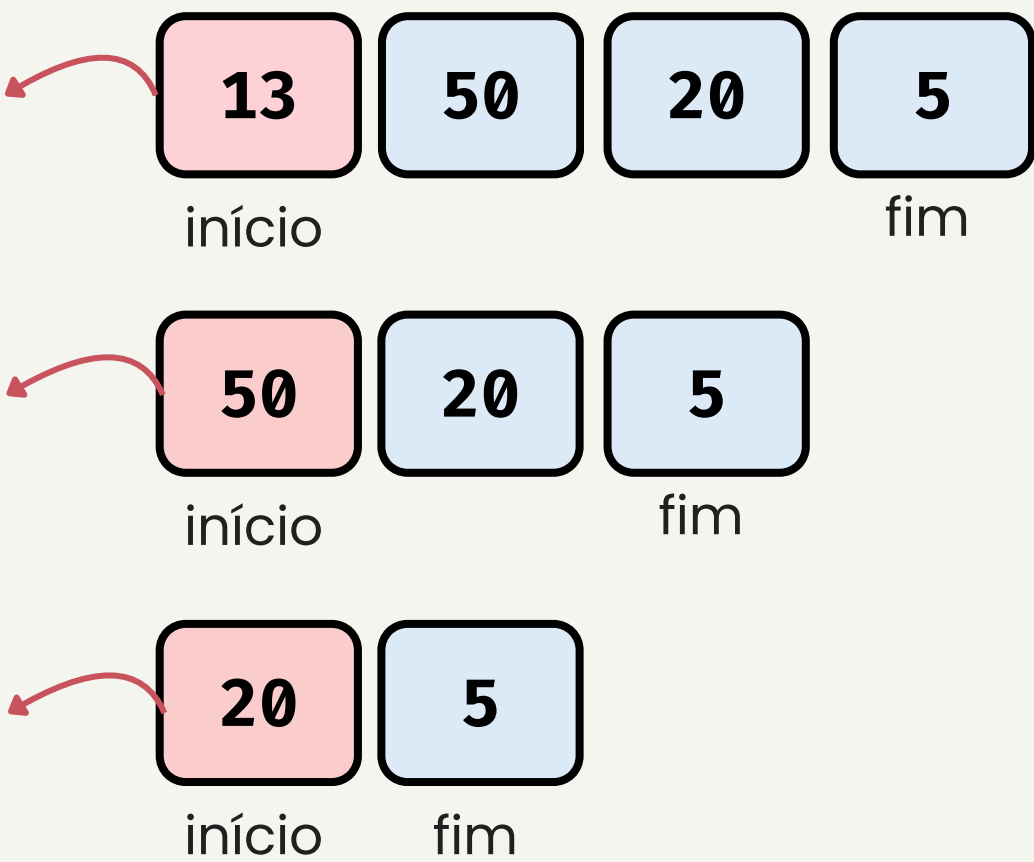
Enfileirar (Enqueue)

Inserir um novo elemento no final da fila



Desenfileirar (Dequeue)

Remover um elemento no início da fila



FILAS



- Implementação **simples e operações bem definidas**
- Útil para controlar **ordem de processamento** em sistemas de tempo real

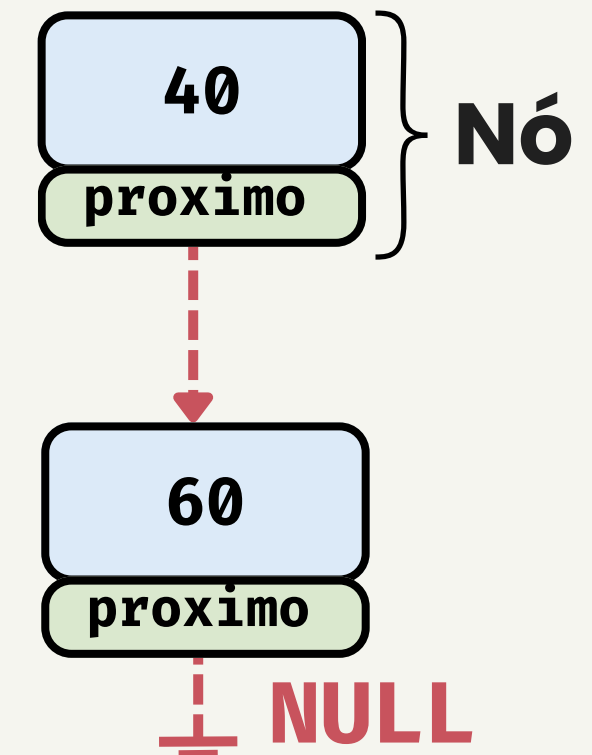


- Necessidade de **alocação/liberação** de memória a cada operação.

FILAS

- Montagem como uma **lista simplesmente encadeada**

```
typedef struct NoFila {  
    int valor;  
    struct NoFila* prox;  
} NoFila;
```

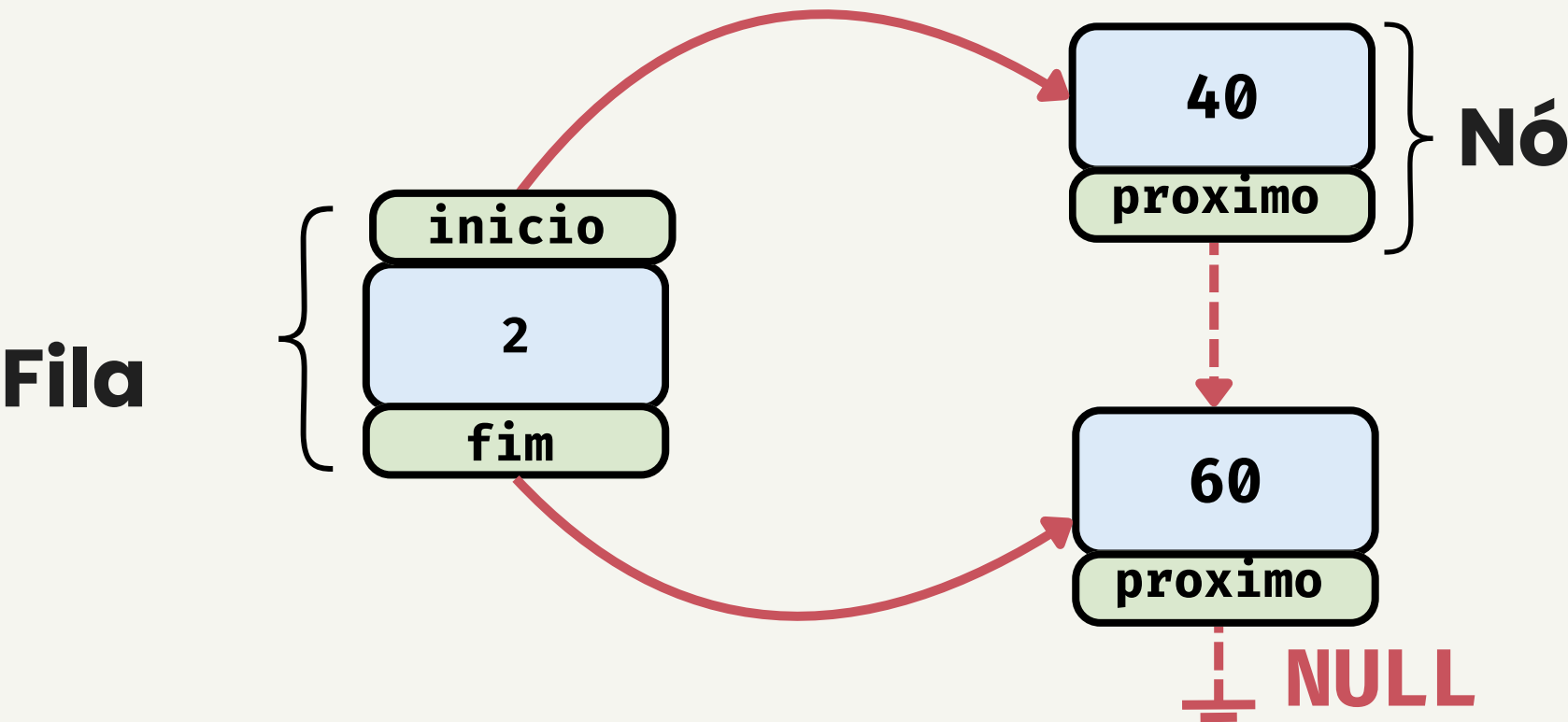


FILAS

- Montagem como uma **lista simplesmente encadeada**

```
typedef struct NoFila {
    int valor;
    struct NoFila* prox;
} NoFila;
```

```
typedef struct Fila {
    NoFila* inicio;
    NoFila* fim;
    int tamanho;
} Fila;
```



FILAS

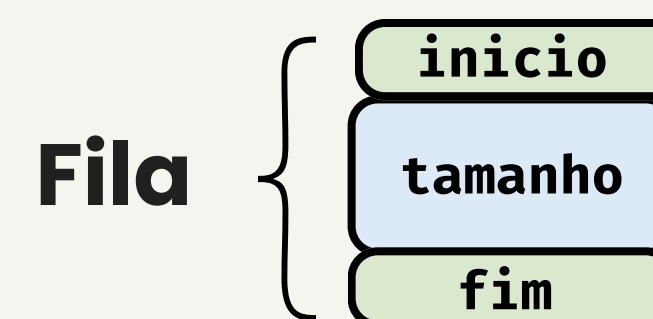
Criando uma **struct Fila**

```
Fila* criarFila() {  
    Fila* fila = (Fila*)malloc(sizeof(Fila));  
    fila->inicio = NULL;  
    fila->fim = NULL;  
    fila->tamanho = 0;  
    return fila;  
}
```

FILAS

Criando uma **struct Fila**

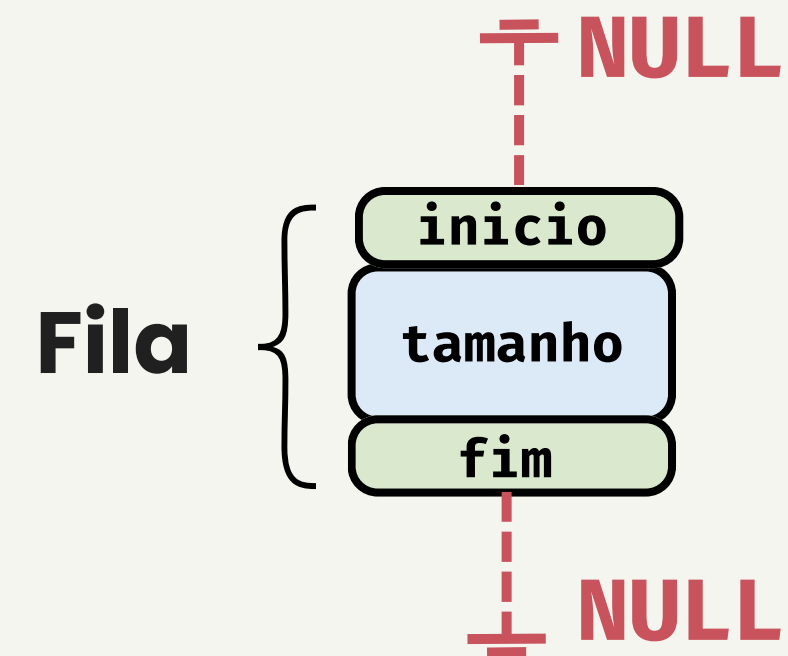
```
Fila* criarFila() {  
    Fila* fila = (Fila*)malloc(sizeof(Fila));  
    fila->inicio = NULL;  
    fila->fim = NULL;  
    fila->tamanho = 0;  
    return fila;  
}
```



FILAS

Criando uma **struct Fila**

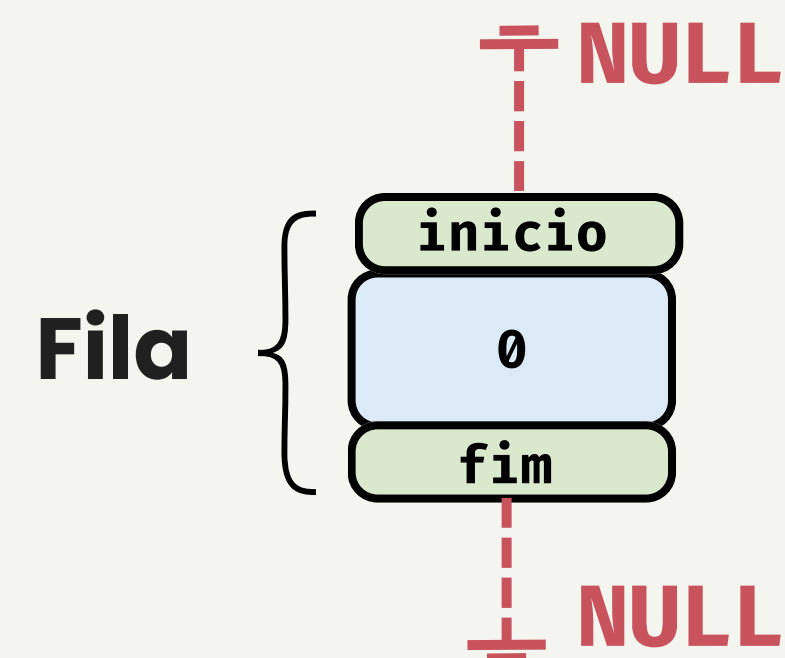
```
Fila* criarFila() {  
    Fila* fila = (Fila*)malloc(sizeof(Fila));  
    fila->inicio = NULL;  
    fila->fim = NULL;  
    fila->tamanho = 0;  
    return fila;  
}
```



FILAS

Criando uma **struct Fila**

```
Fila* criarFila() {  
    Fila* fila = (Fila*)malloc(sizeof(Fila));  
    fila->inicio = NULL;  
    fila->fim = NULL;  
    fila->tamanho = 0;  
    return fila;  
}
```



FILAS

Criando uma **struct Nó**

```
static NoFila* criarNo(int valor) {  
    NoFila* novoNo = (NoFila*)malloc(sizeof(NoFila));  
    if(!novoNo){  
        printf("Erro ao alocar memória para o nó da fila.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```

FILAS

Criando uma **struct** Nó

Indica que a função é uma implementação interna e não faz parte da interface pública do módulo.

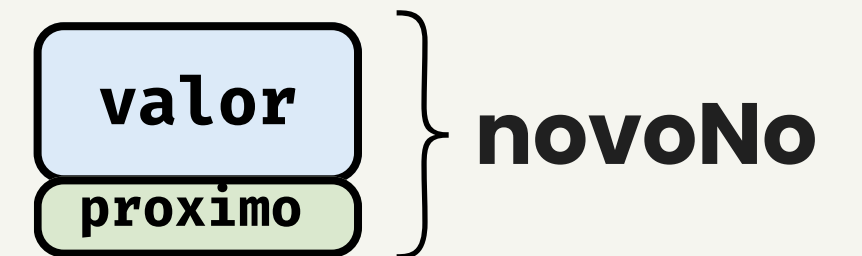
Garante que ela seja usada apenas no arquivo em que foi criada

```
static NoFila* criarNo(int valor) {  
    NoFila* novoNo = (NoFila*)malloc(sizeof(NoFila));  
    if(!novoNo){  
        printf("Erro ao alocar memória para o nó da fila.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```

FILAS

Criando uma **struct Nó**

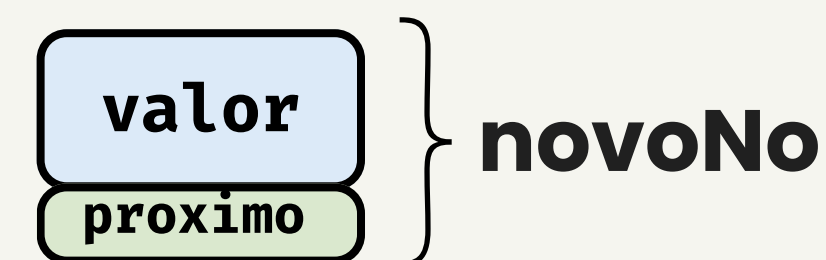
```
static NoFila* criarNo(int valor) {  
    NoFila* novoNo = (NoFila*)malloc(sizeof(NoFila));  
    if(!novoNo){  
        printf("Erro ao alocar memória para o nó da fila.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```



FILAS

Criando uma **struct Nó**

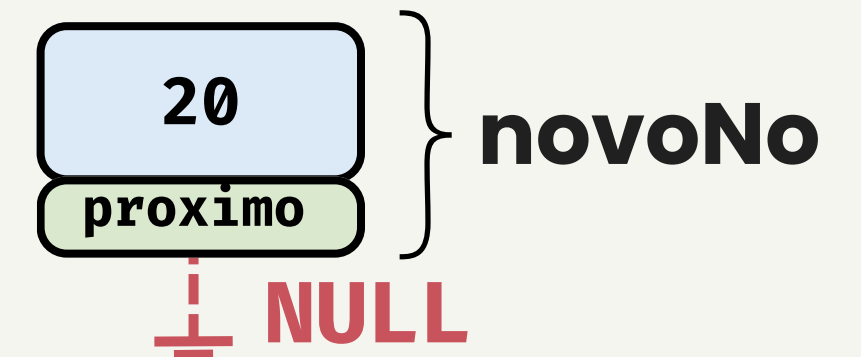
```
static NoFila* criarNo(int valor) {  
    NoFila* novoNo = (NoFila*)malloc(sizeof(NoFila));  
    if(!novoNo){  
        printf("Erro ao alocar memória para o nó da fila.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```



FILAS

Criando uma **struct Nó**

```
static NoFila* criarNo(int valor) {  
    NoFila* novoNo = (NoFila*)malloc(sizeof(NoFila));  
    if(!novoNo){  
        printf("Erro ao alocar memória para o nó da fila.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```



FILAS

- Verificando se a **fila está vazia**

```
int filaVazia(const Fila* fila) {  
    return (fila->tamanho == 0);  
}
```

- **int** → 4 bytes
- **bool** → 1 byte

```
bool filaVazia(const Fila* fila) {  
    return (fila->tamanho == 0);  
}
```

```
#include <stdbool.h>
```

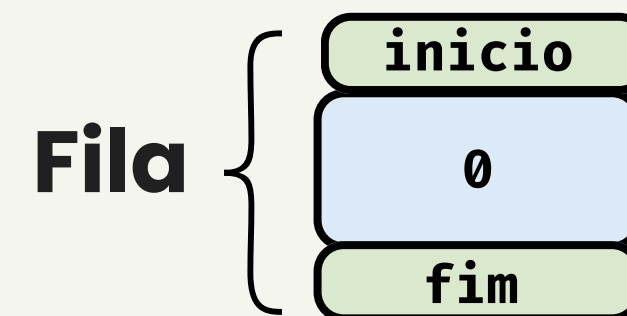
FILAS

- **Enfileirando** (Enqueue)

Inserir um novo elemento no **final** da fila

Caso 1: Fila vazia

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



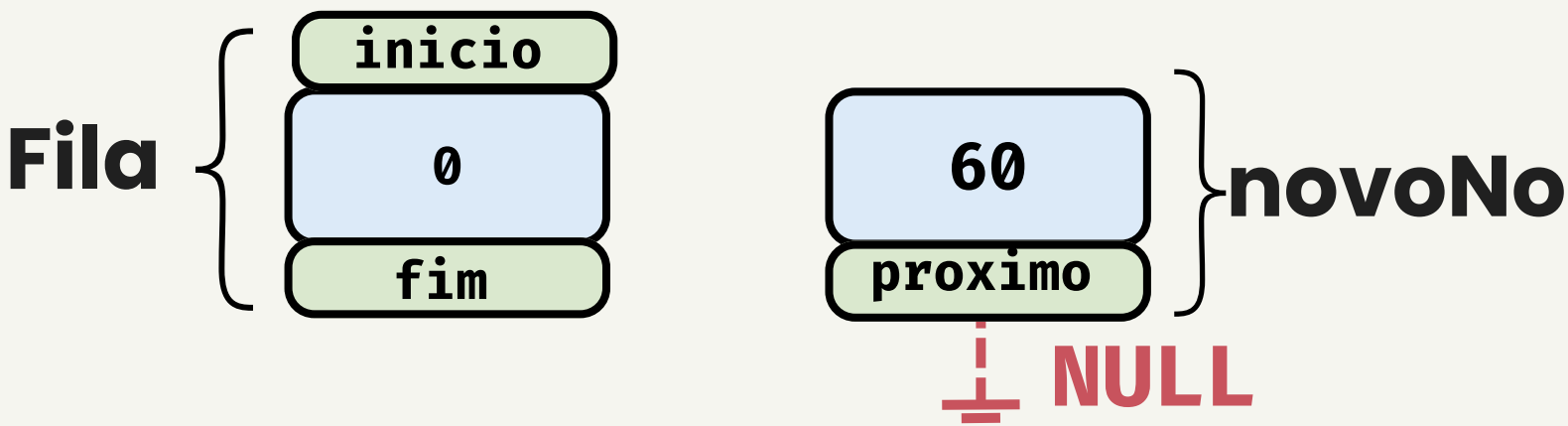
FILAS

- **Enfileirando** (Enqueue)

Inserir um novo elemento no **final** da fila

Caso 1: Fila vazia

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



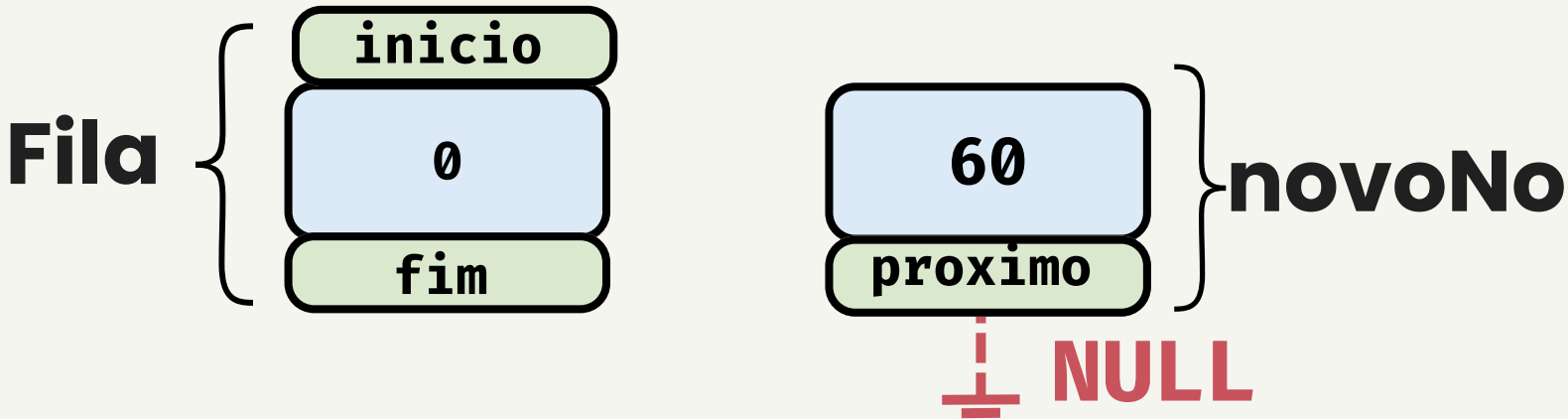
FILAS

- **Enfileirando** (Enqueue)

Inserir um novo elemento no **final** da fila

Caso 1: Fila vazia

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



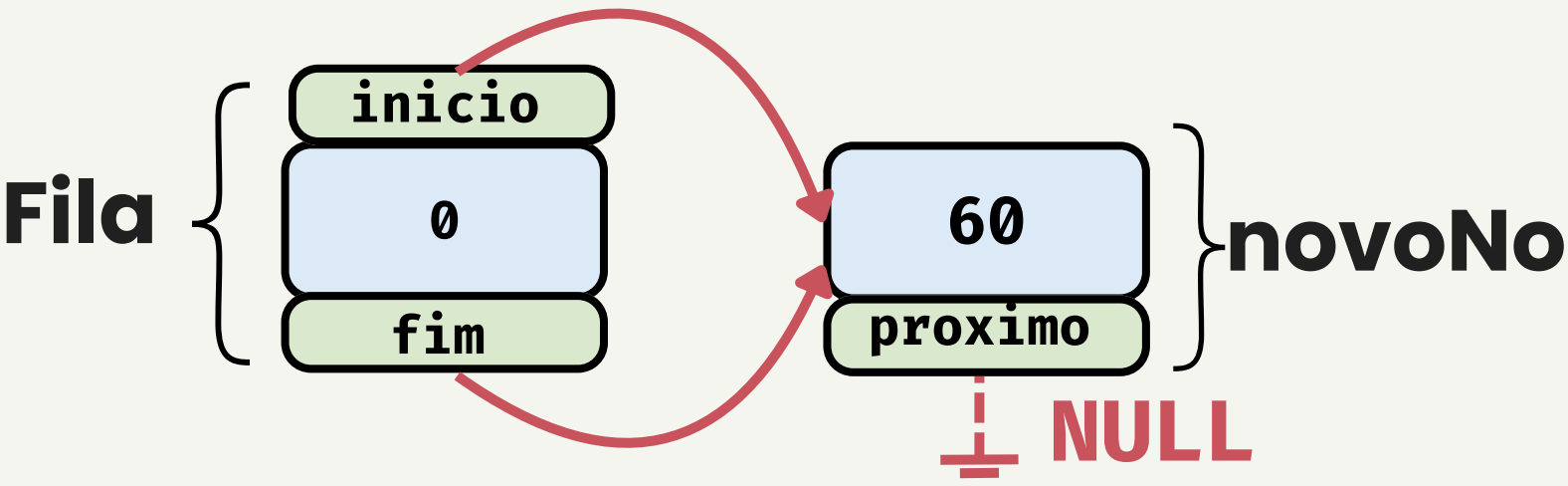
FILAS

- **Enfileirando** (Enqueue)

Inserir um novo elemento no **final** da fila

Caso 1: Fila vazia

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



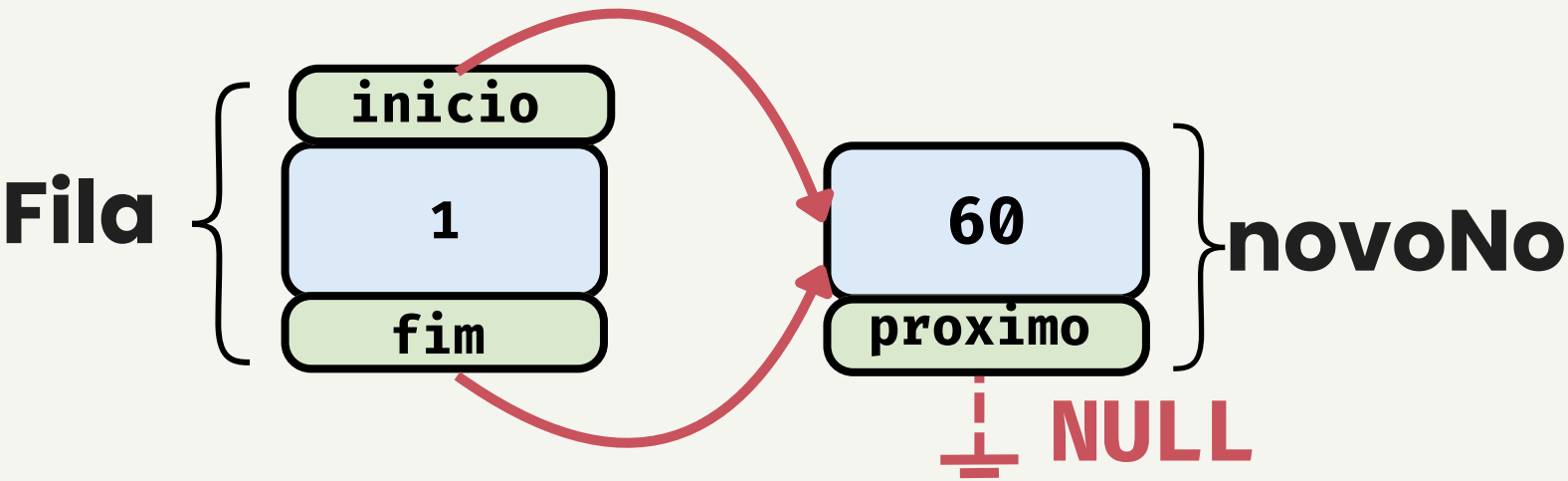
FILAS

- **Enfileirando** (Enqueue)

Inserir um novo elemento no **final** da fila

Caso 1: Fila vazia

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```

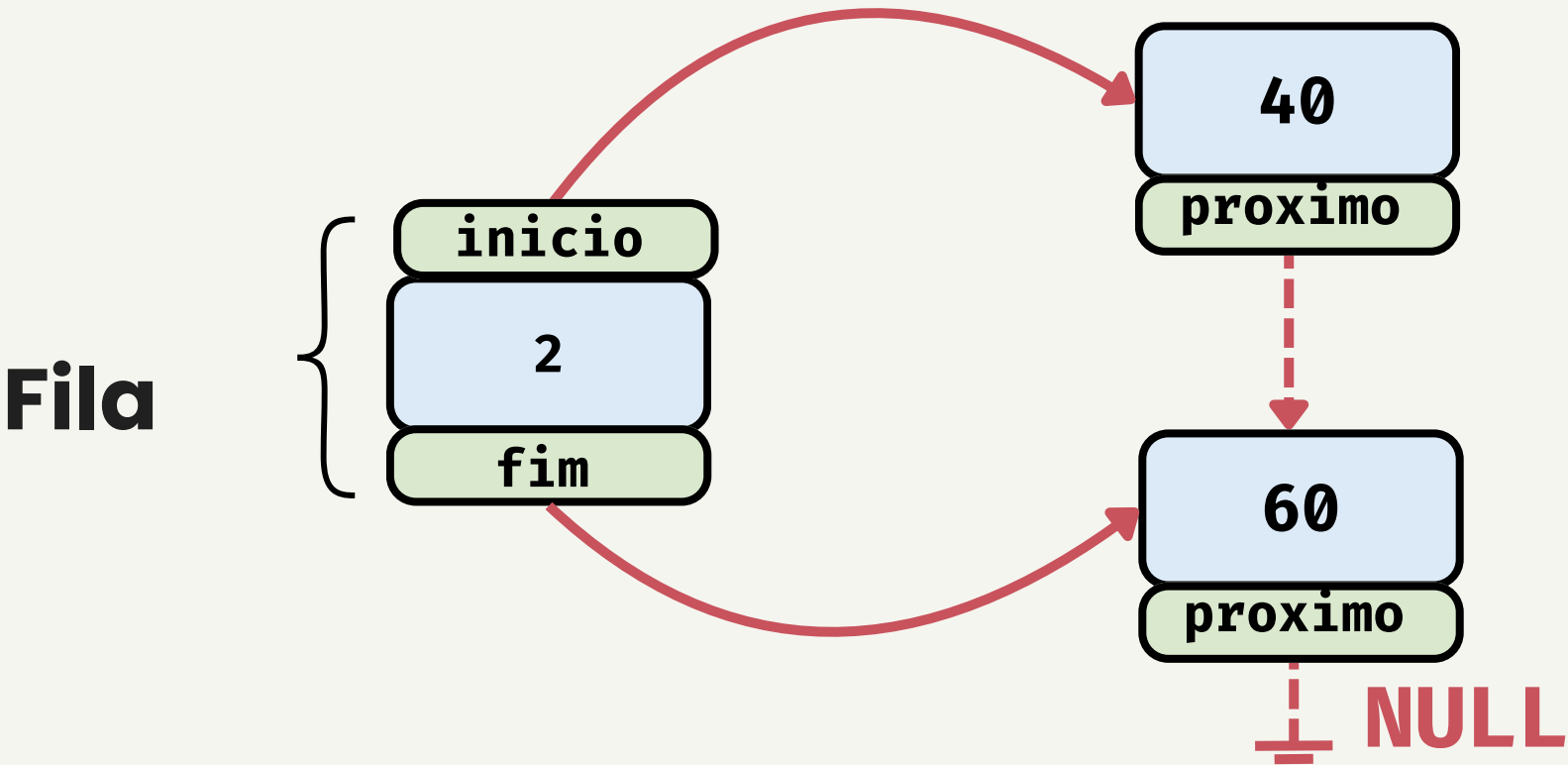


FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



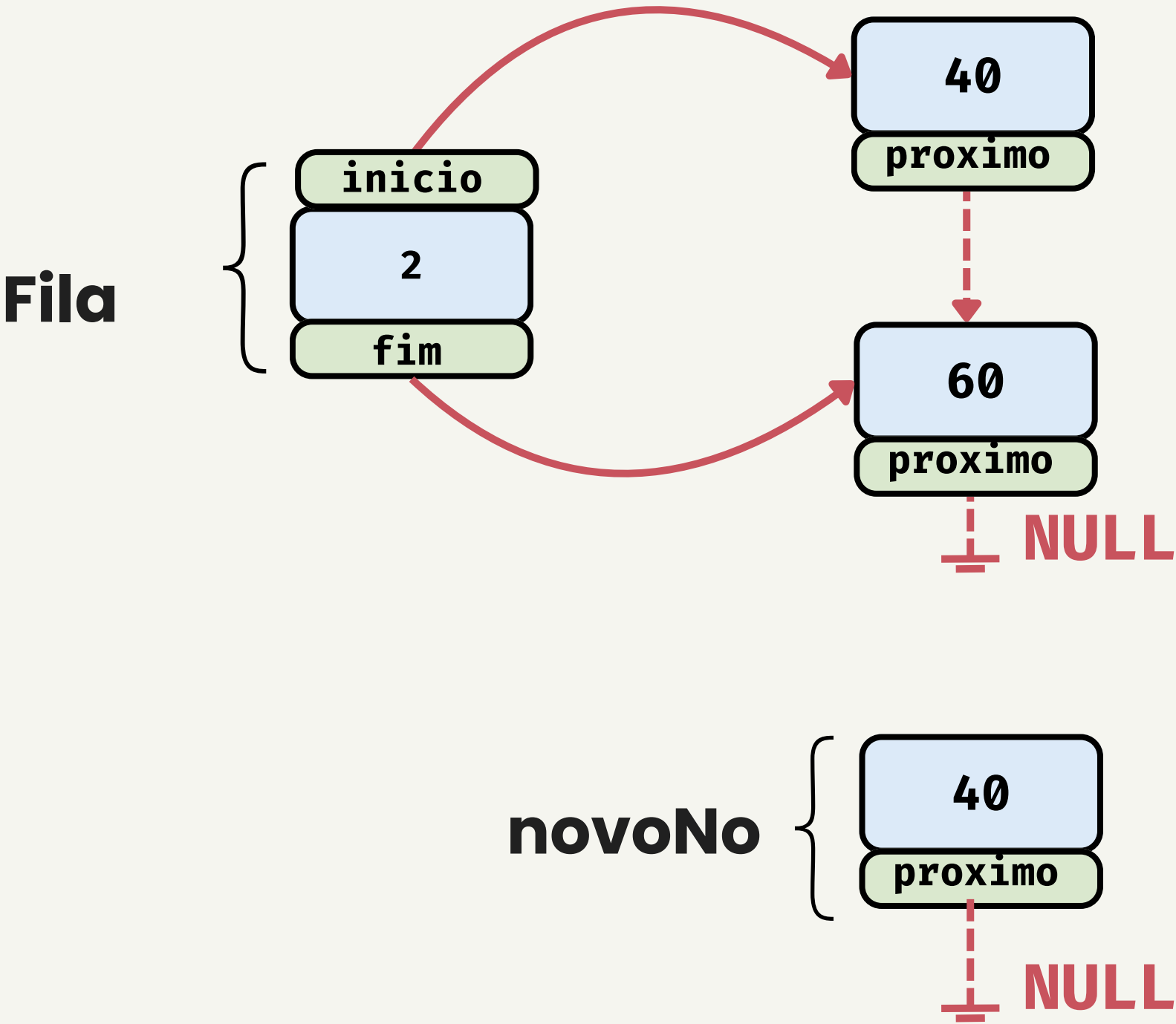
FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```

void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
    
```

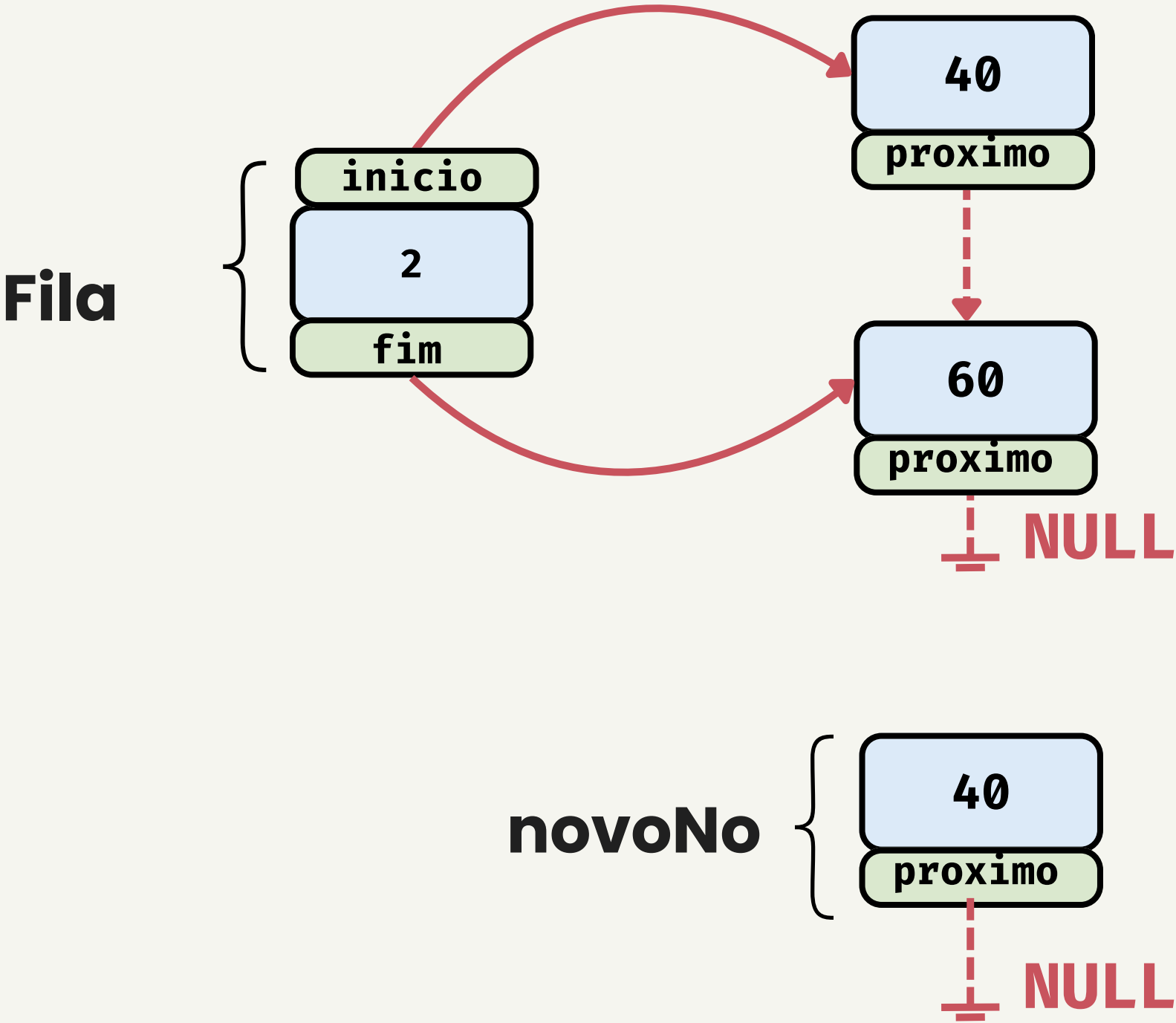


FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```

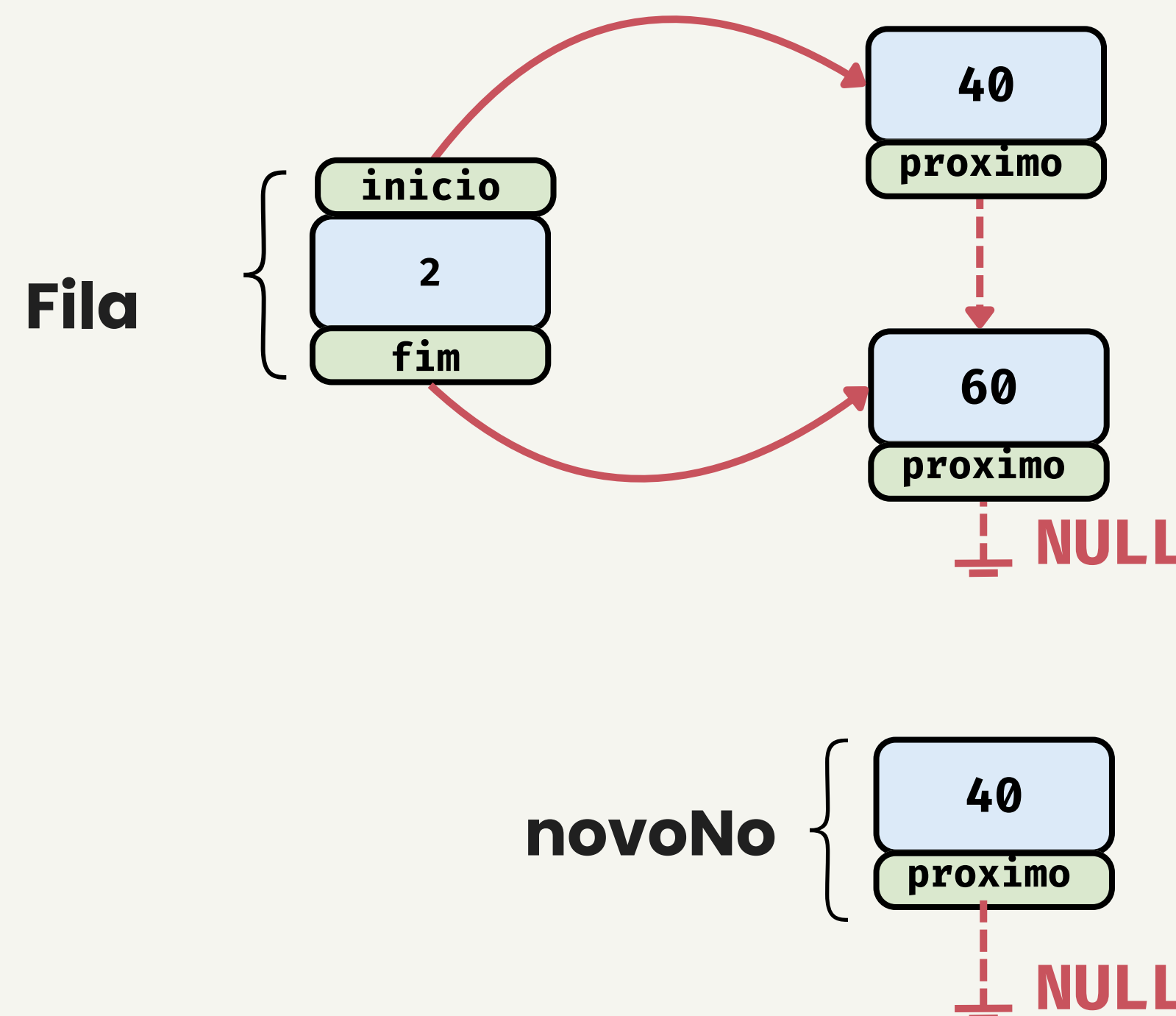


FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```

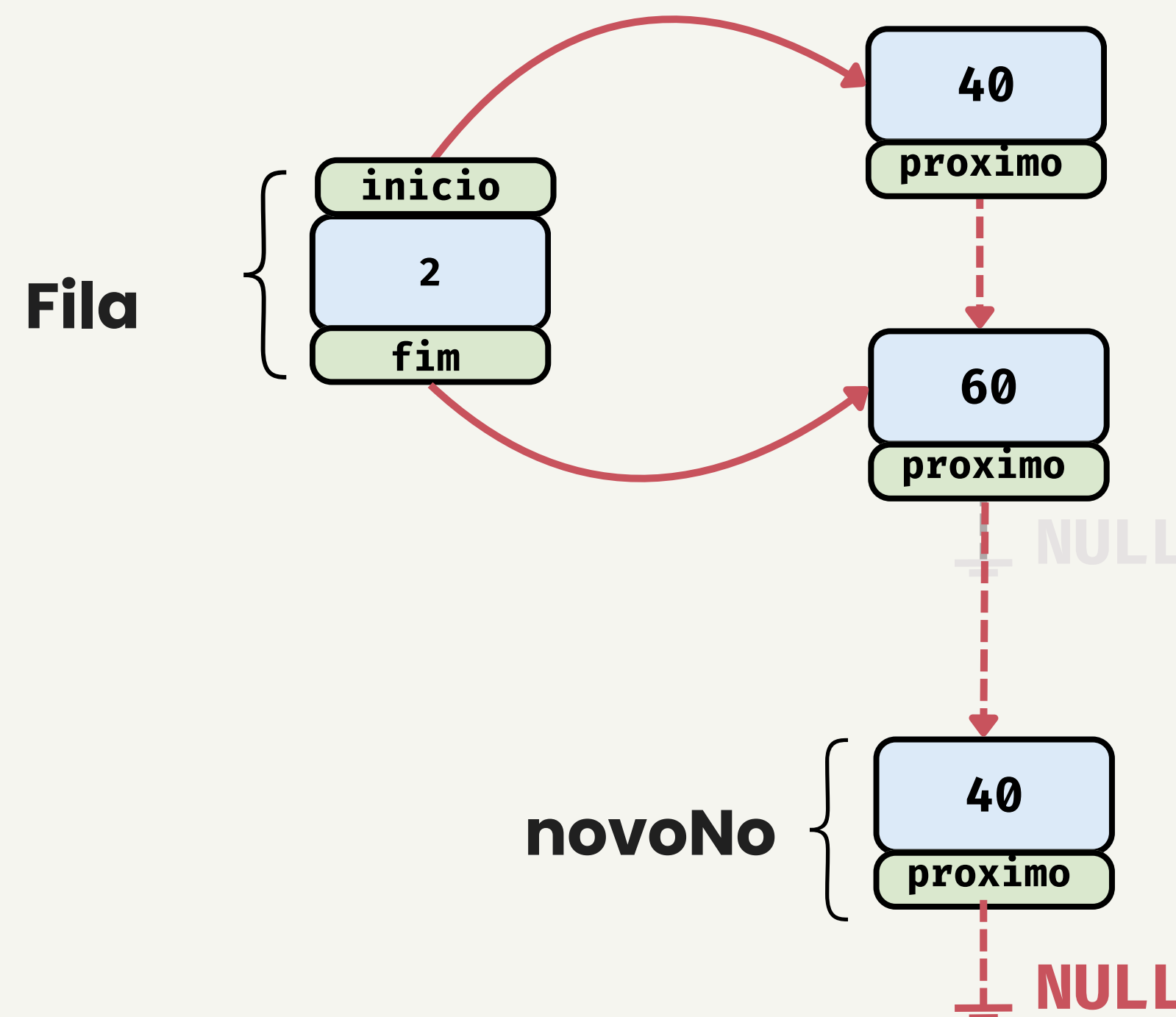


FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



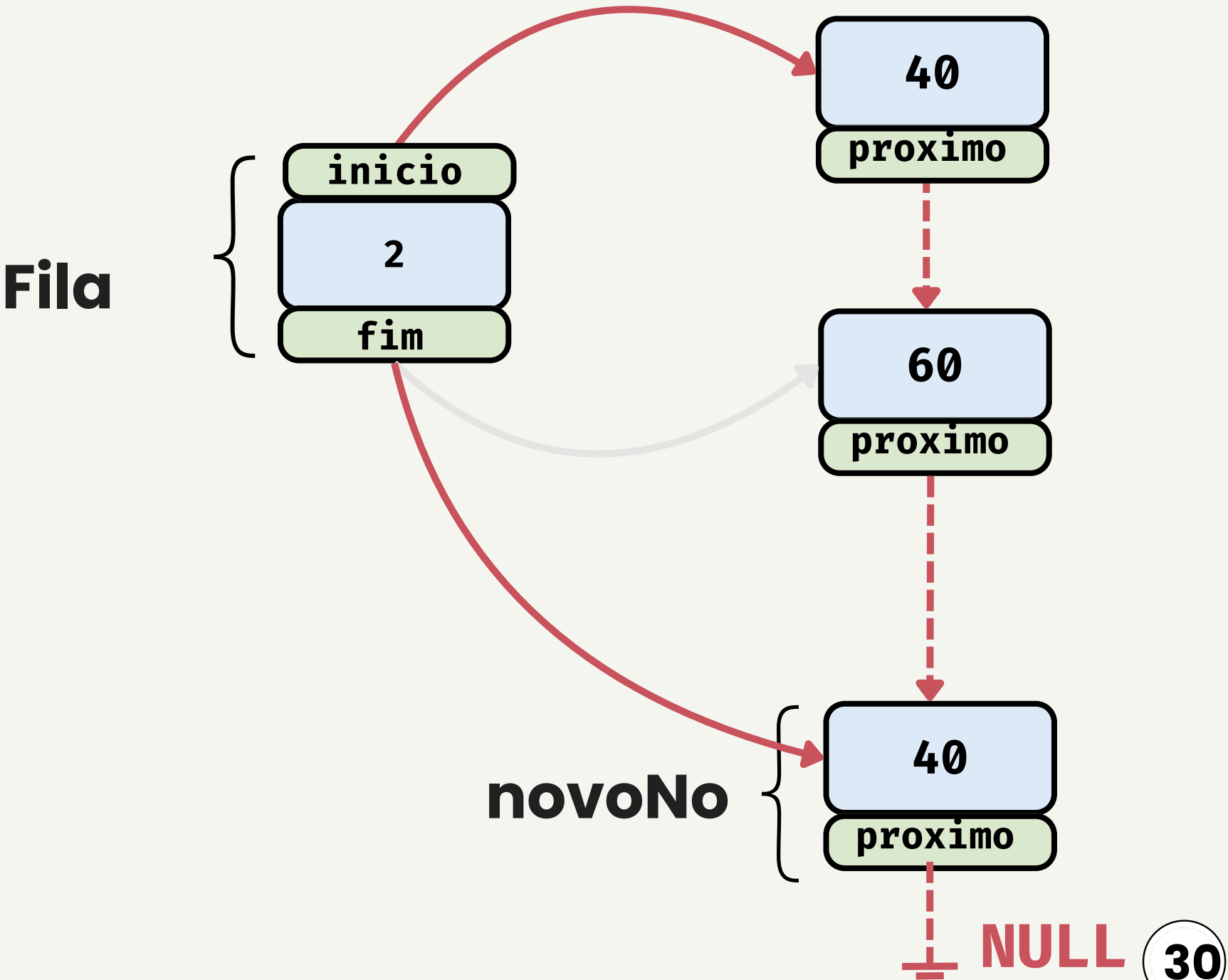
FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```

void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
    
```

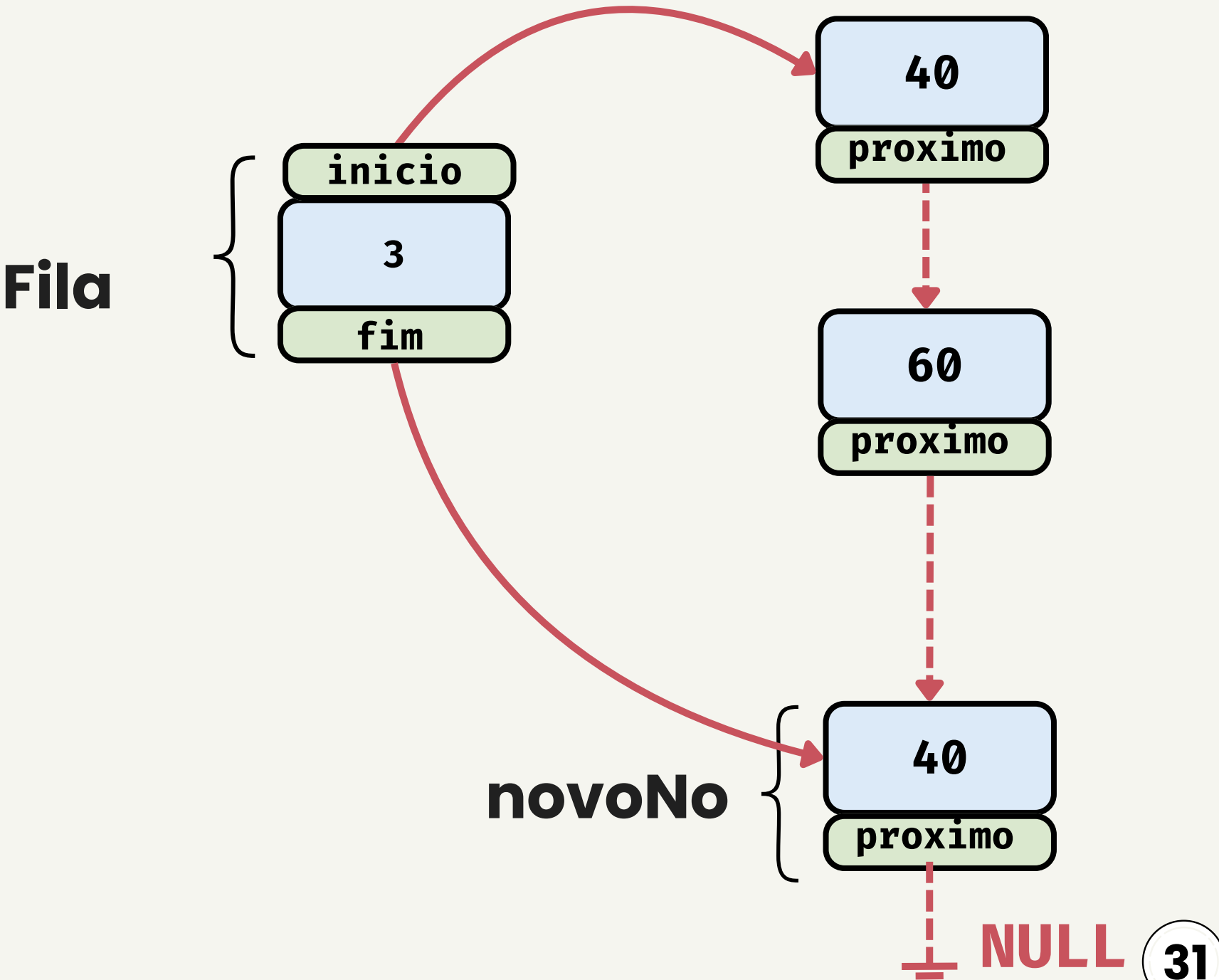


FILAS

- **Enfileirando** (Enqueue)

Caso 2: Fila com elementos

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novoNo = criarNo(valor);
    if (filaVazia(fila)) {
        fila->inicio = novoNo;
        fila->fim = novoNo;
    } else {
        fila->fim->prox = novoNo;
        fila->fim = novoNo;
    }
    fila->tamanho++;
}
```



FILAS

- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

```
void desenfileirar(Fila* fila) {  
    if (filaVazia(fila)) {  
        printf("Fila vazia! Não é possível desenfileirar.\n");  
        return;  
    }  
    NoFila* temp = fila->inicio;  
    fila->inicio = temp->prox;  
    free(temp);  
    fila->tamanho--;  
    if (filaVazia(fila)) {  
        fila->fim = NULL;  
    }  
}
```

FILAS

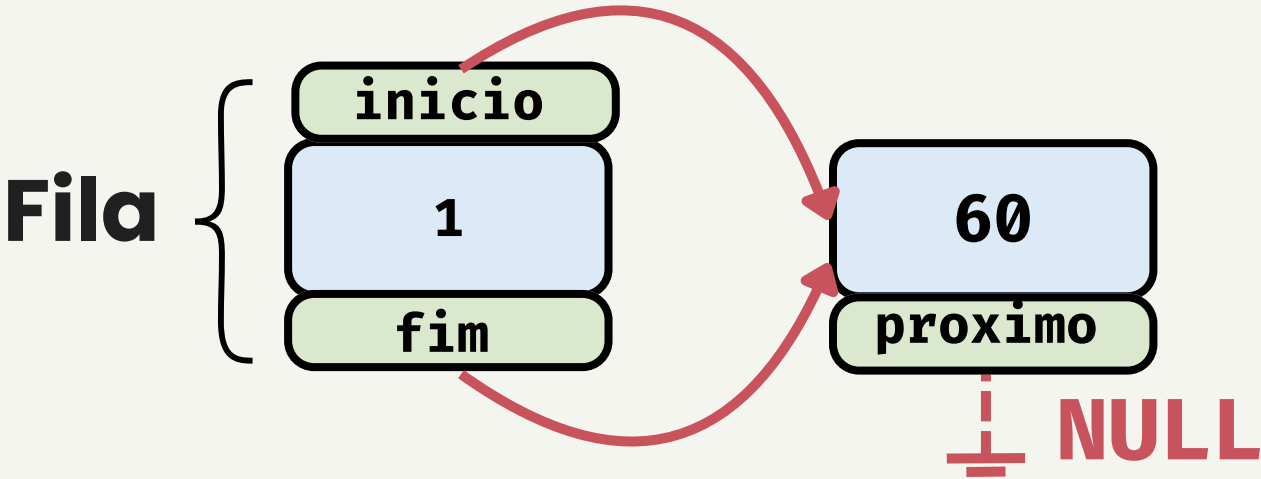
- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```

void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}

```



FILAS

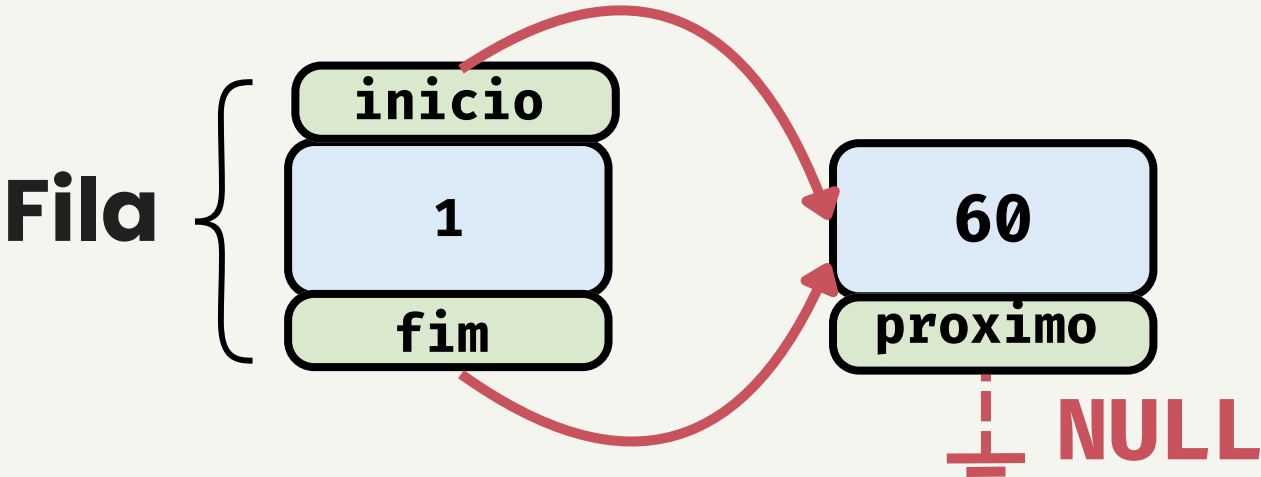
- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```

void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}

```

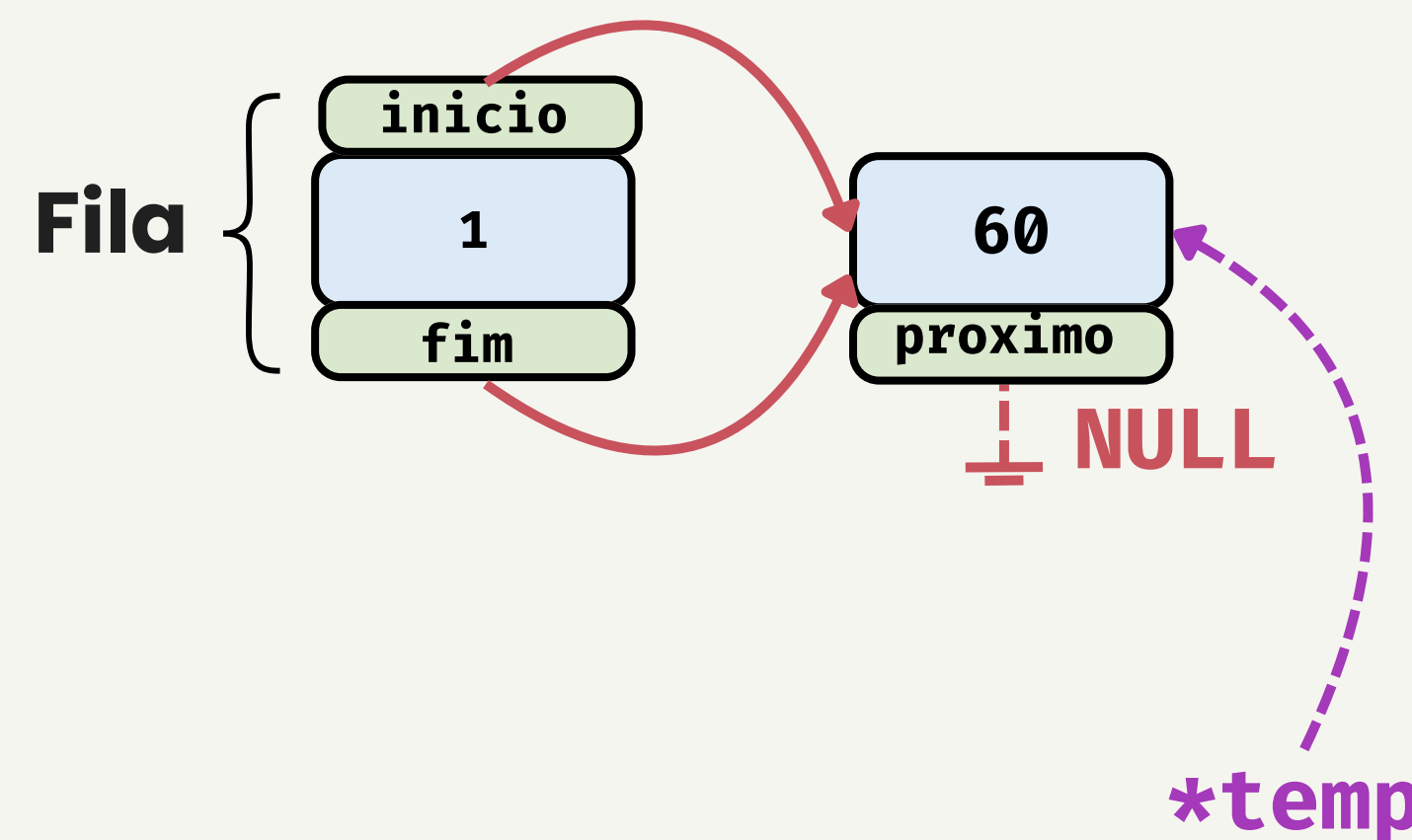


FILAS

- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```



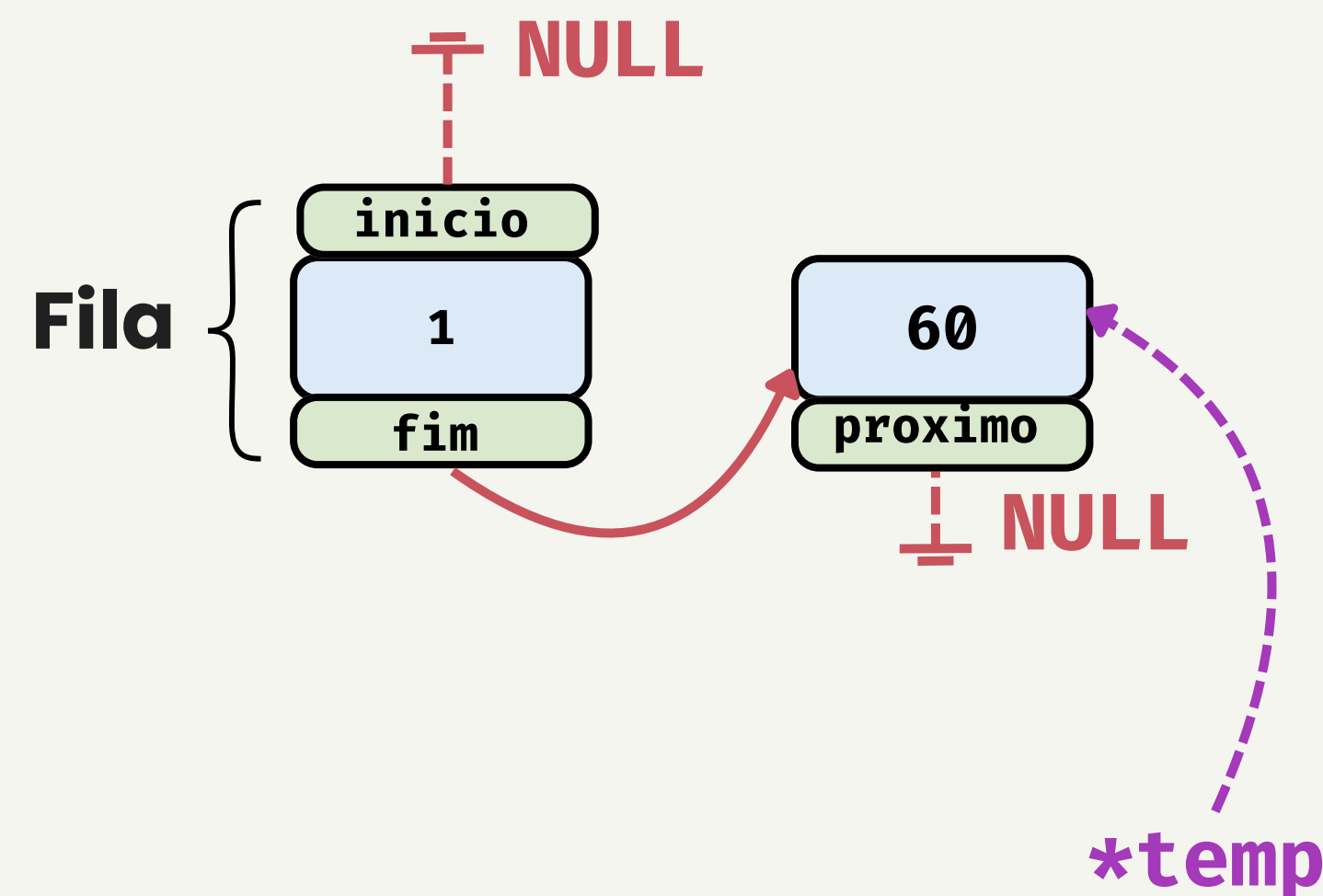
⚠️ Ponteiro auxiliar: Avança com o nó e não perde a referência para remover.

FILAS

- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

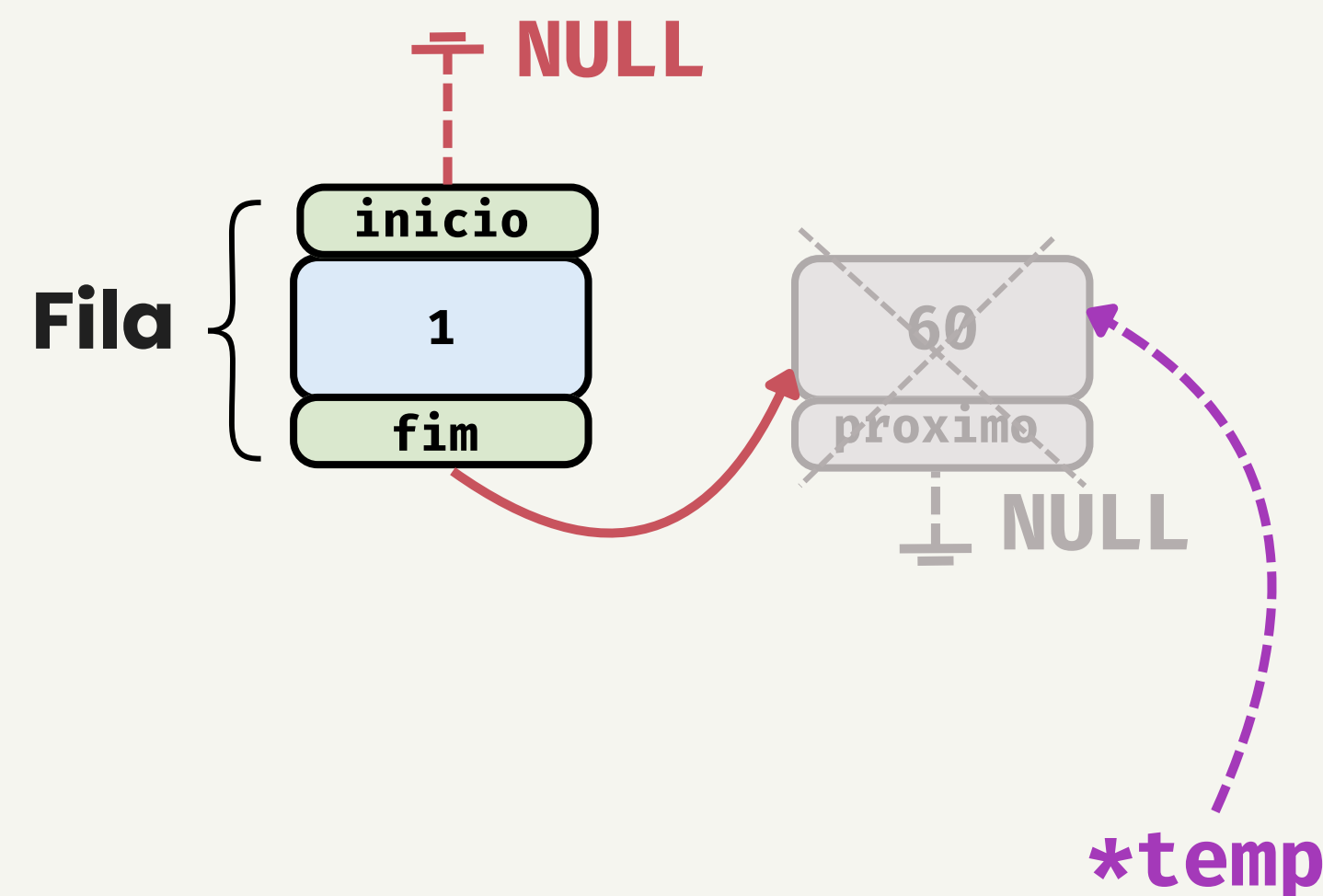


FILAS

- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```



FILAS

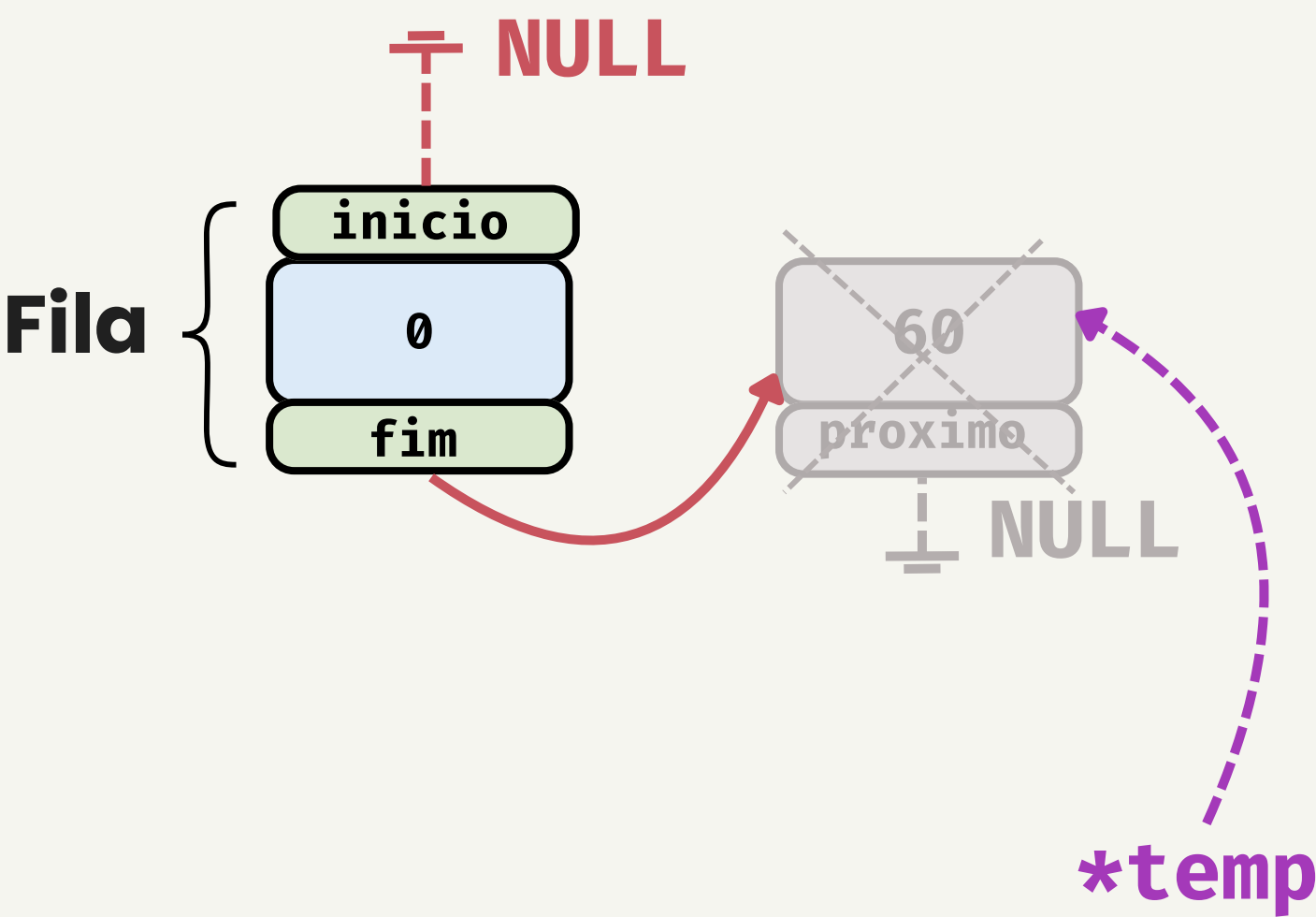
- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```

void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}

```



FILAS

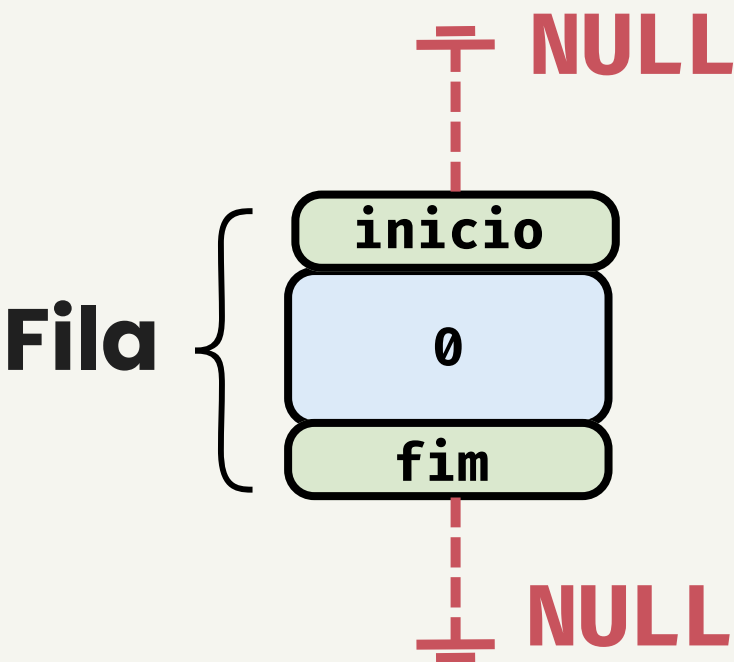
- **Desenfileirando** (Dequeue)
Remover um elemento no **início** da fila

Caso 1: Fila com 1 elemento

```

void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}

```

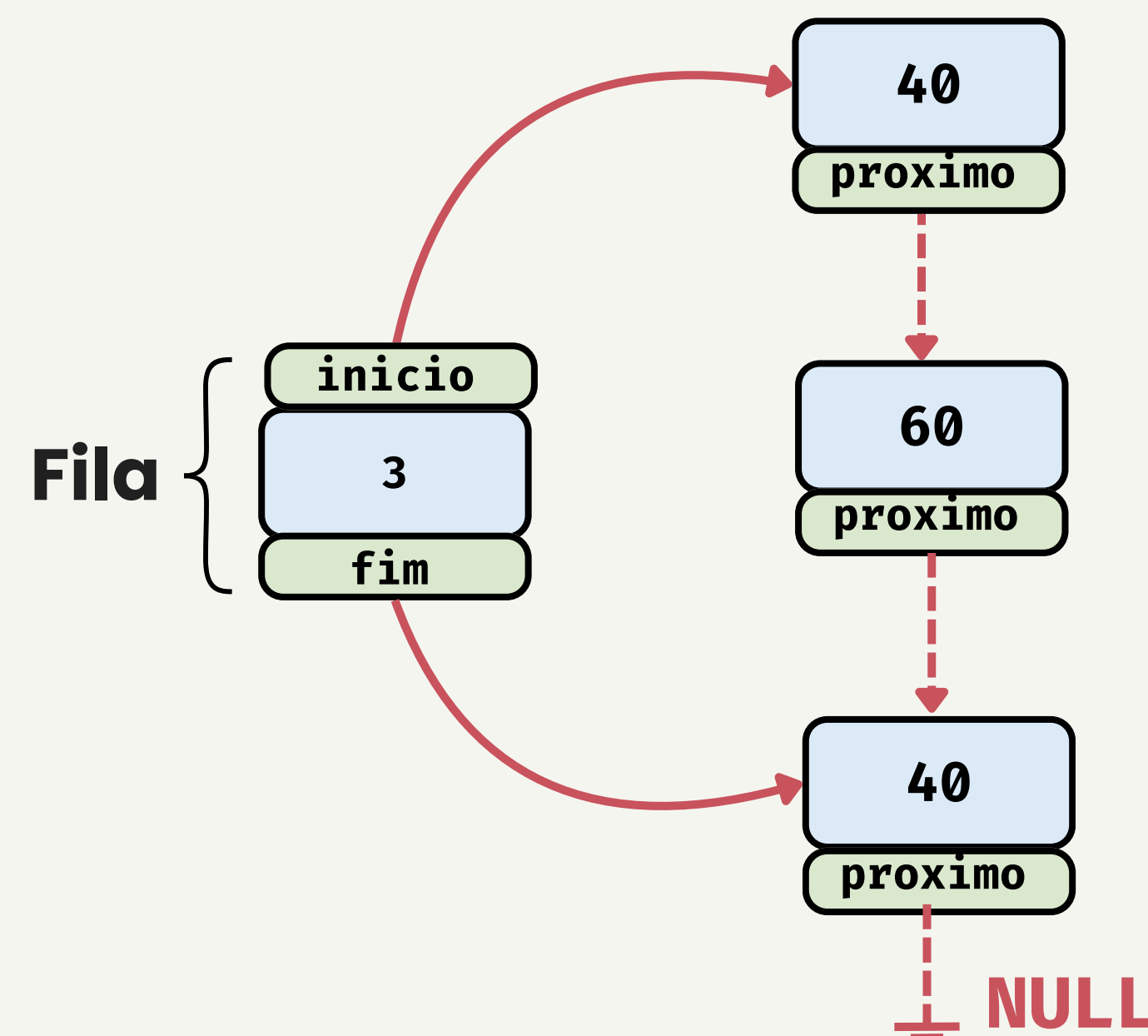


FILAS

- **Desenfileirando** (Dequeue)

Caso 2: Fila com mais de um elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

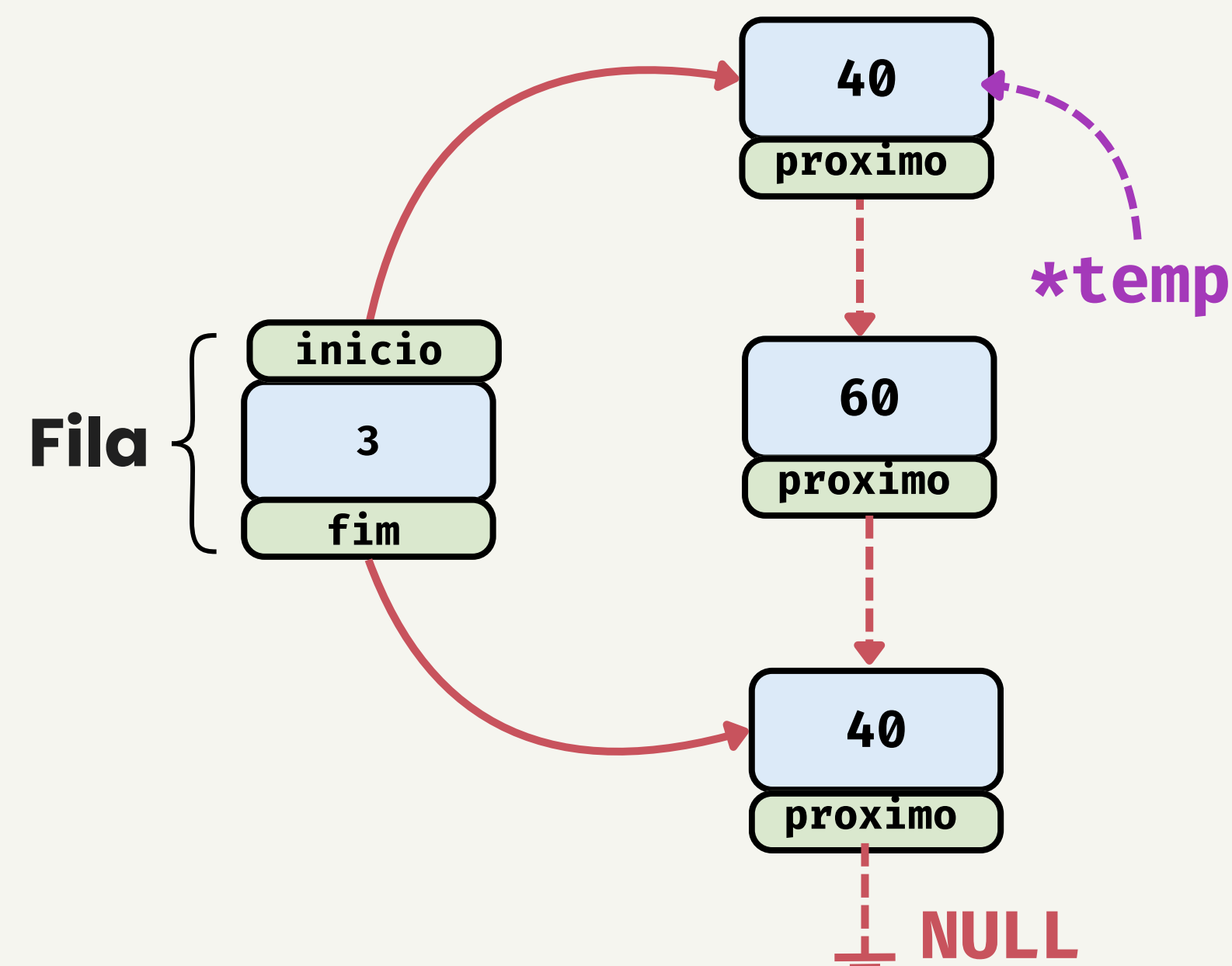


FILAS

- Desenfileirando (Dequeue)

Caso 2: Fila com mais de um elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

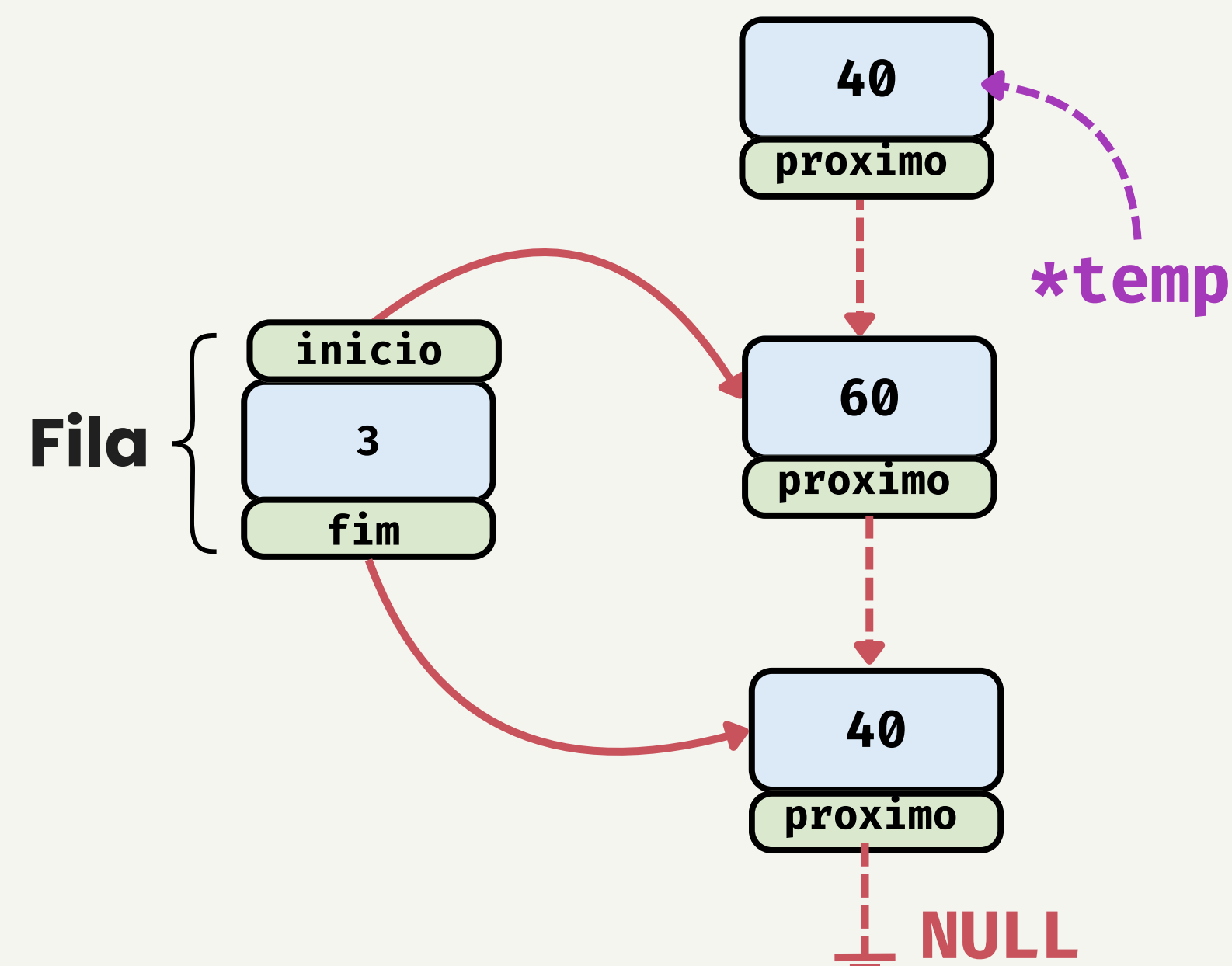


FILAS

- Desenfileirando (Dequeue)

Caso 2: Fila com mais de um elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

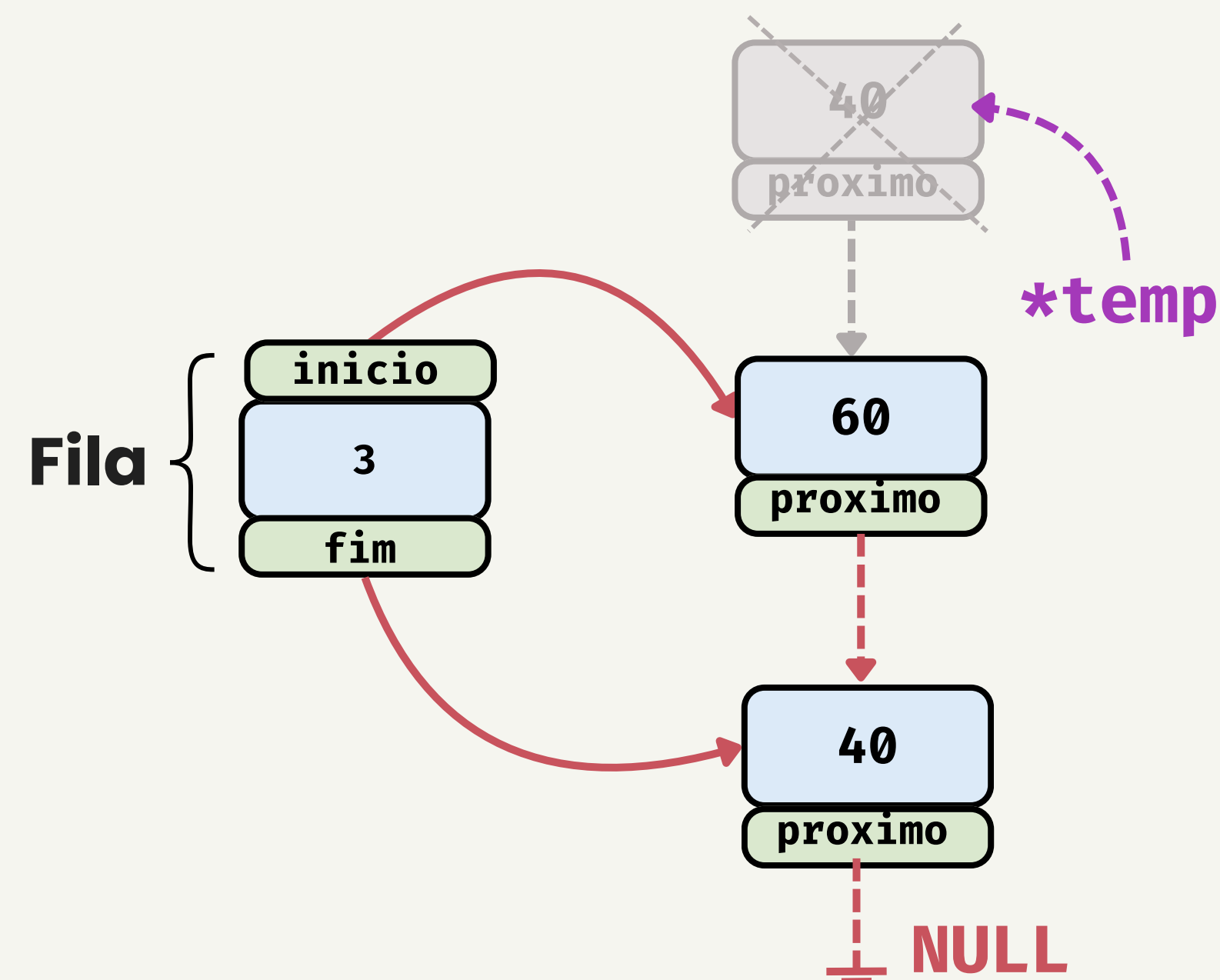


FILAS

- Desenfileirando (Dequeue)

Caso 2: Fila com mais de um elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

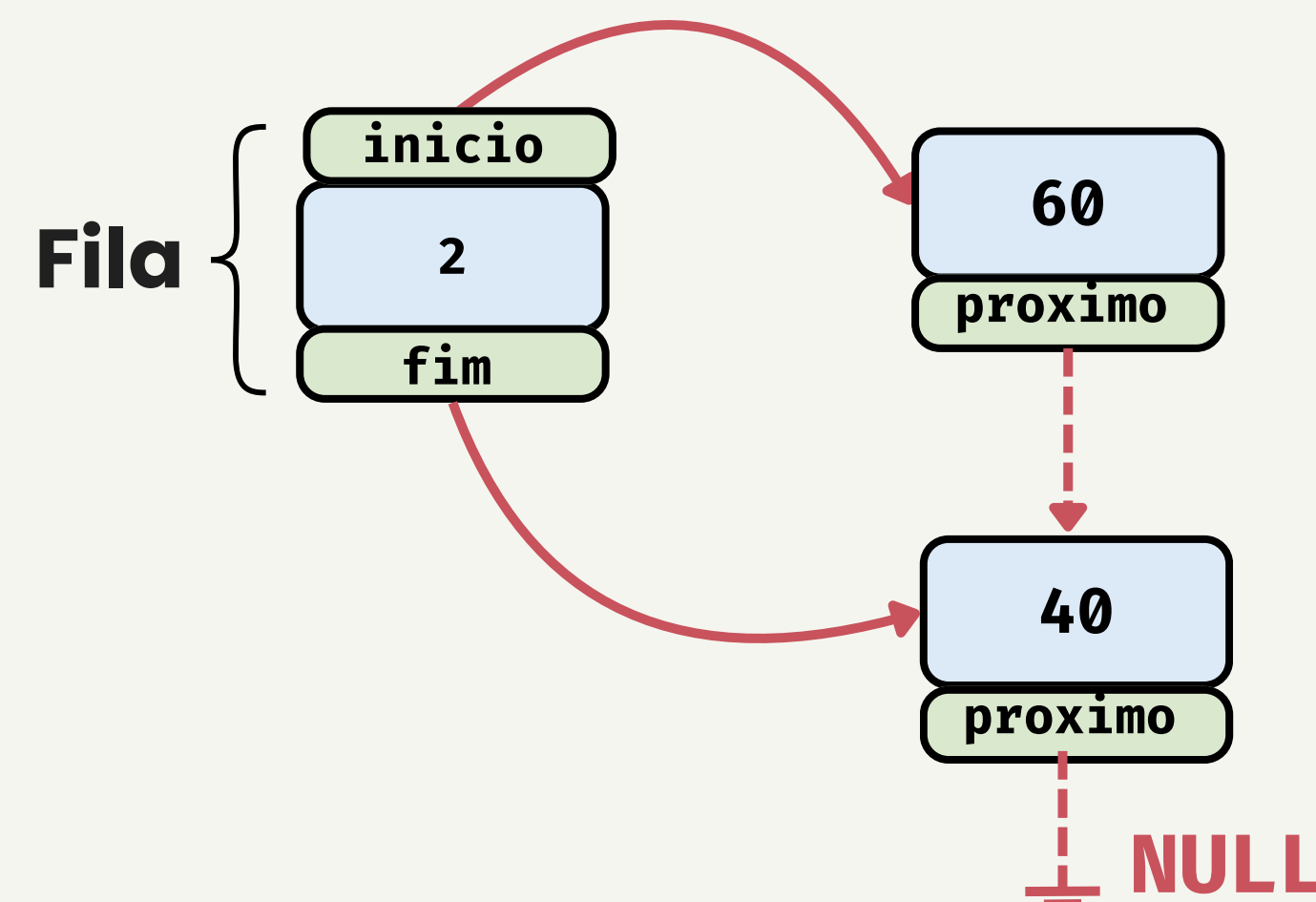


FILAS

- Desenfileirando (Dequeue)

Caso 2: Fila com mais de um elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

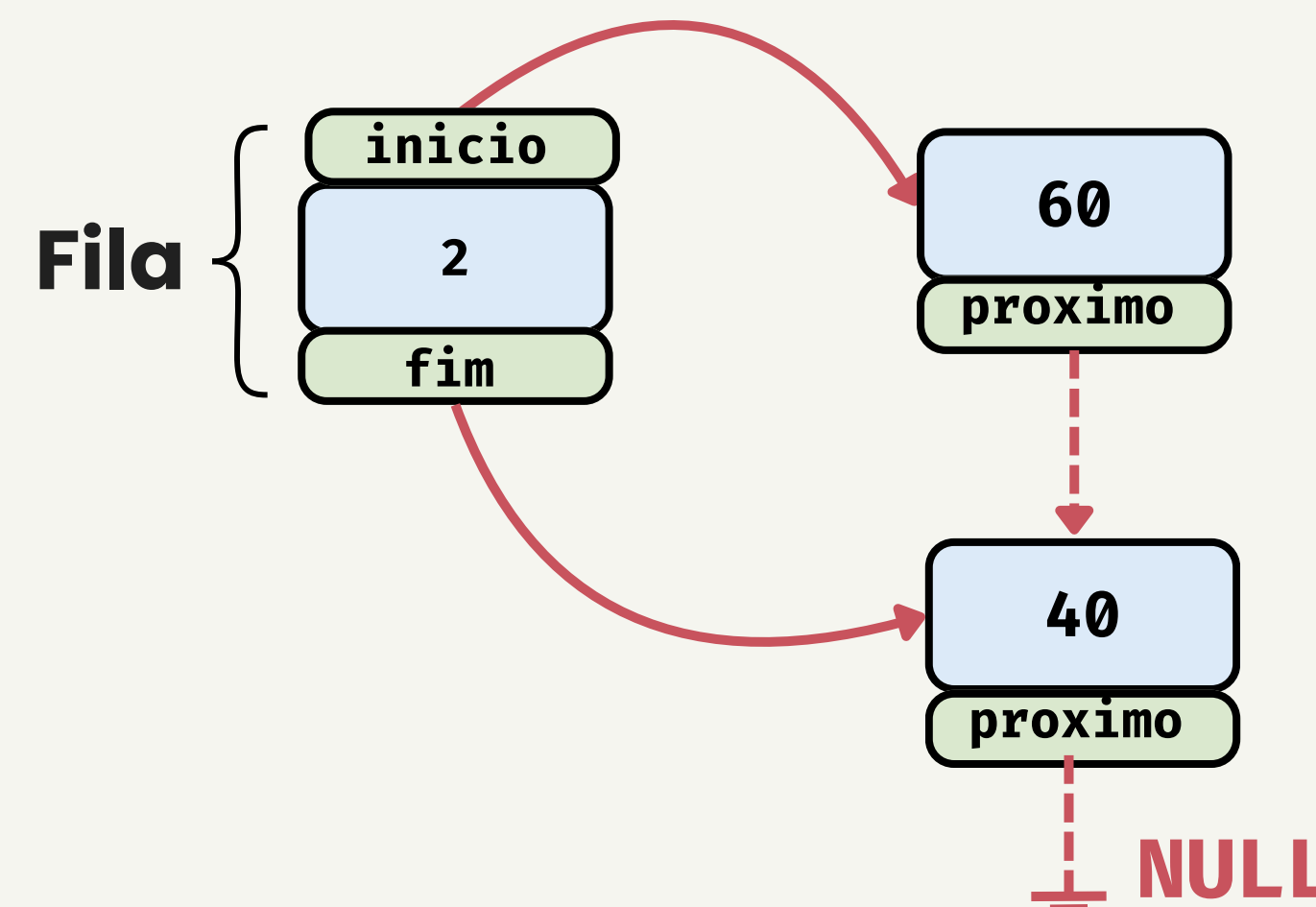


FILAS

- **Desenfileirando** (Dequeue)

Caso 2: Fila com mais de um elemento

```
void desenfileirar(Fila* fila) {
    if (filaVazia(fila)) {
        printf("Fila vazia! Não é possível desenfileirar.\n");
        return;
    }
    NoFila* temp = fila->inicio;
    fila->inicio = temp->prox;
    free(temp);
    fila->tamanho--;
    if (filaVazia(fila)) {
        fila->fim = NULL;
    }
}
```

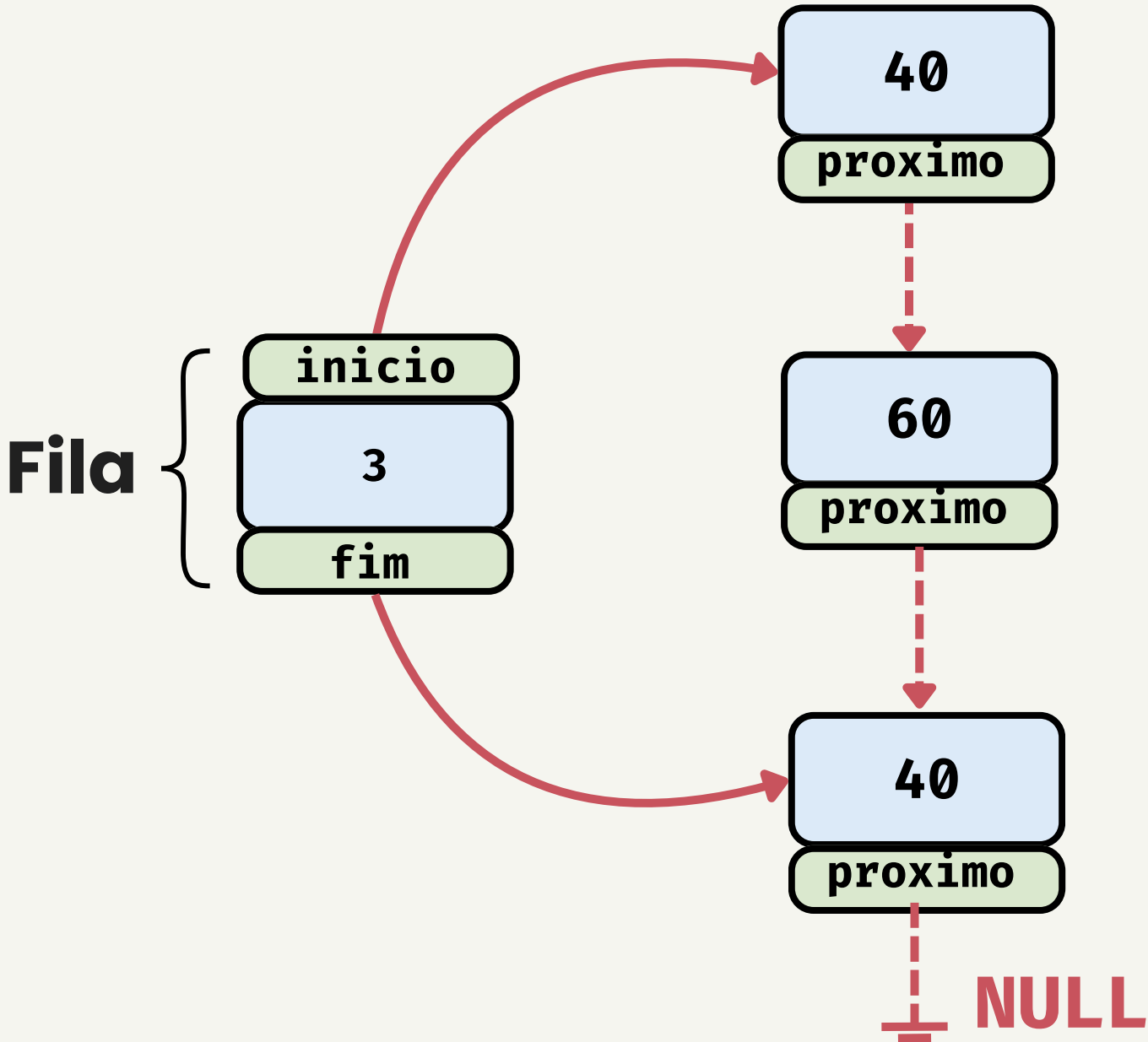


Só atualizo o fim quando a lista fica vazia.
Se a lista não fica vazia, o fim dela não é alterado.

FILAS

- **Limpando** a Fila → liberando os **nós**

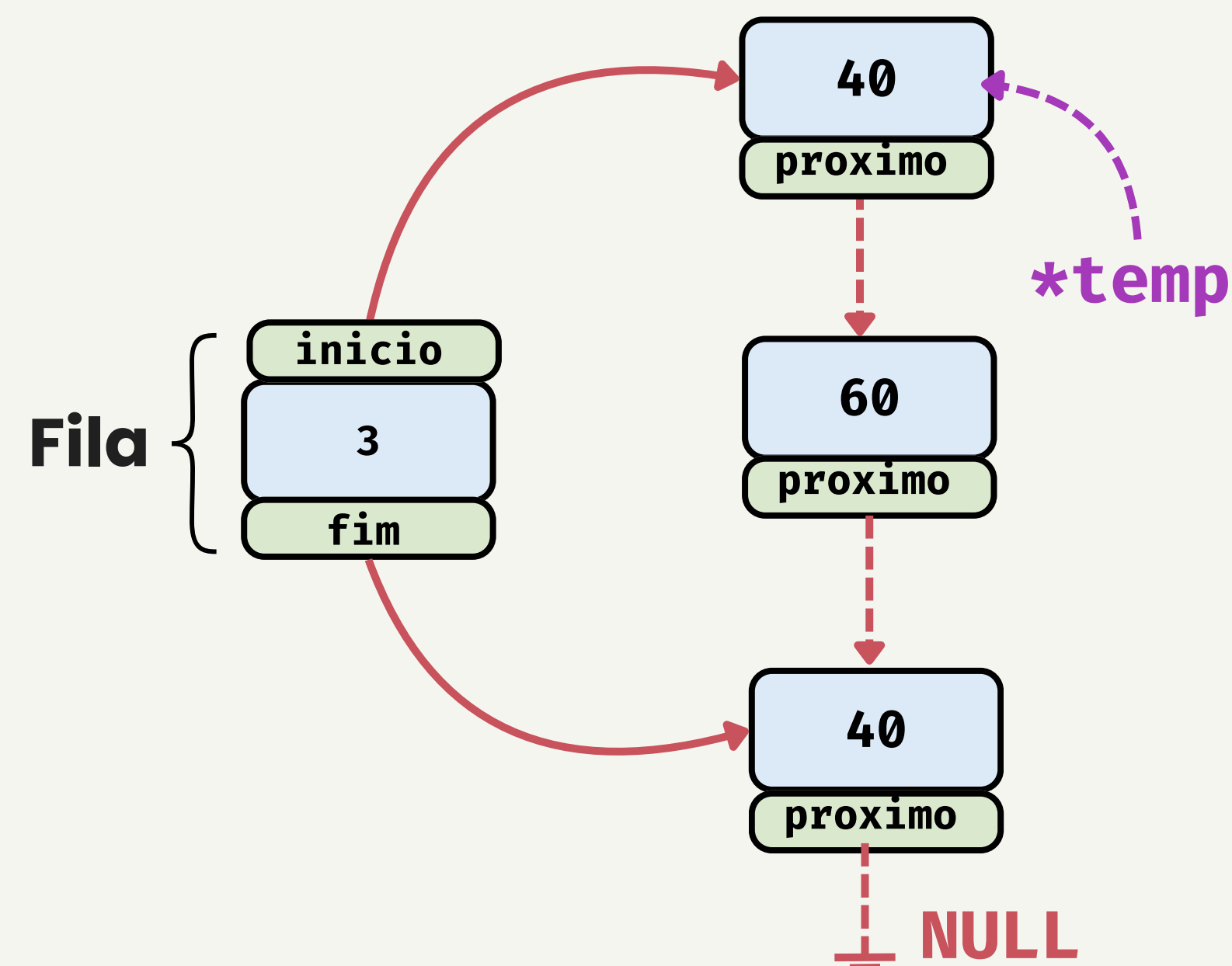
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

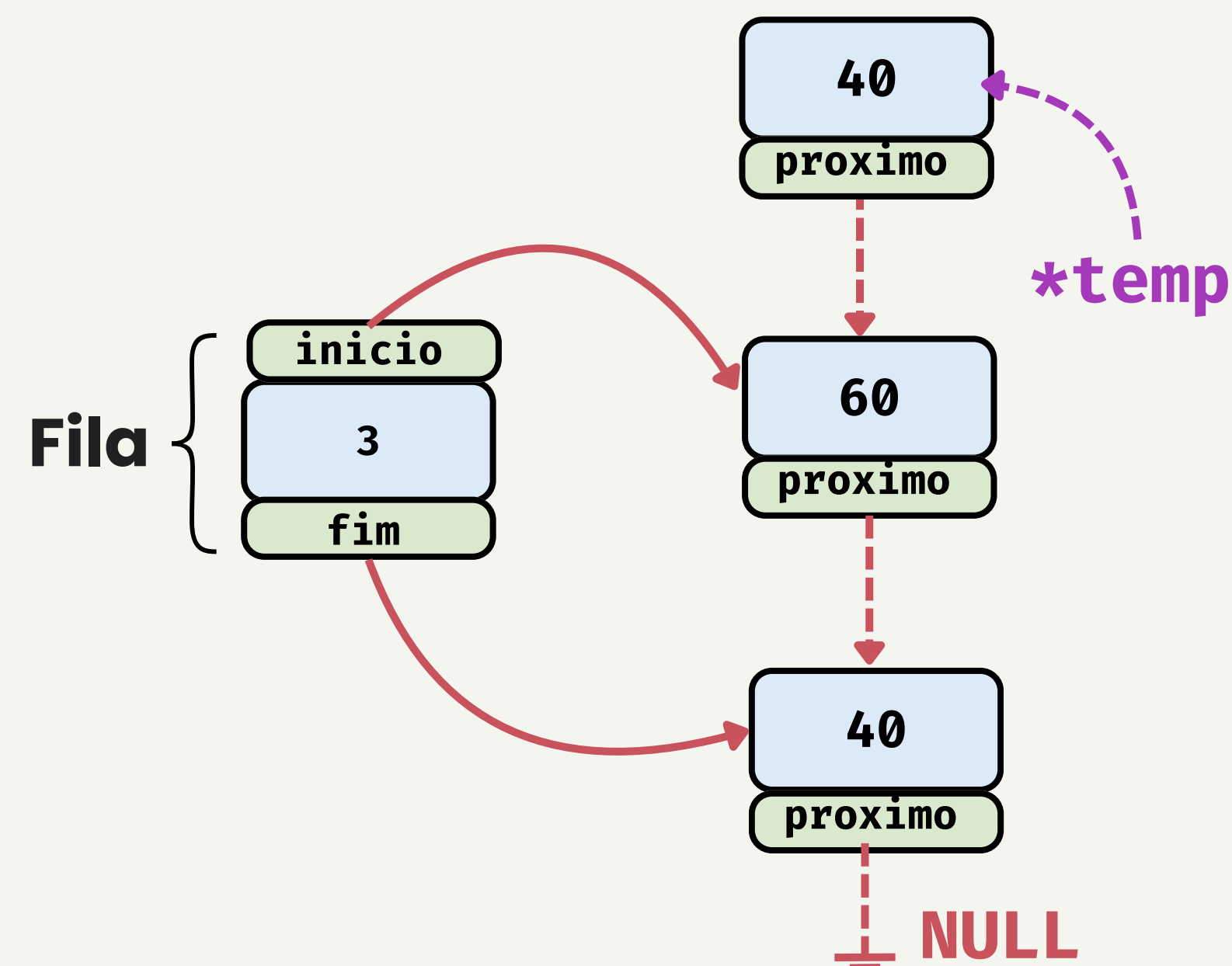
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

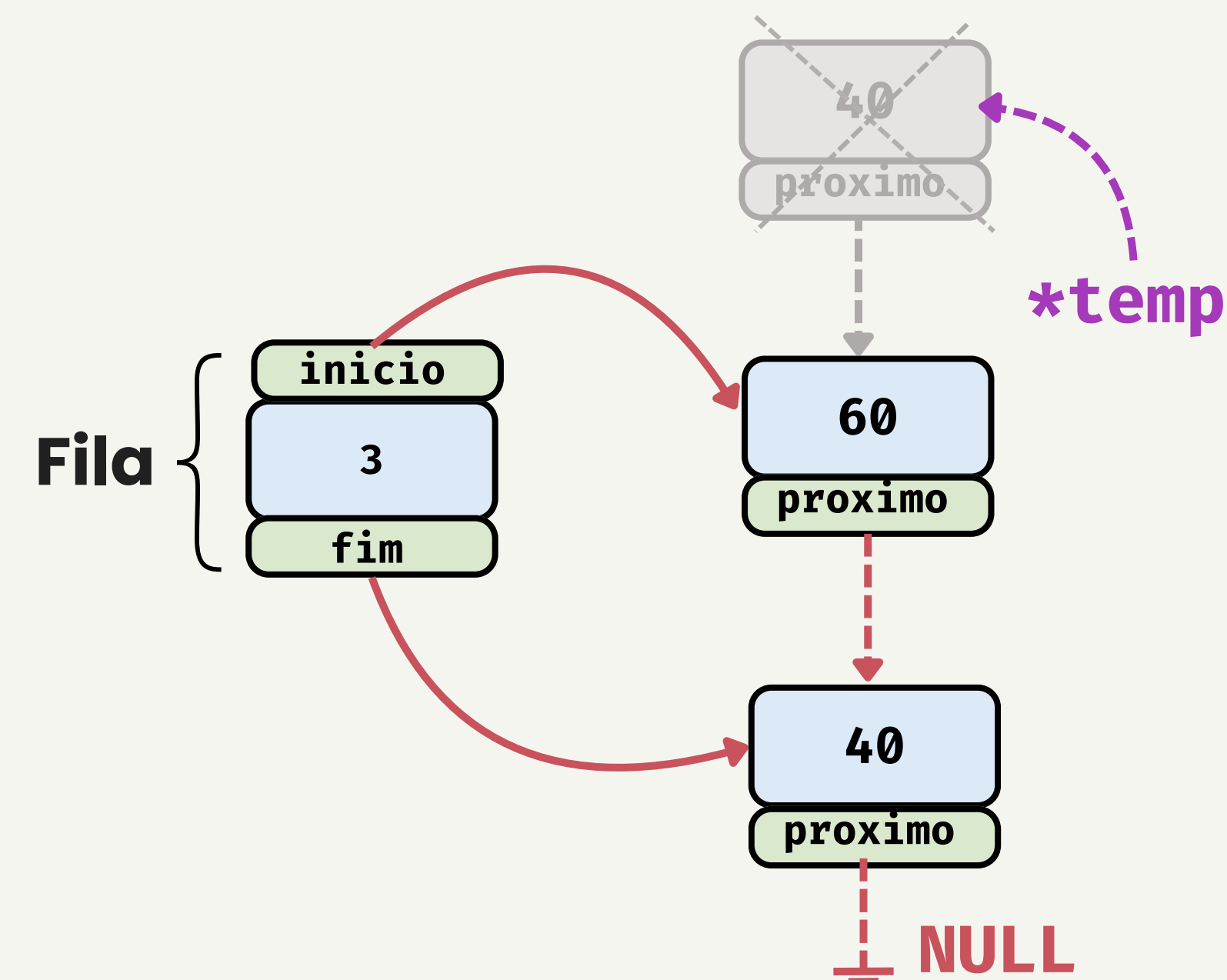
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

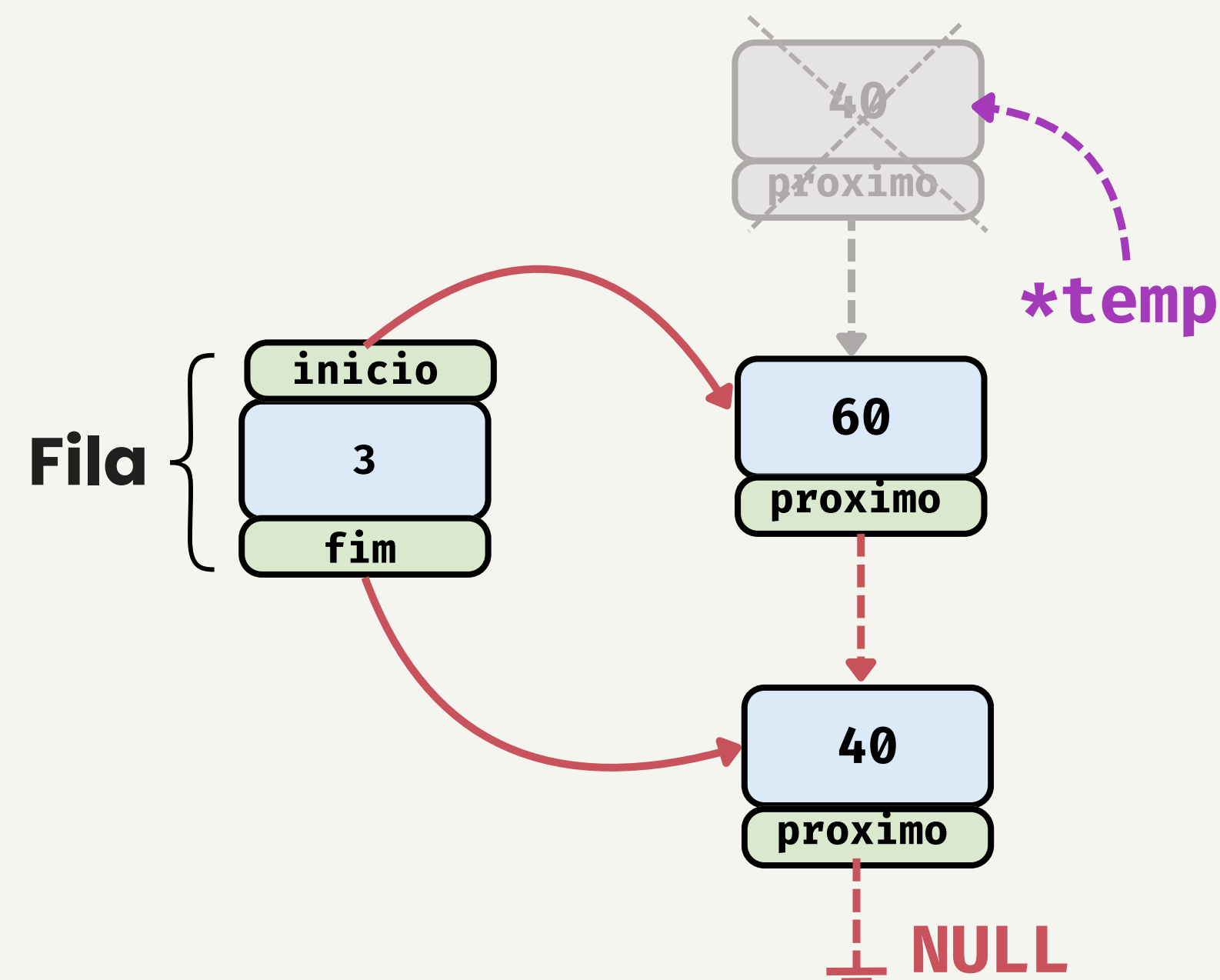
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

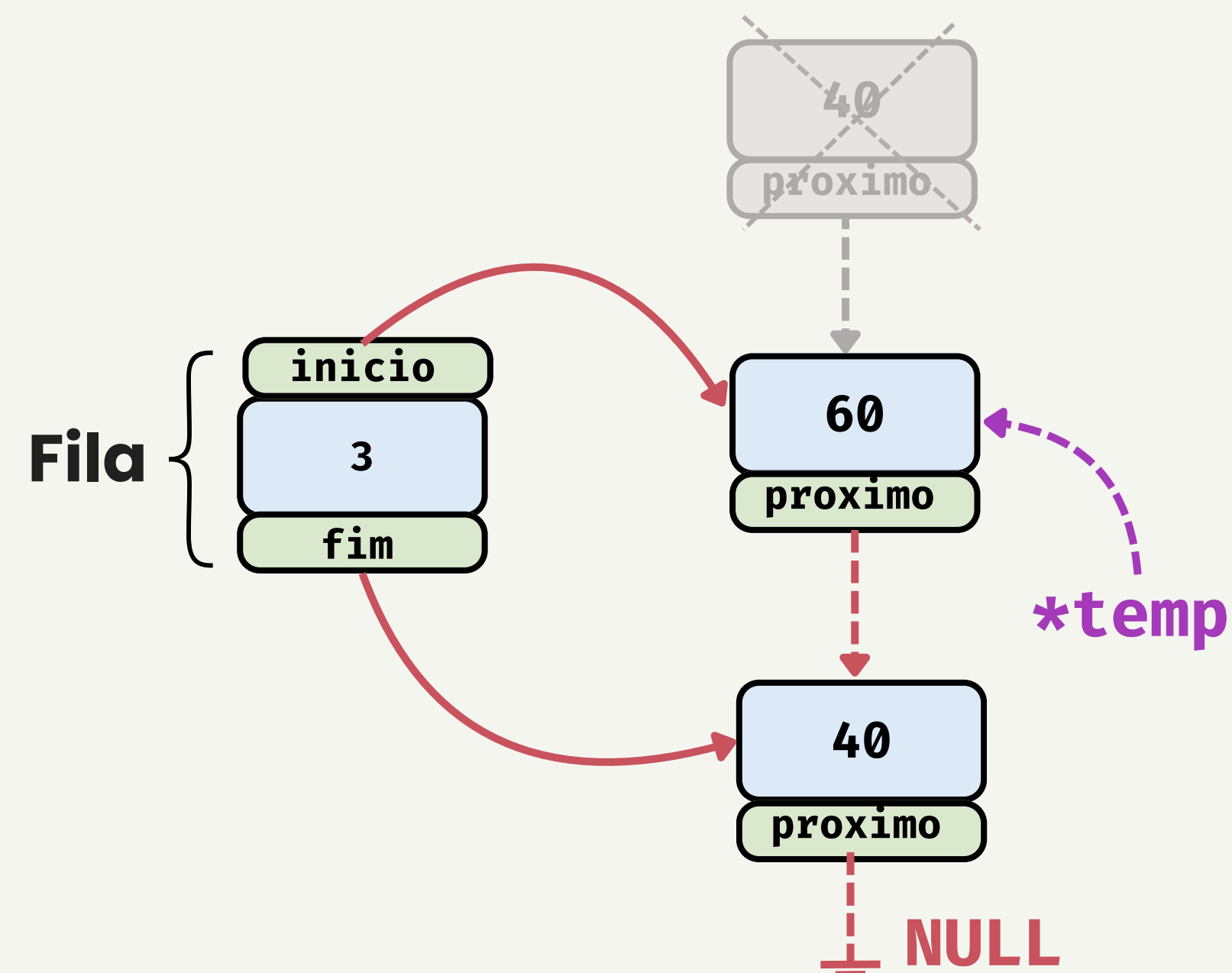
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

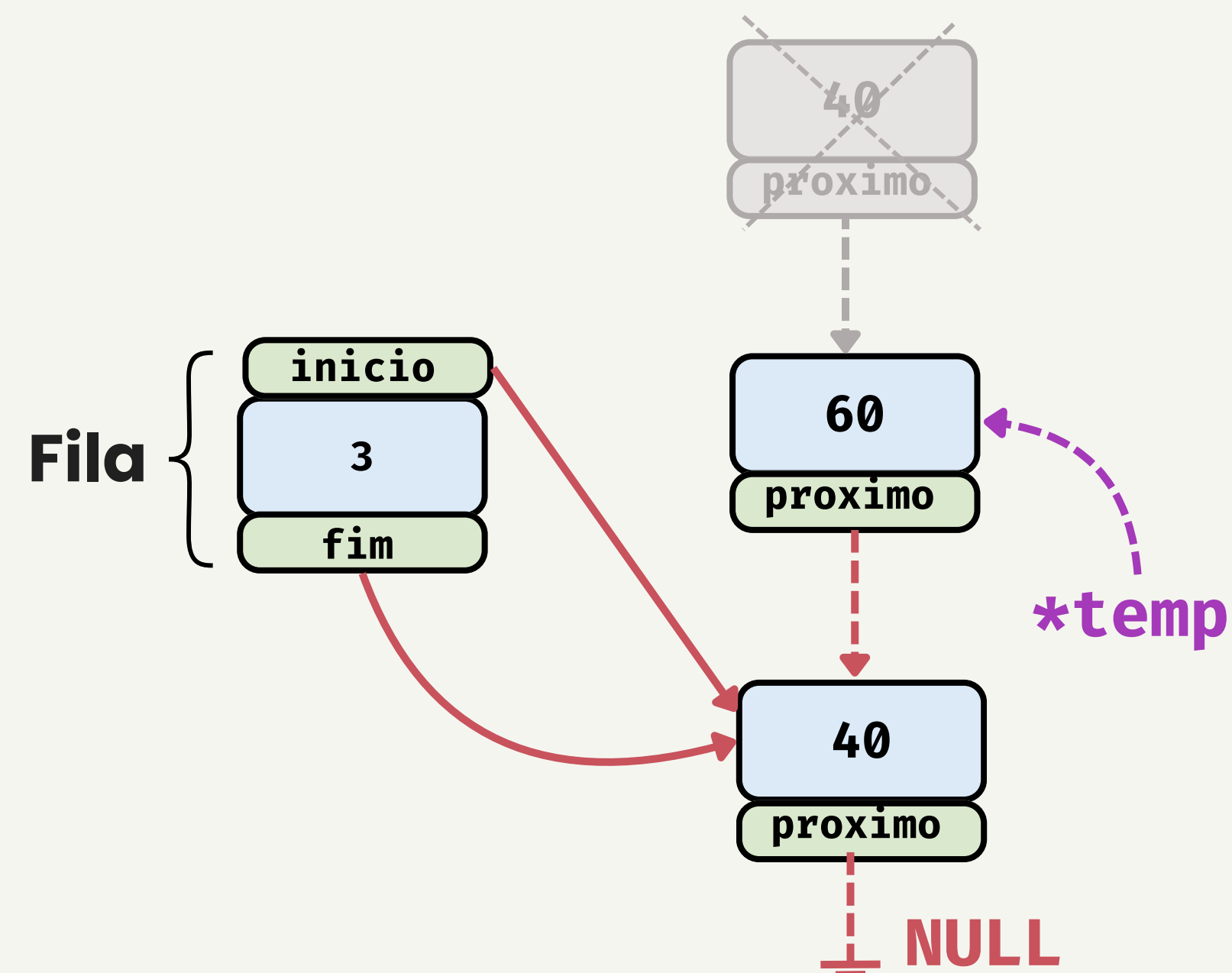
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

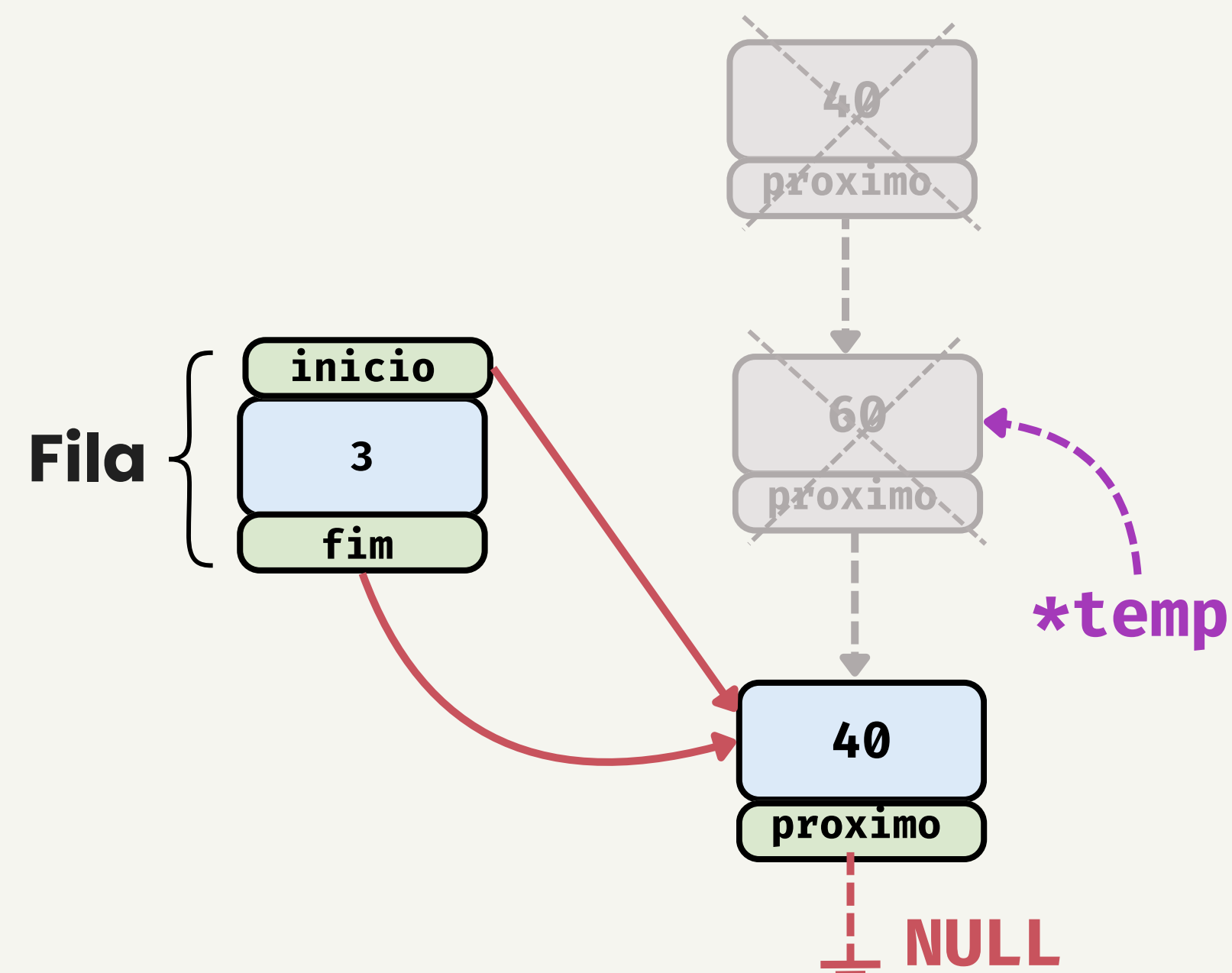
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

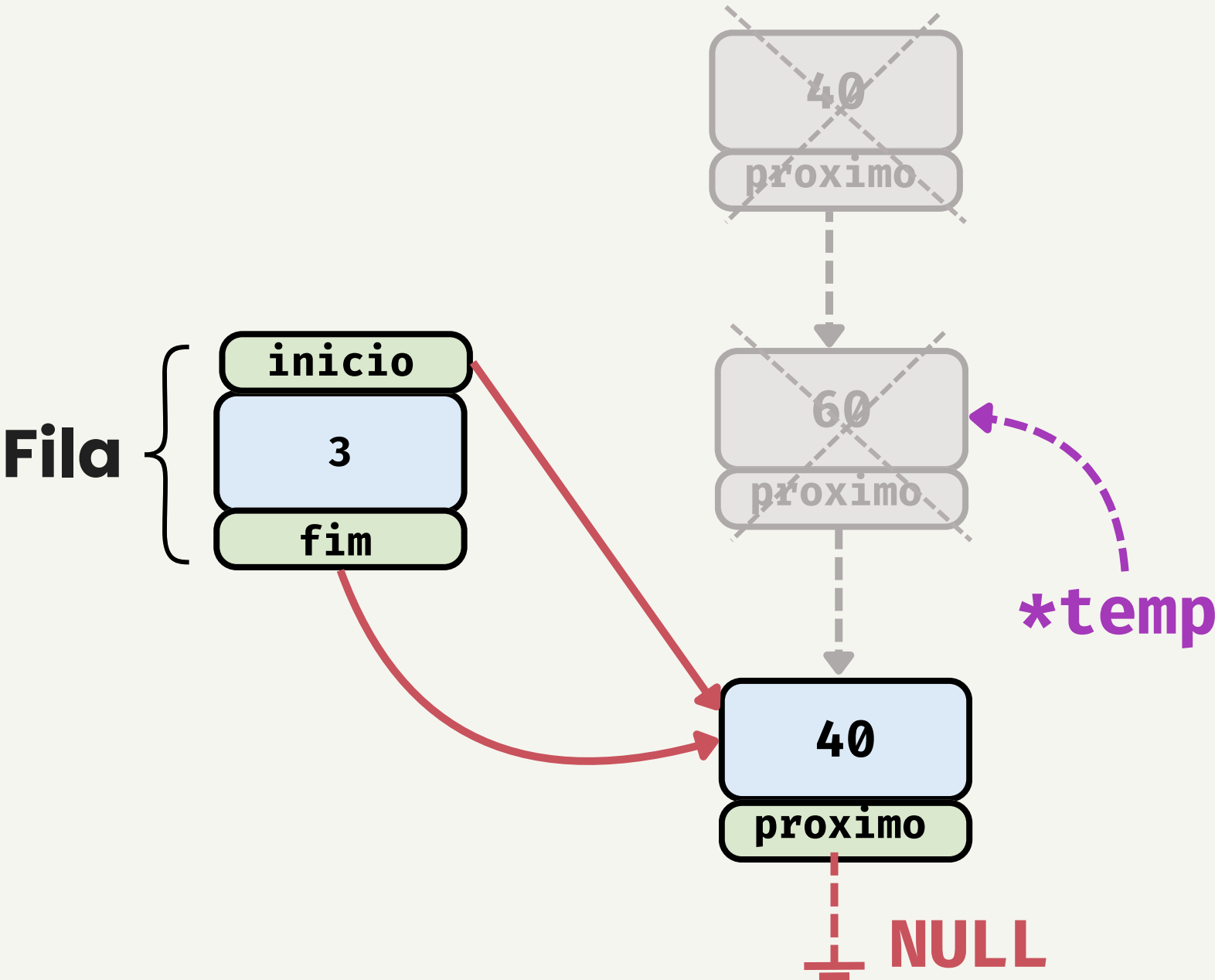
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```

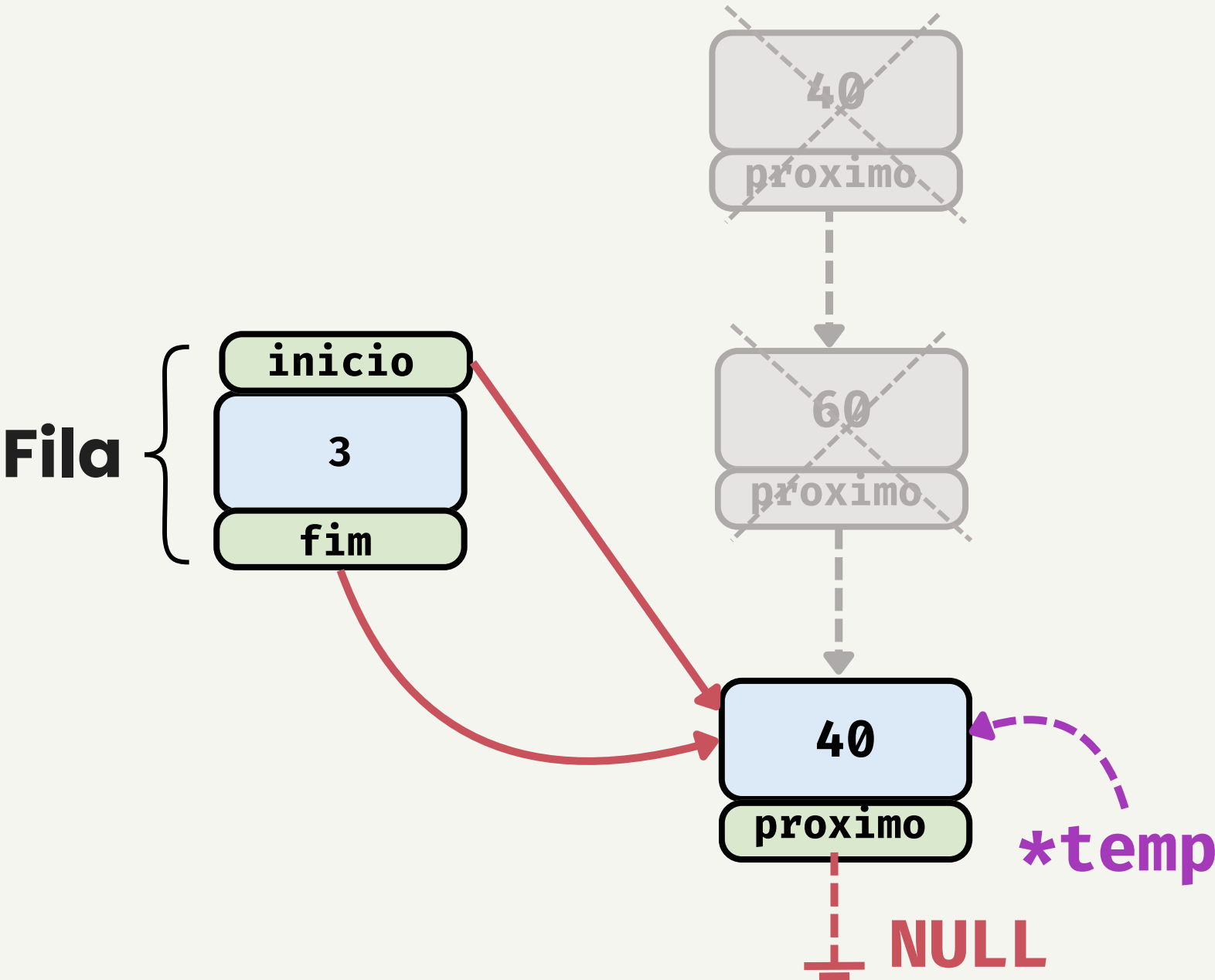


FILAS

- **Limpando** a Fila → liberando os **nós**

```

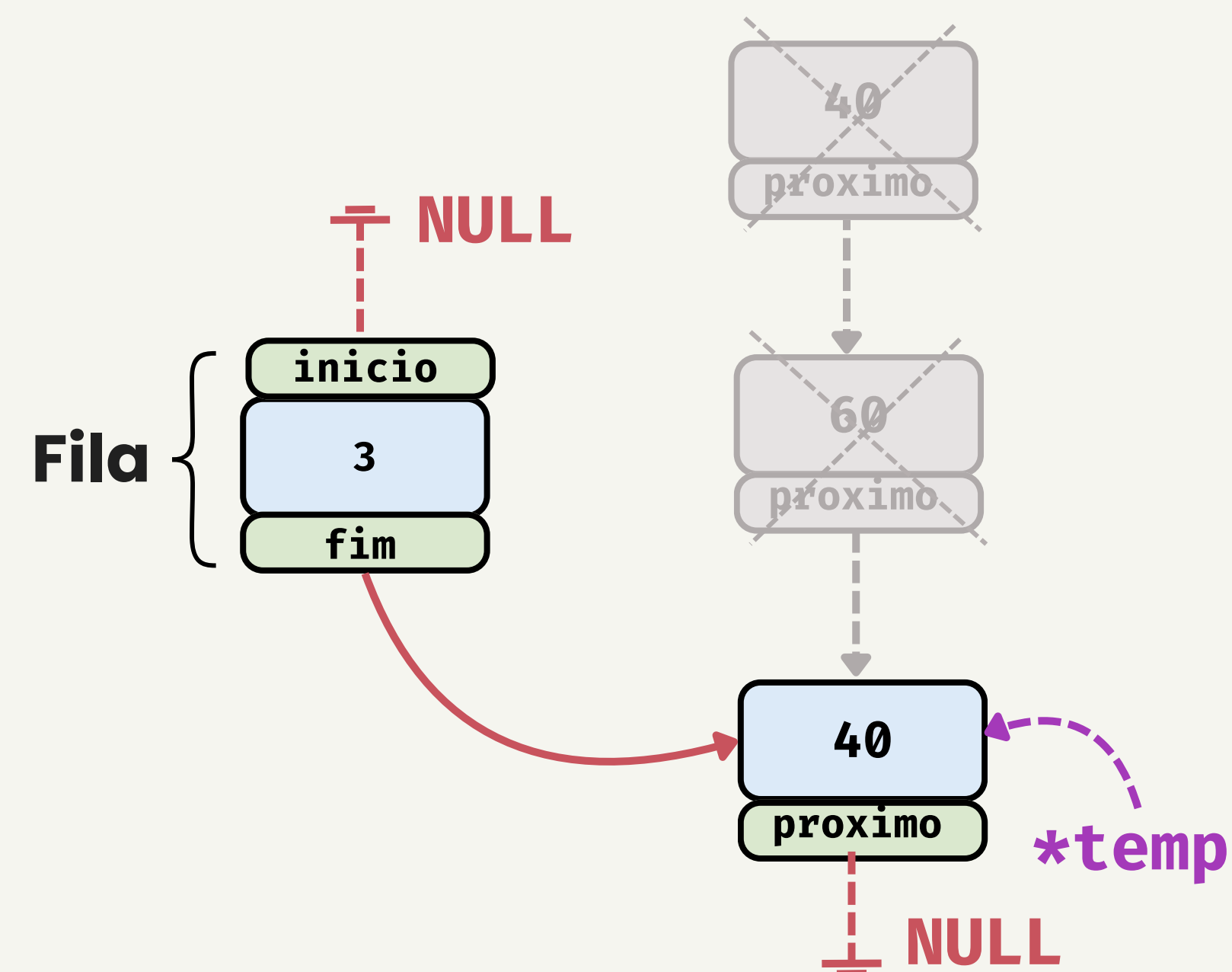
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
    
```



FILAS

- **Limpando** a Fila → liberando os **nós**

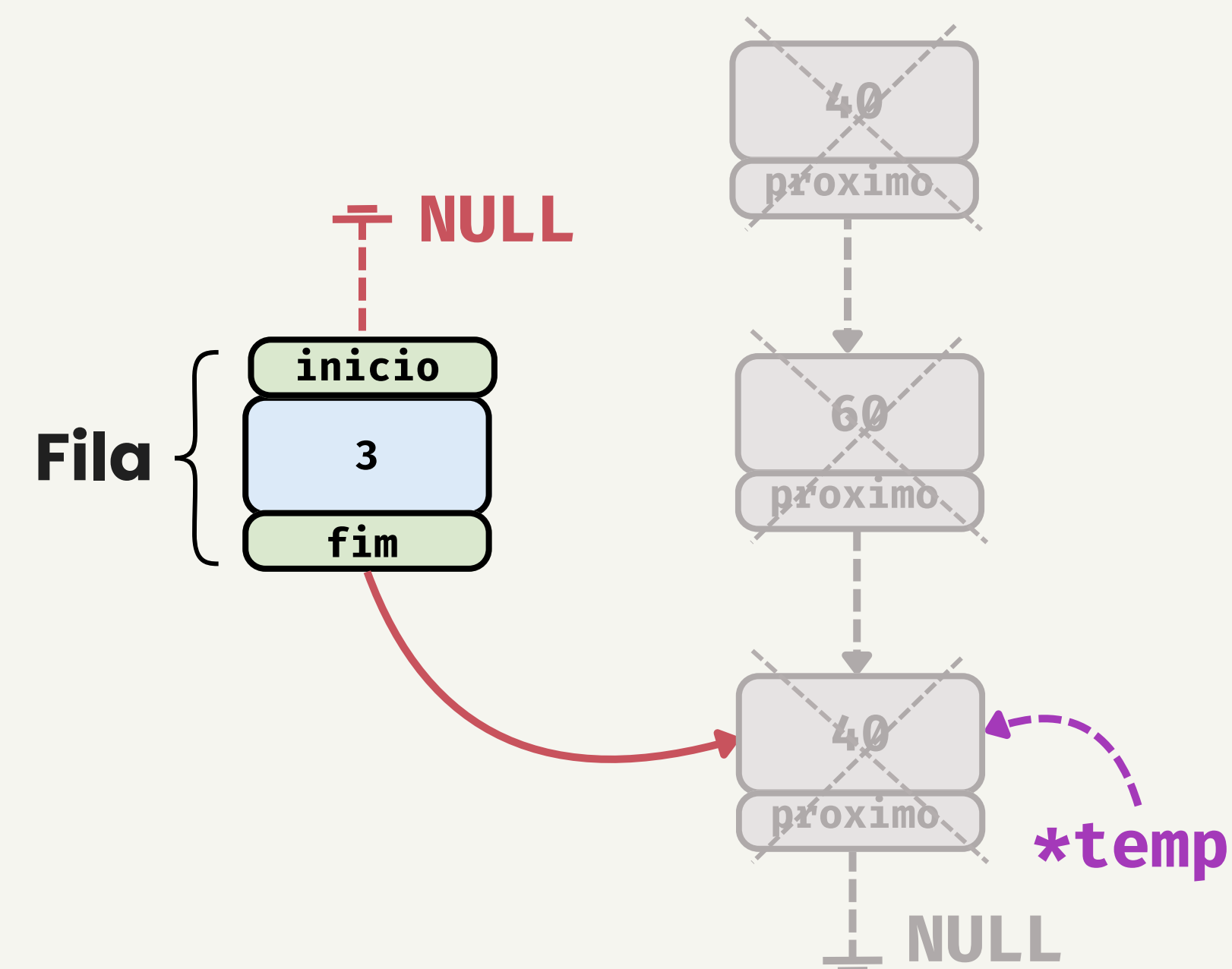
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

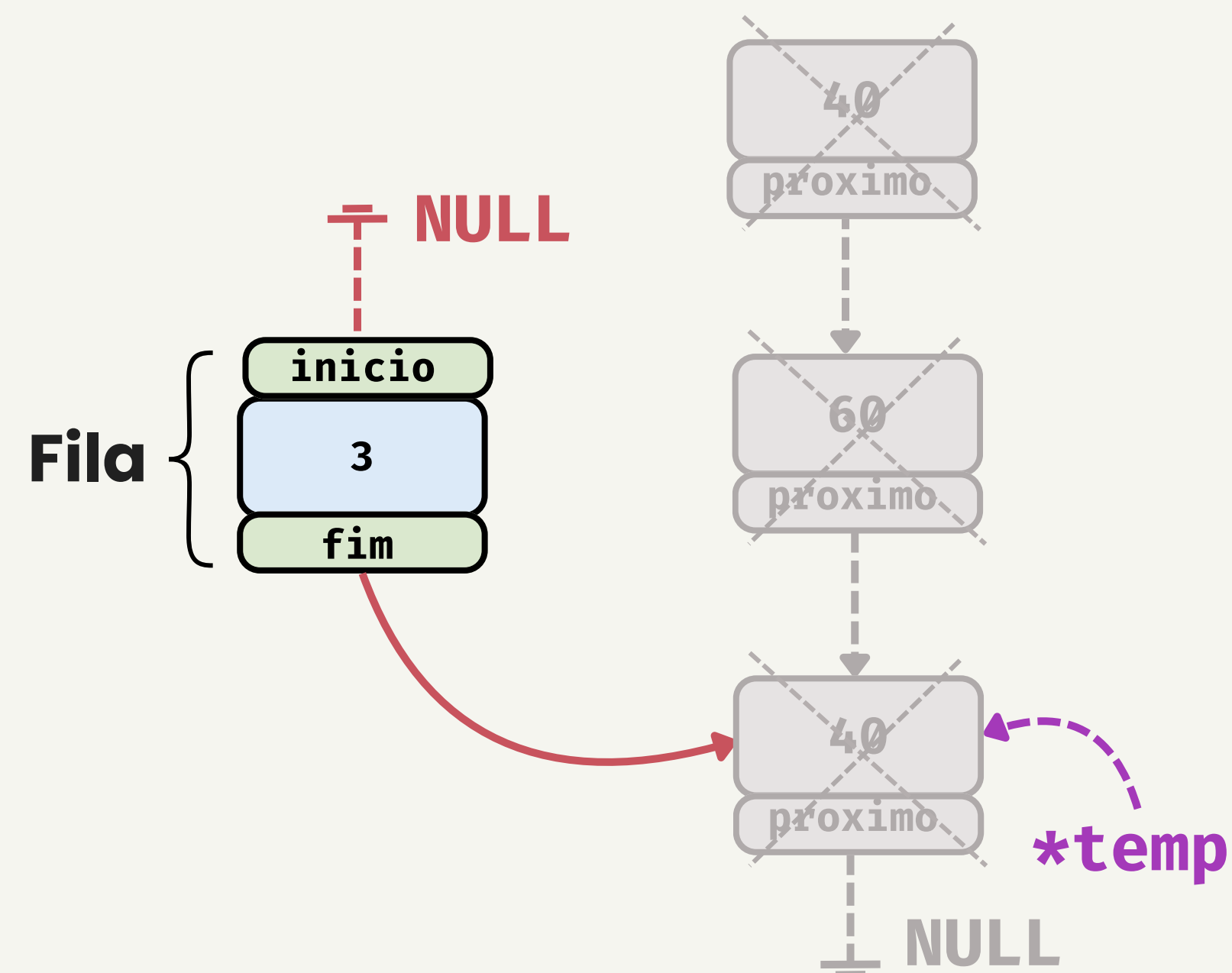
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

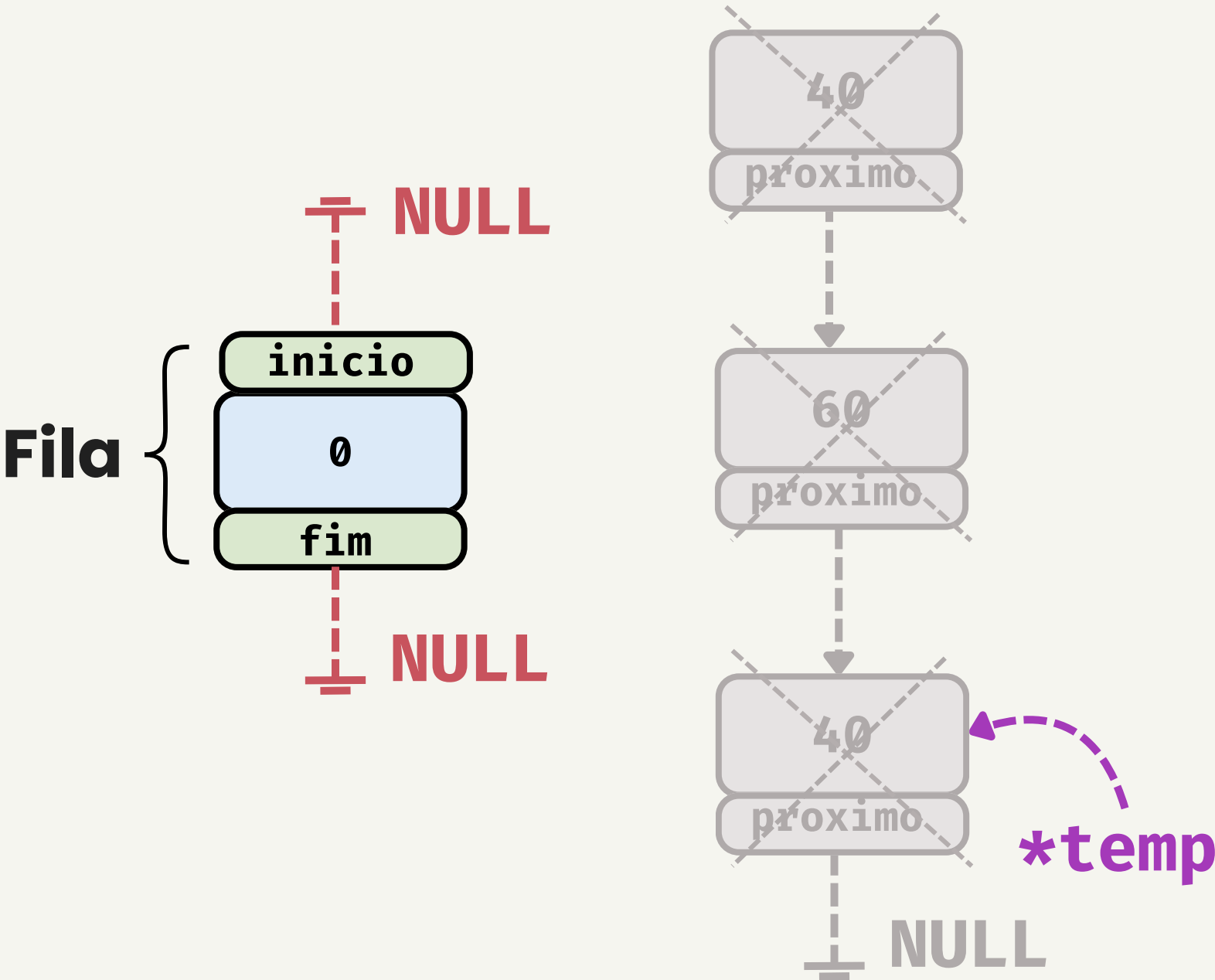
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Limpando** a Fila → liberando os **nós**

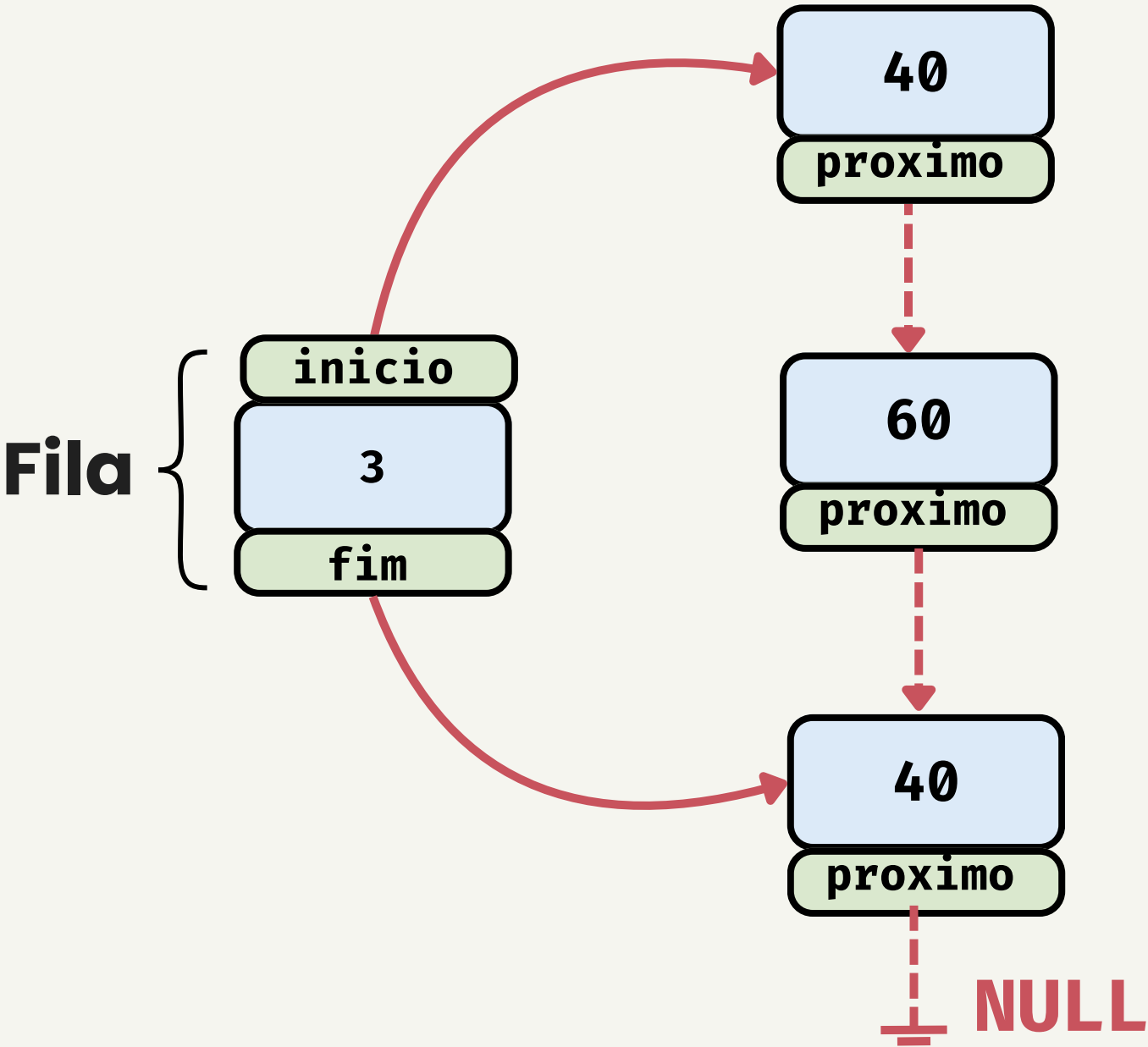
```
void limparFila(Fila* fila) {
    while (fila->inicio != NULL) {
        NoFila* temp = fila->inicio;
        fila->inicio = temp->prox;
        free(temp);
    }
    fila->fim = NULL;
    fila->tamanho = 0;
}
```



FILAS

- **Destruindo** a Fila → liberar os **nós** e a **fila**

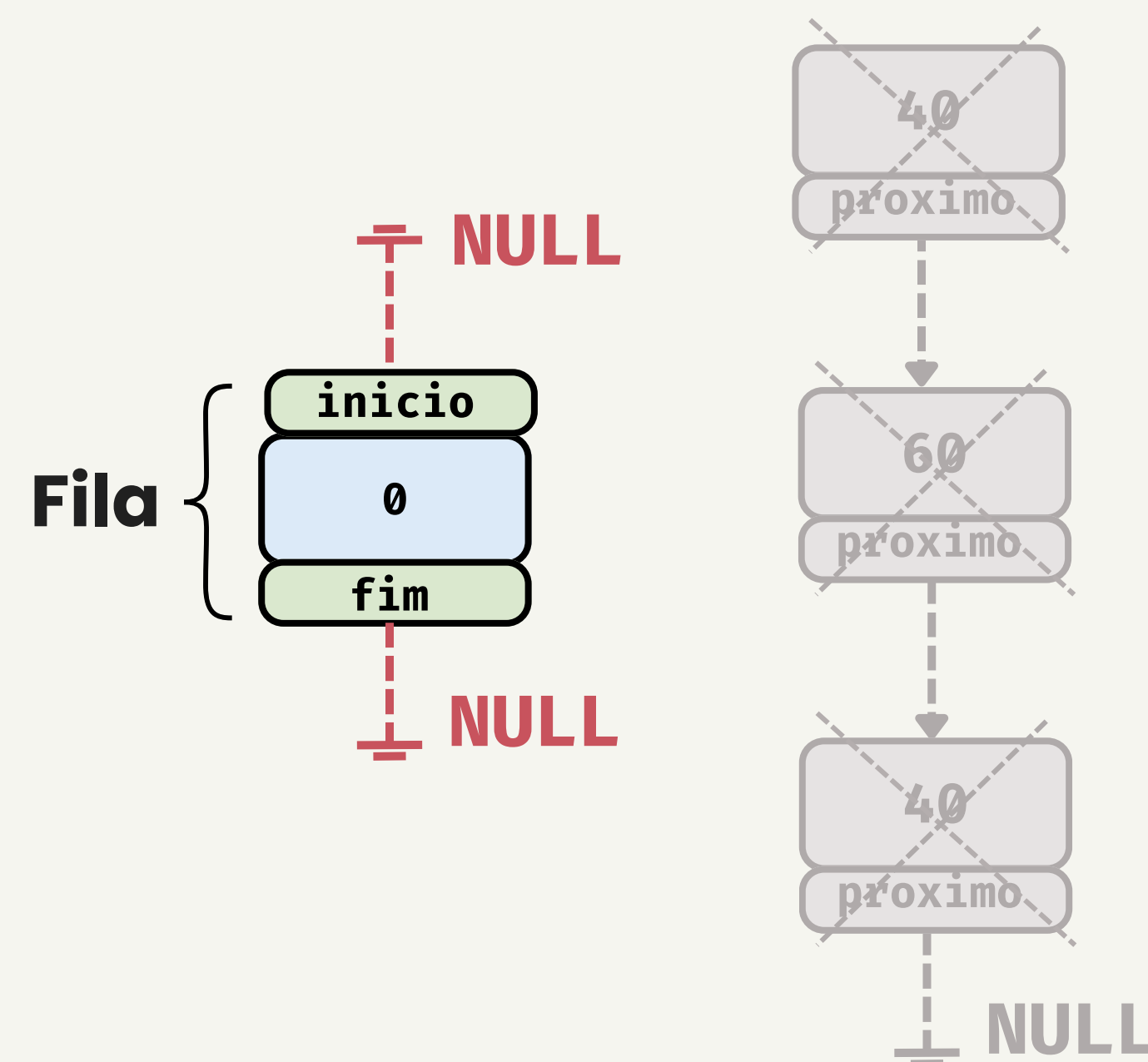
```
void destruirFila(Fila* fila) {  
    limparFila(fila);  
    free(fila);  
}
```



FILAS

- **Destruindo** a Fila → liberar os **nós** e a **fila**

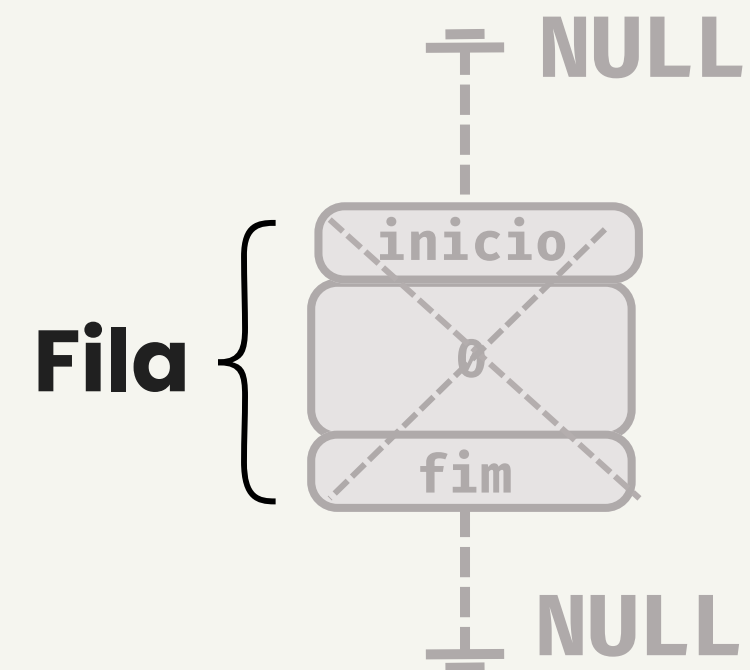
```
void destruirFila(Fila* fila) {
    limparFila(fila);
    free(fila);
}
```



FILAS

- **Destruindo** a Fila → liberar os **nós** e a **fila**

```
void destruirFila(Fila* fila) {  
    limparFila(fila);  
    free(fila);  
}
```



FILAS

- Obter o valor do início

```
int obterInicio(const Fila* fila) {  
    if (filaVazia(fila)) {  
        printf("Fila vazia! Não há elemento na frente.\n");  
        return -1; // Retorna um valor inválido para indicar erro  
    }  
    return fila->inicio->valor;  
}
```

FILAS

- Obter o valor do início

```
int obterInicio(const Fila* fila) {  
    if (filaVazia(fila)) {  
        printf("Fila vazia! Não há elemento na frente.\n");  
        return -1; // Retorna um valor inválido para indicar erro  
    }  
    return fila->inicio->valor;  
}
```

- Obter o tamanho

```
int obterTamanho(const Fila* fila) {  
    return fila->tamanho;  
}
```

FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

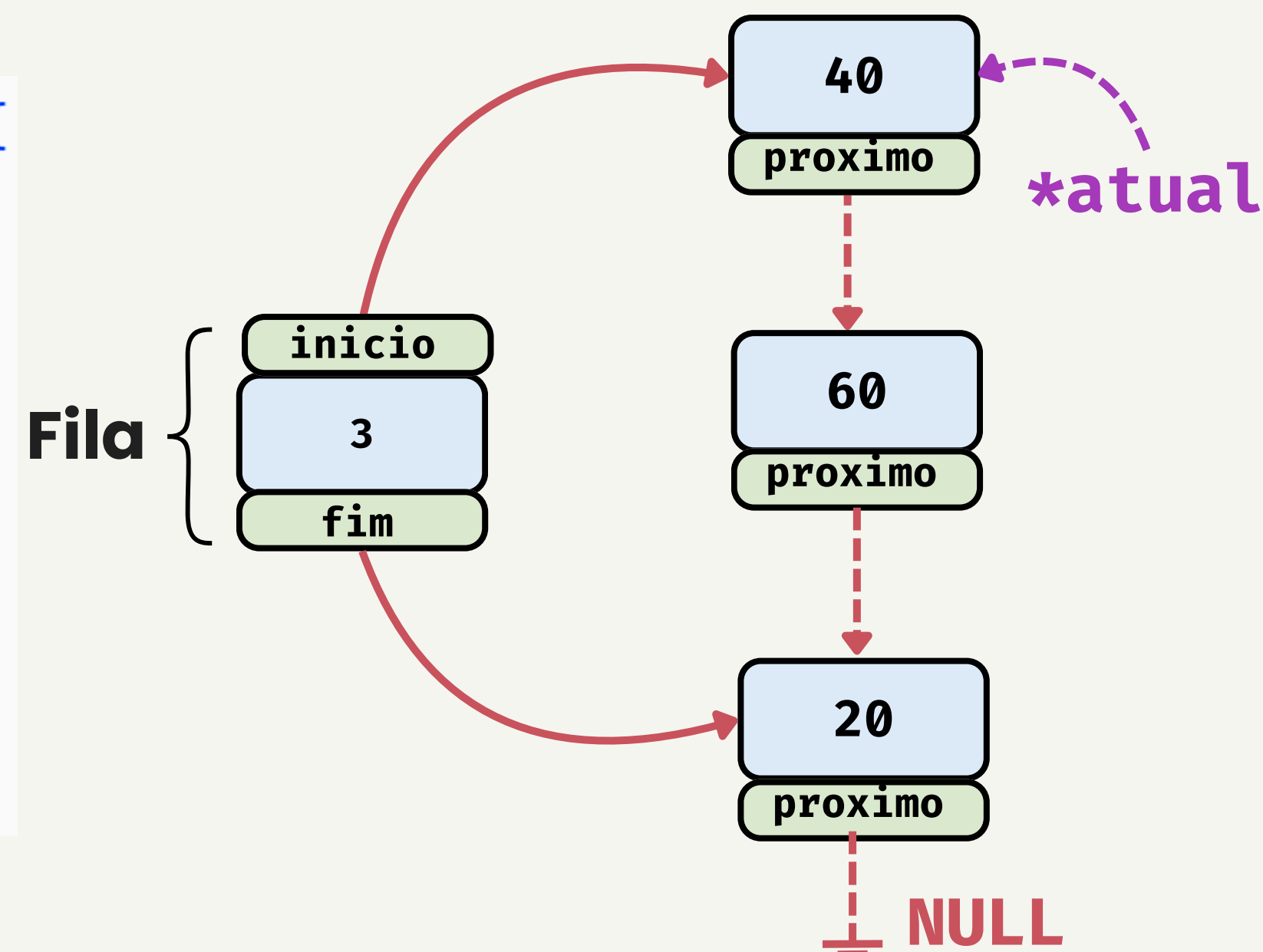
```
bool buscarValor(const Fila* fila, int valor) {  
    NoFila* atual = fila->inicio;  
    while (atual != NULL) {  
        if (atual->valor == valor) {  
            return true;  
        }  
        atual = atual->prox;  
    }  
    return false;  
}
```


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```

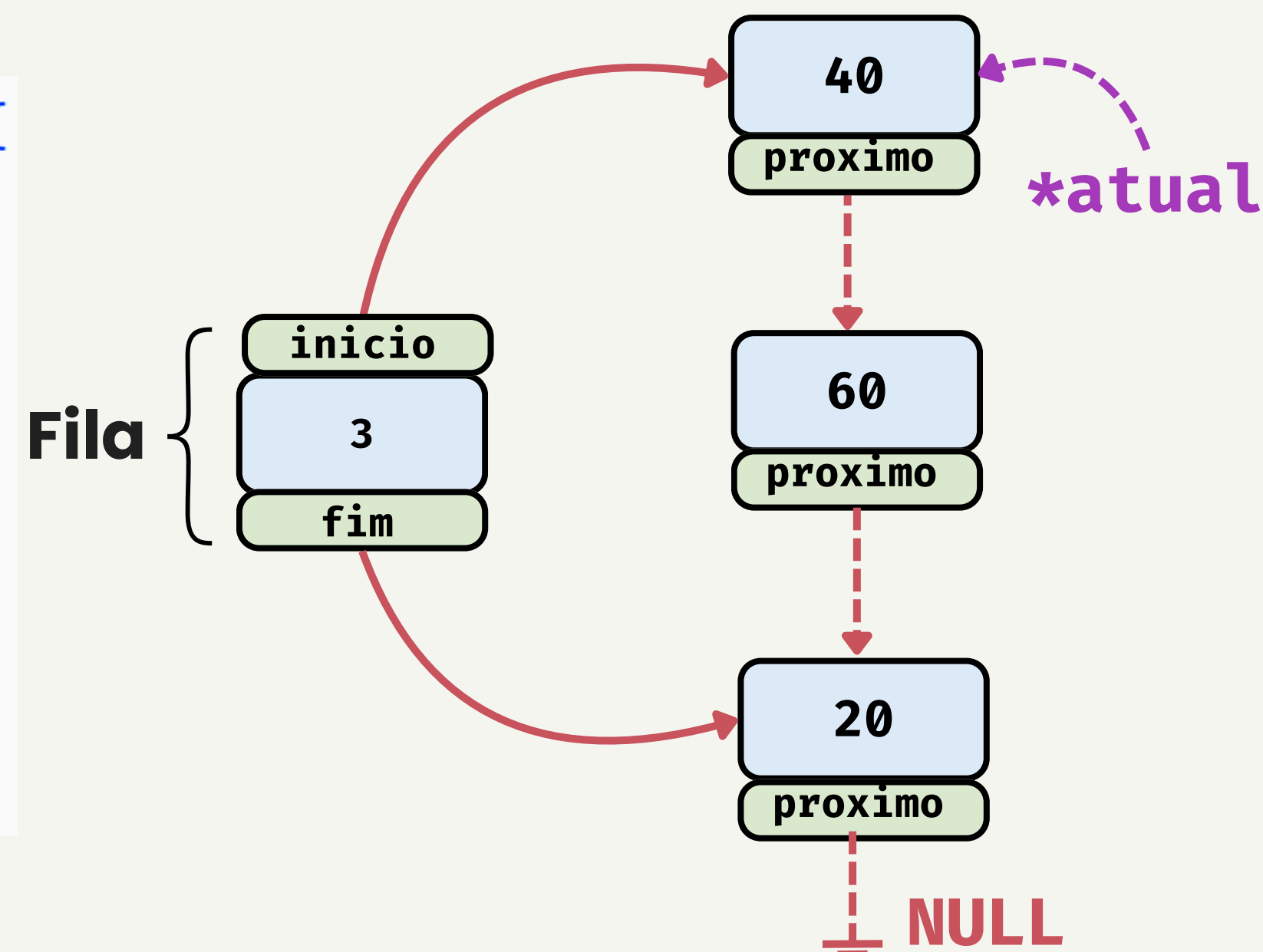


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```

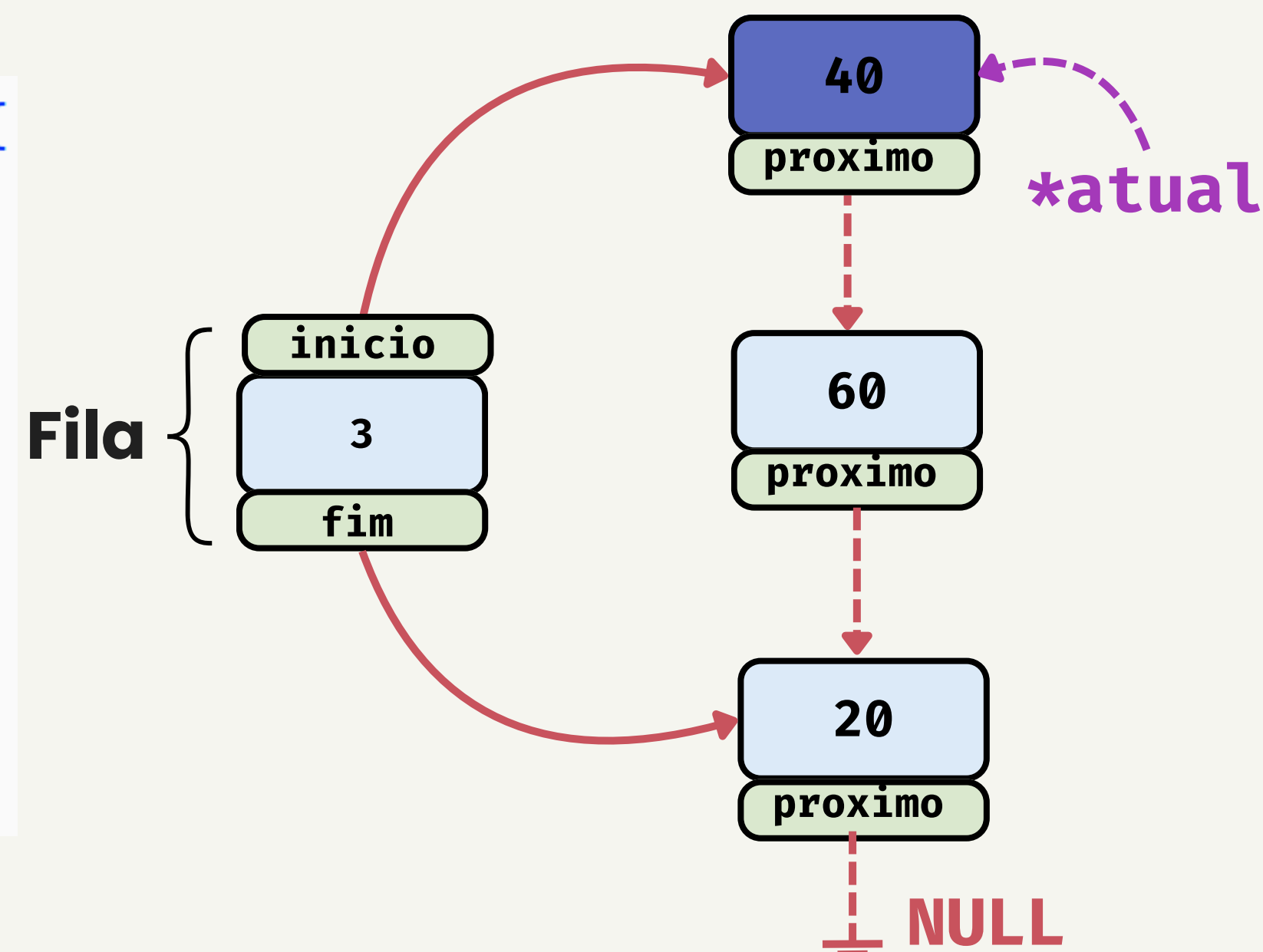


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```

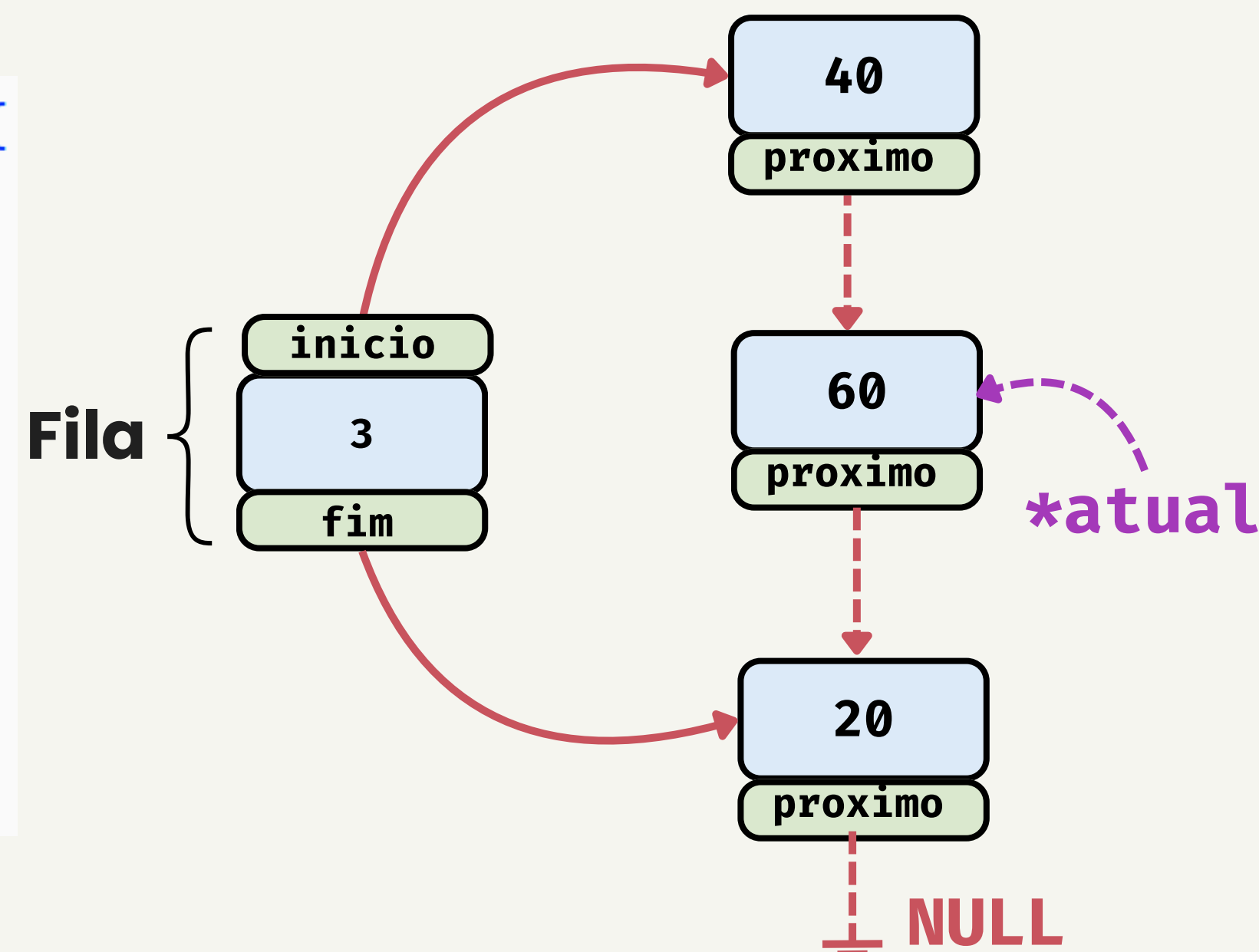


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```

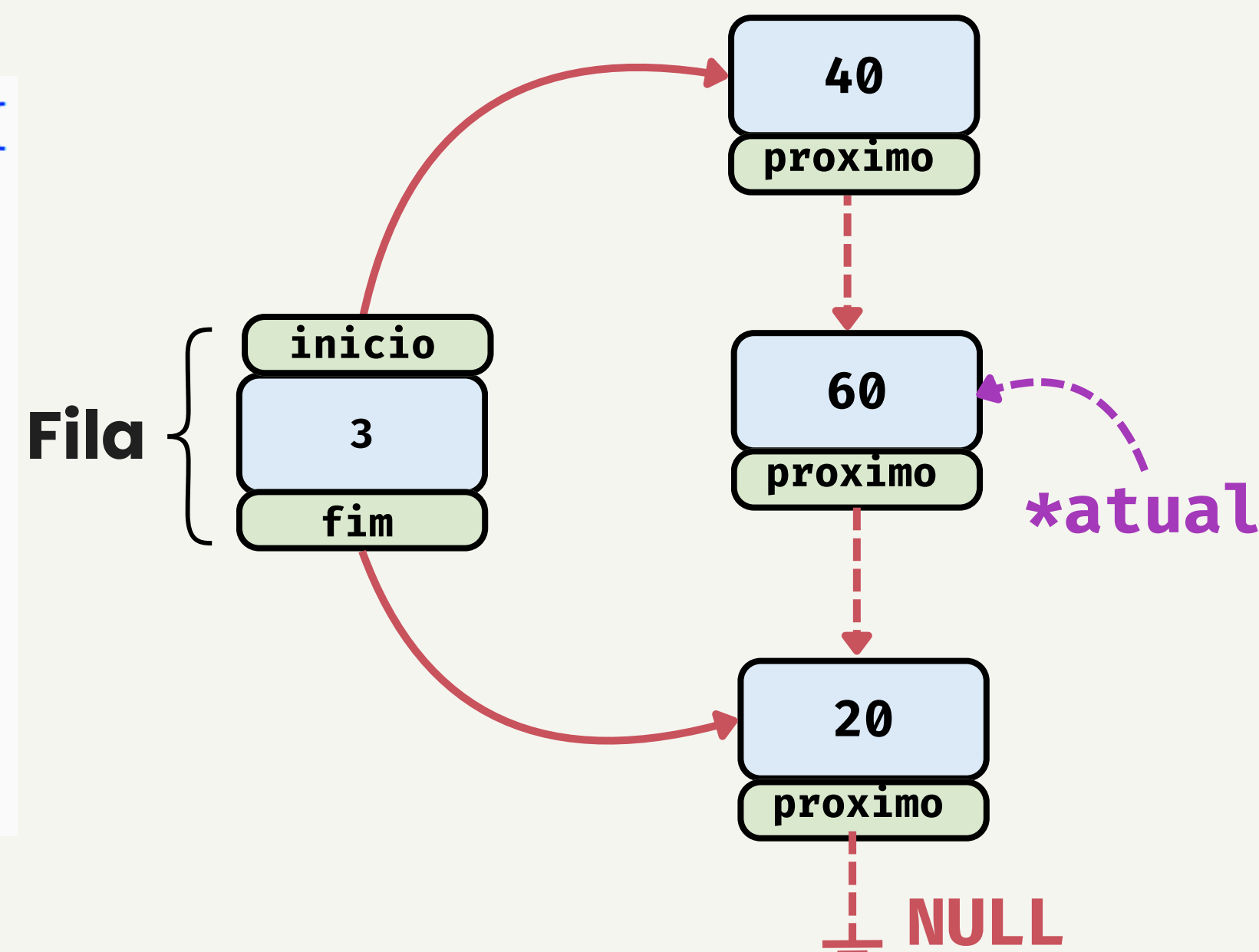


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```

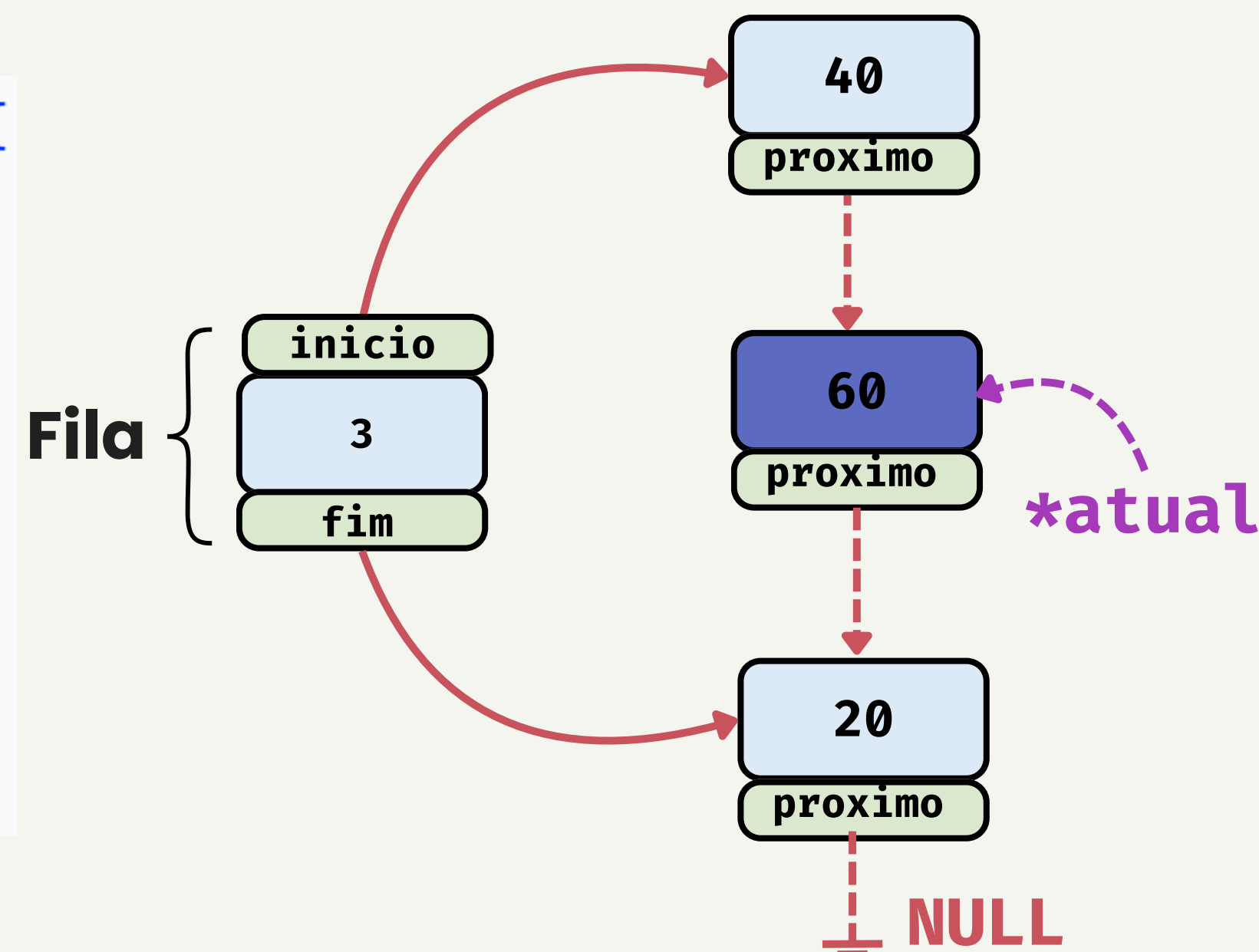


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```

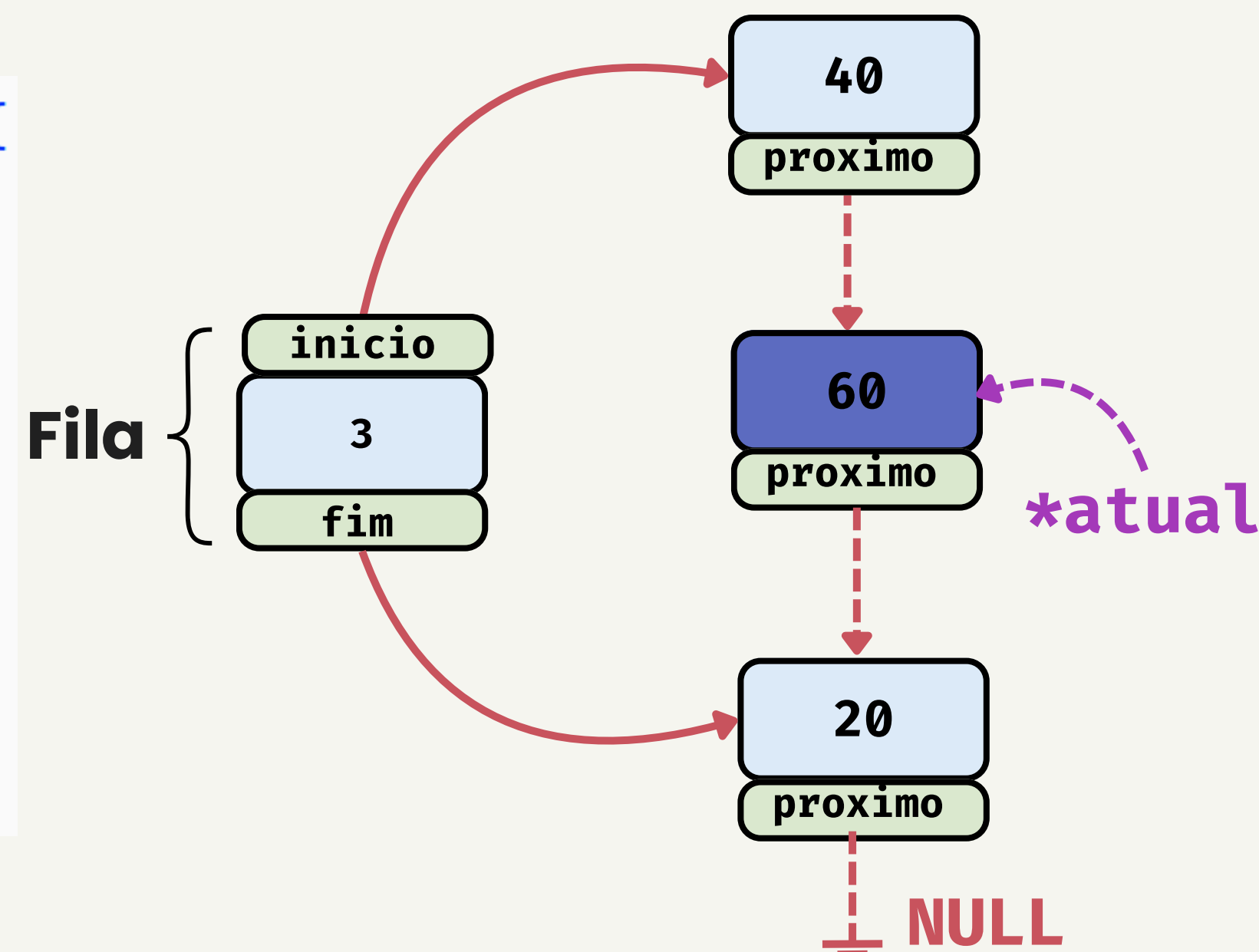


FILAS

- **Buscar valor** → Retorna **true** se encontrar e **false** se não encontrar.

Buscar valor 60

```
bool buscarValor(const Fila* fila, int valor) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valor) {
            return true;
        }
        atual = atual->prox;
    }
    return false;
}
```



FILAS

- Atualizar valor de um nó

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {  
    NoFila* atual = fila->inicio;  
    while (atual != NULL) {  
        if (atual->valor == valorAntigo) {  
            atual->valor = valorNovo;  
            return;  
        }  
        atual = atual->prox;  
    }  
    printf("Valor %d não encontrado na fila.\n", valorAntigo);  
}
```


FILAS

- Atualizar valor de um nó

Porquê eu não estou usando ponteiro ?

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {  
    NoFila* atual = fila->inicio;  
    while (atual != NULL) {  
        if (atual->valor == valorAntigo) {  
            atual->valor = valorNovo;  
            return;  
        }  
        atual = atual->prox;  
    }  
    printf("Valor %d não encontrado na fila.\n", valorAntigo);  
}
```

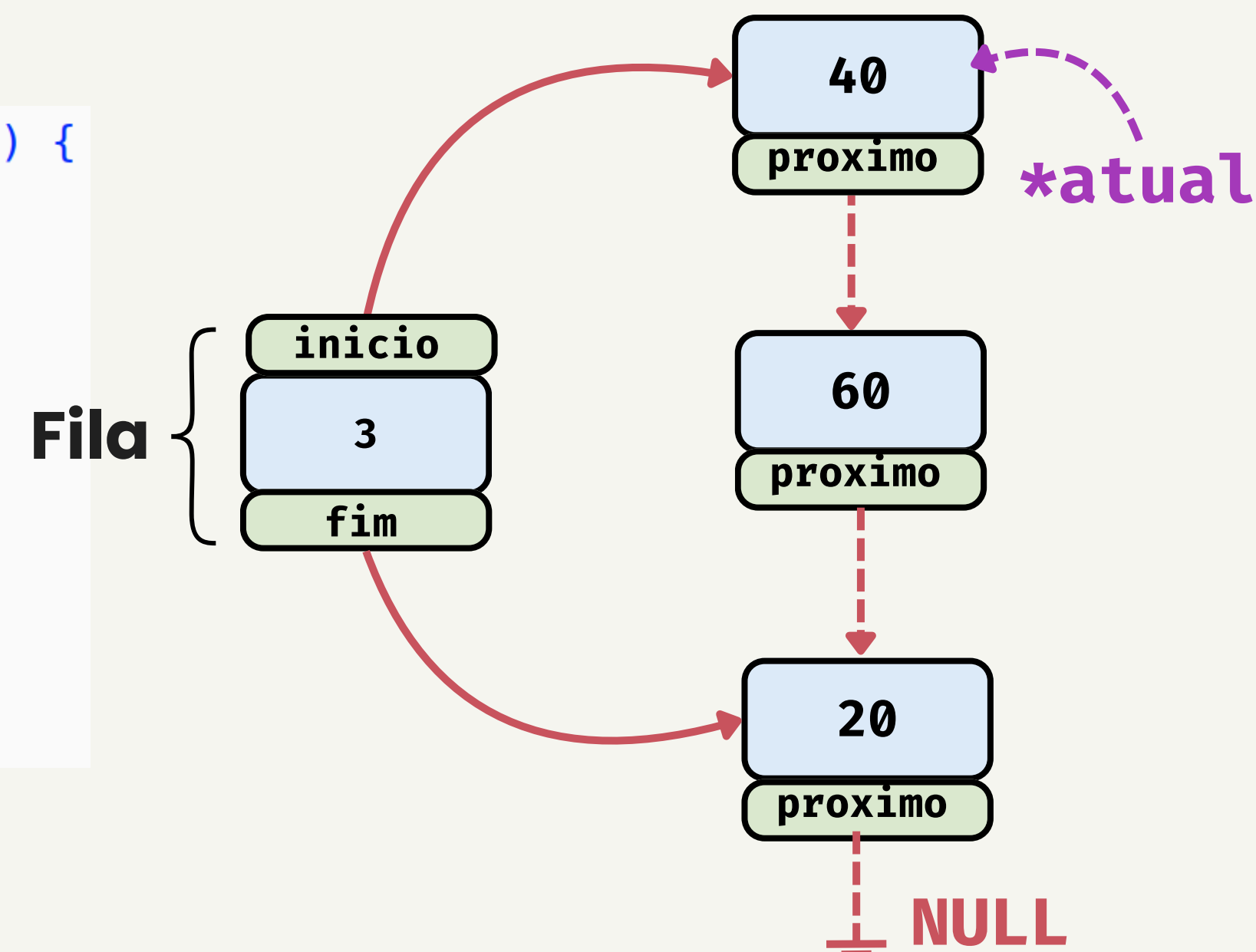
FILAS

- Atualizar valor de um nó

- Valor antigo: 60

- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```



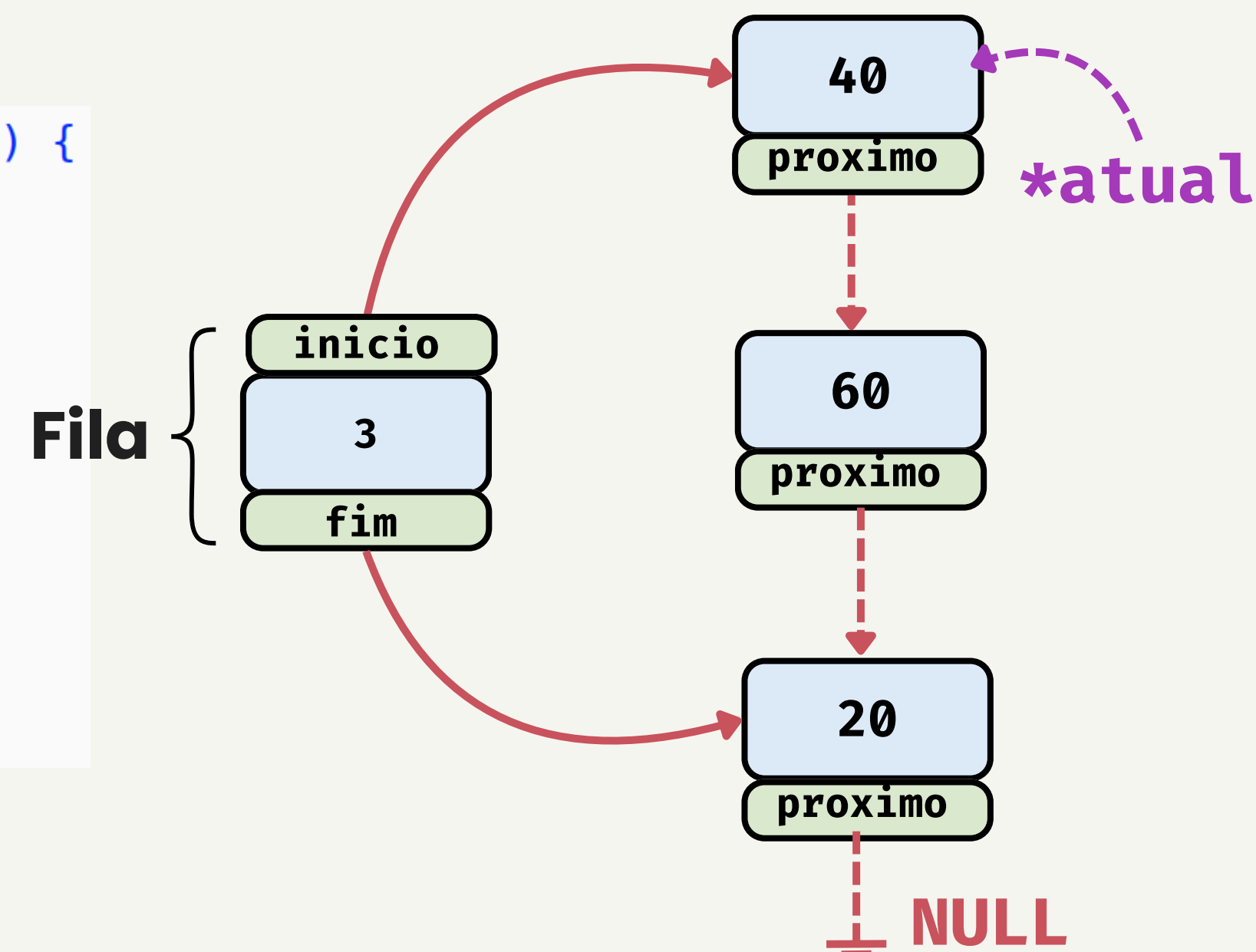
FILAS

- Atualizar valor de um nó

- Valor antigo: 60

- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```

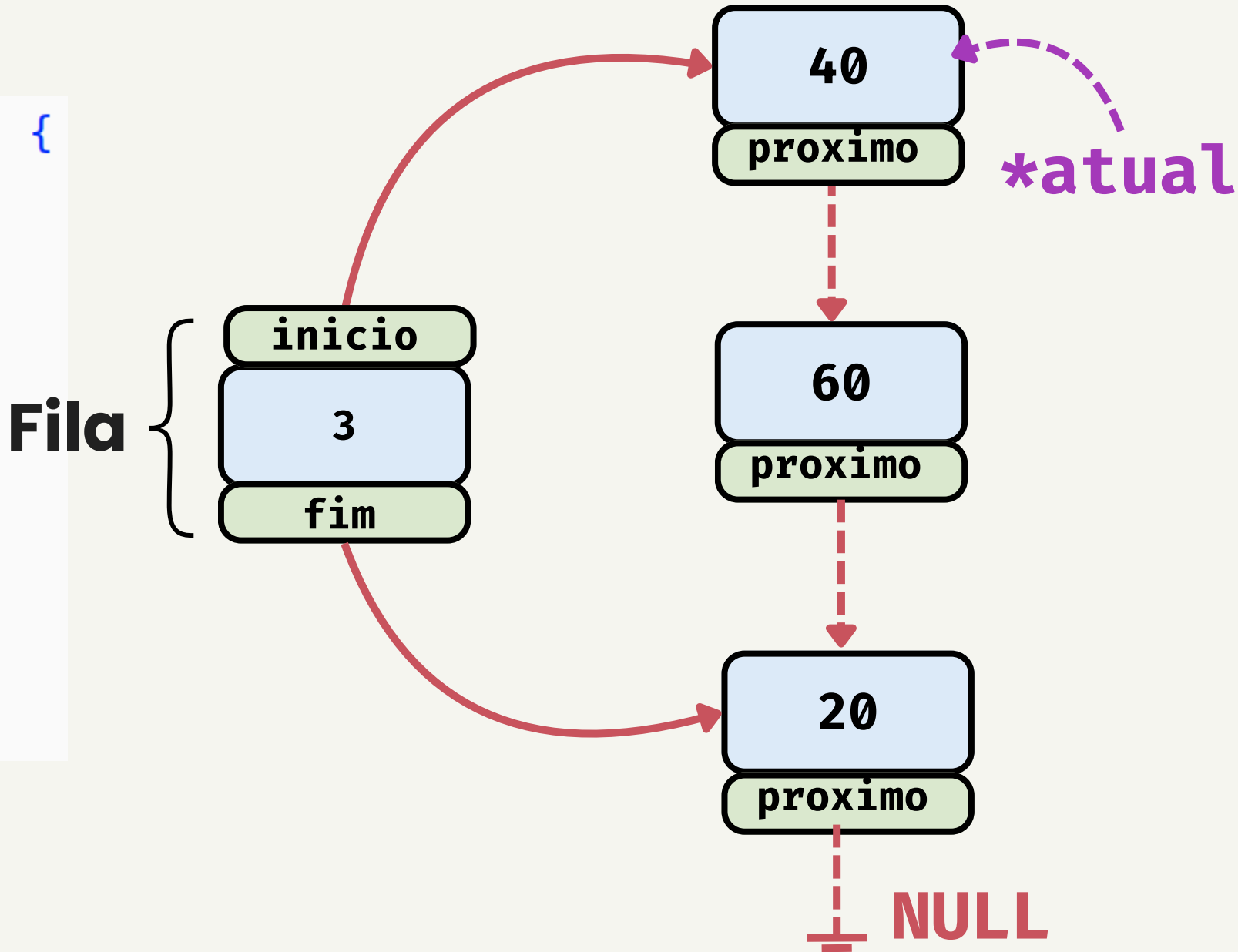


FILAS

- Atualizar valor de um nó

- Valor antigo: 60
- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```



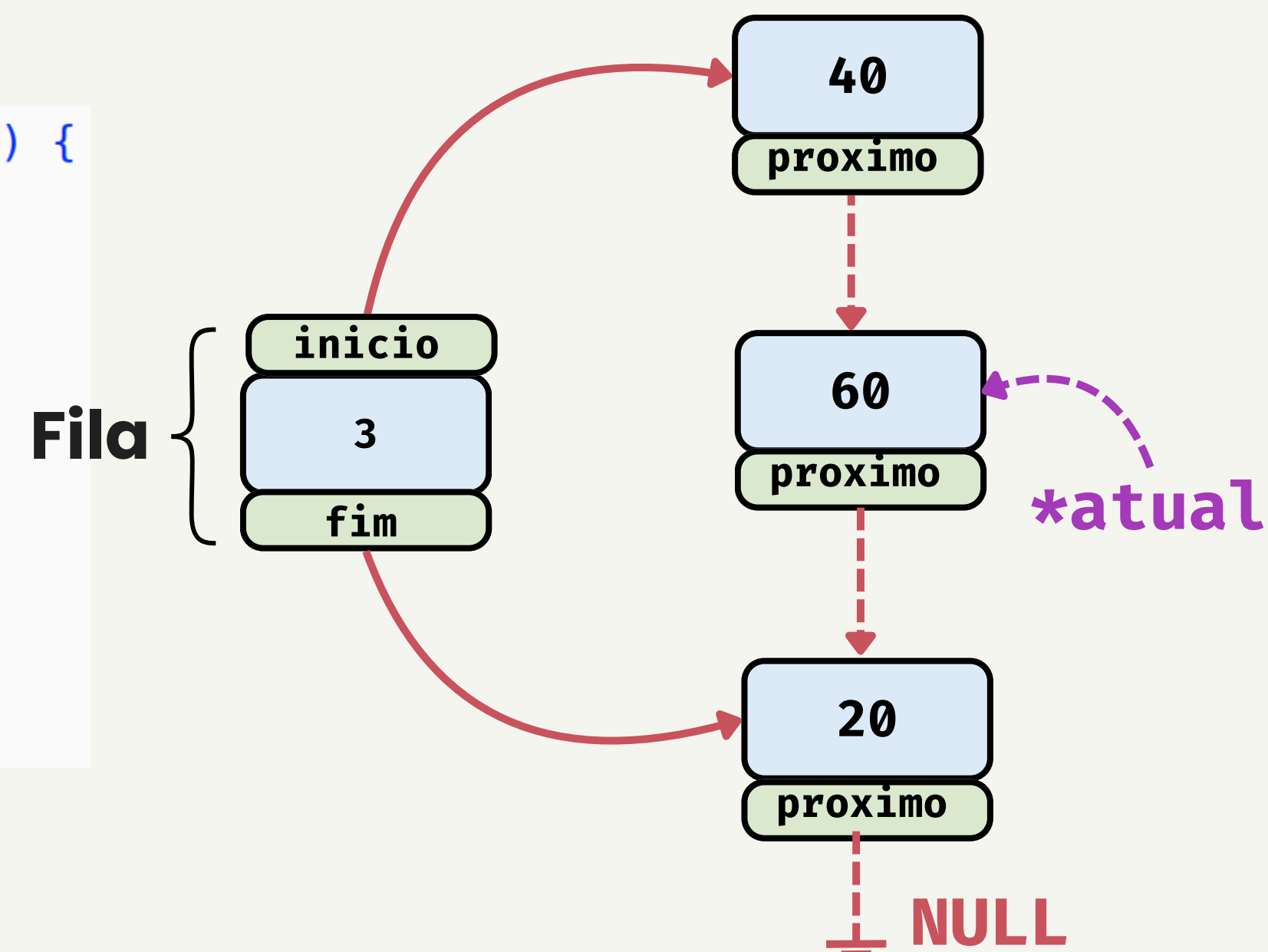
FILAS

- Atualizar valor de um nó

- Valor antigo: 60

- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```

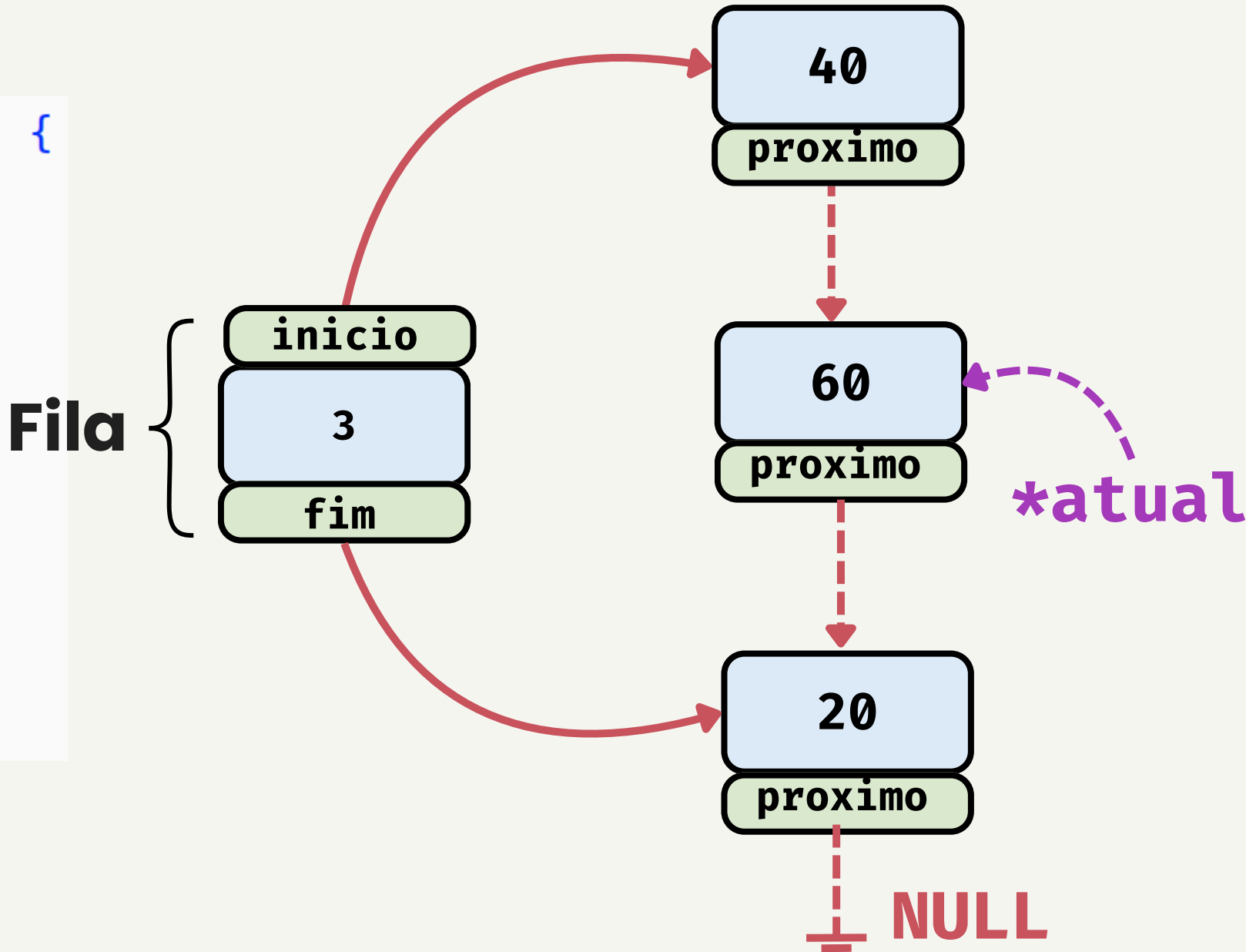


FILAS

- Atualizar valor de um nó

- Valor antigo: 60
- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```



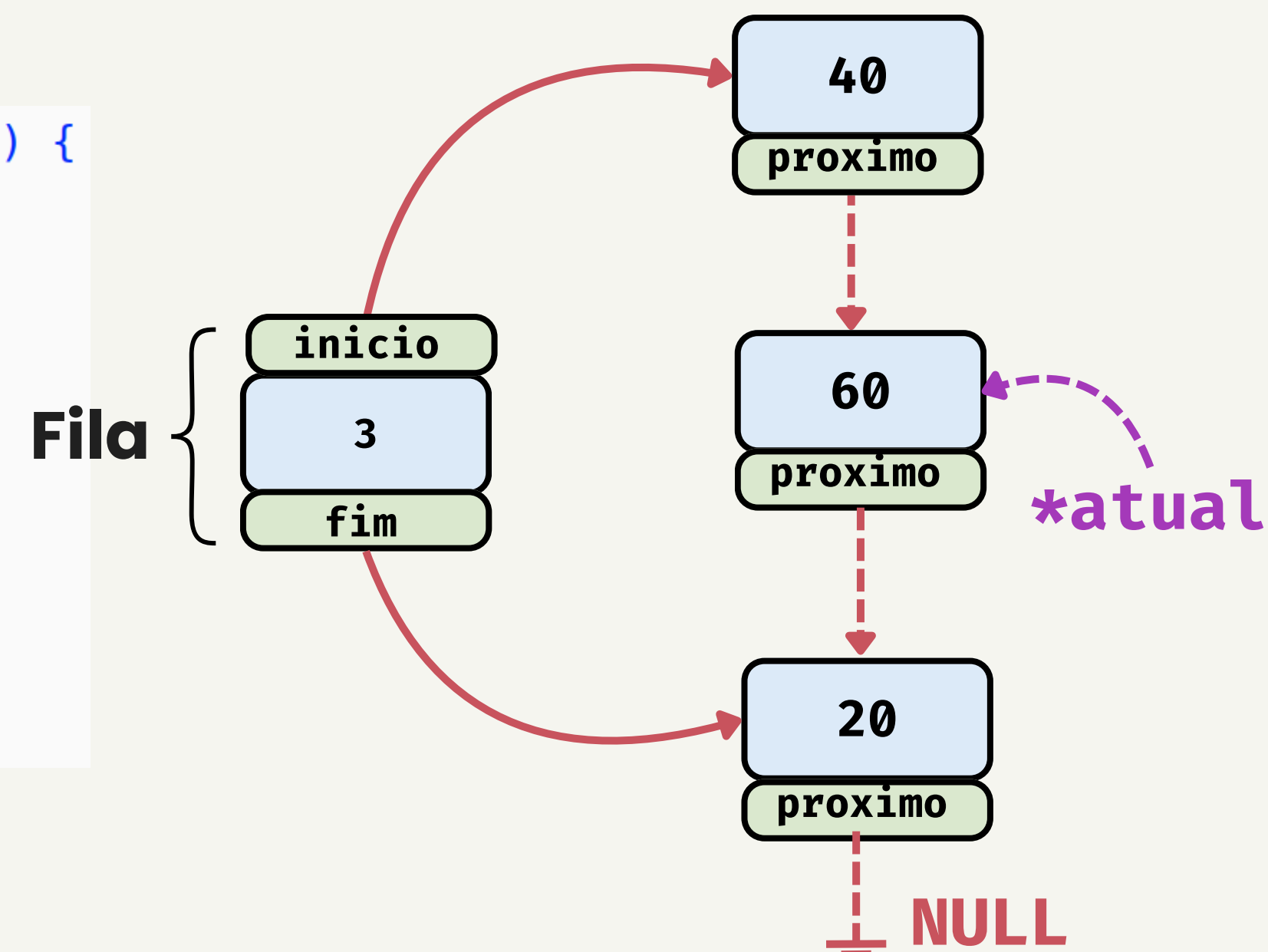
FILAS

- Atualizar valor de um nó

- Valor antigo: 60

- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```



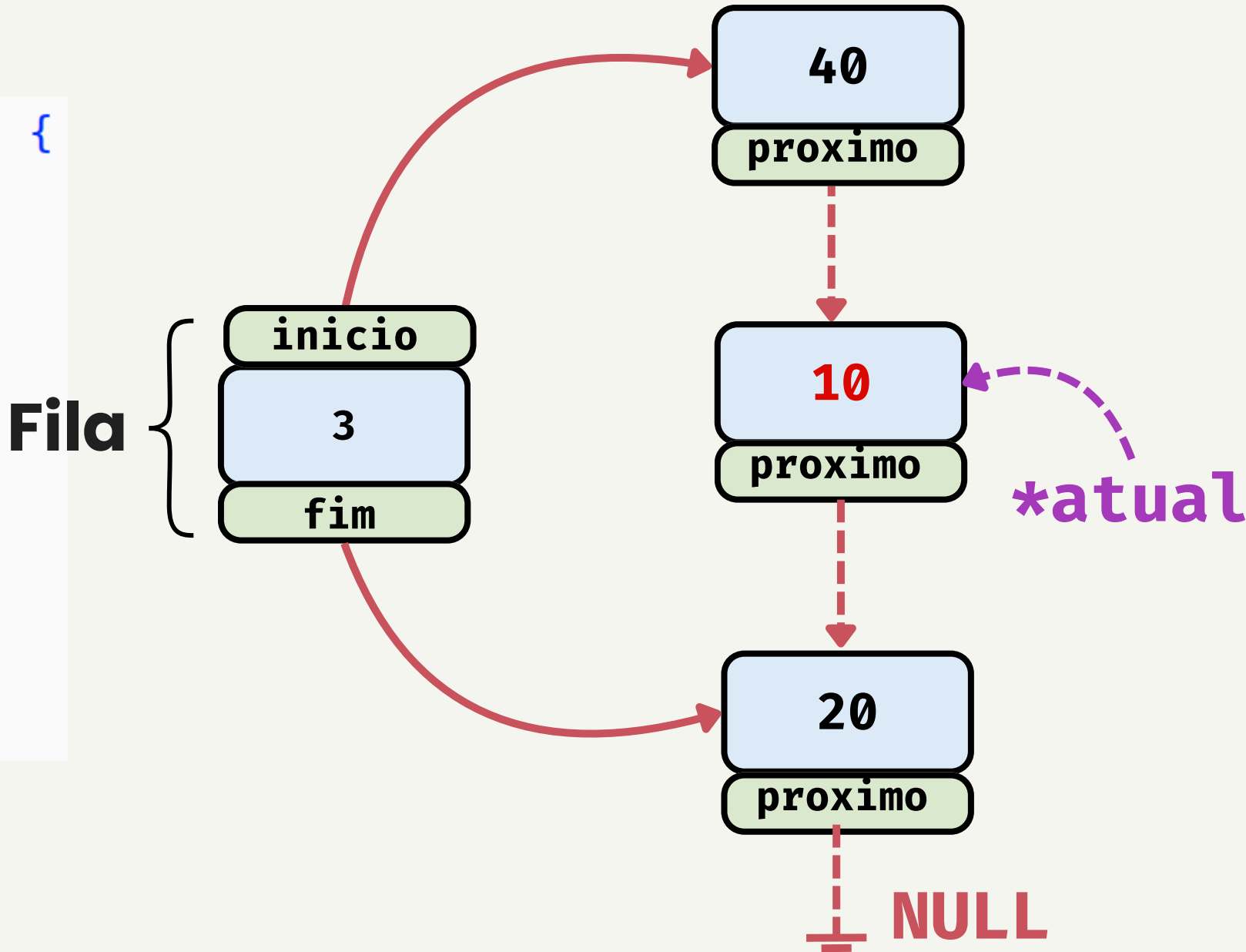
FILAS

- Atualizar valor de um nó

- Valor antigo: 60
- Valor novo: 10

```

void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
    
```



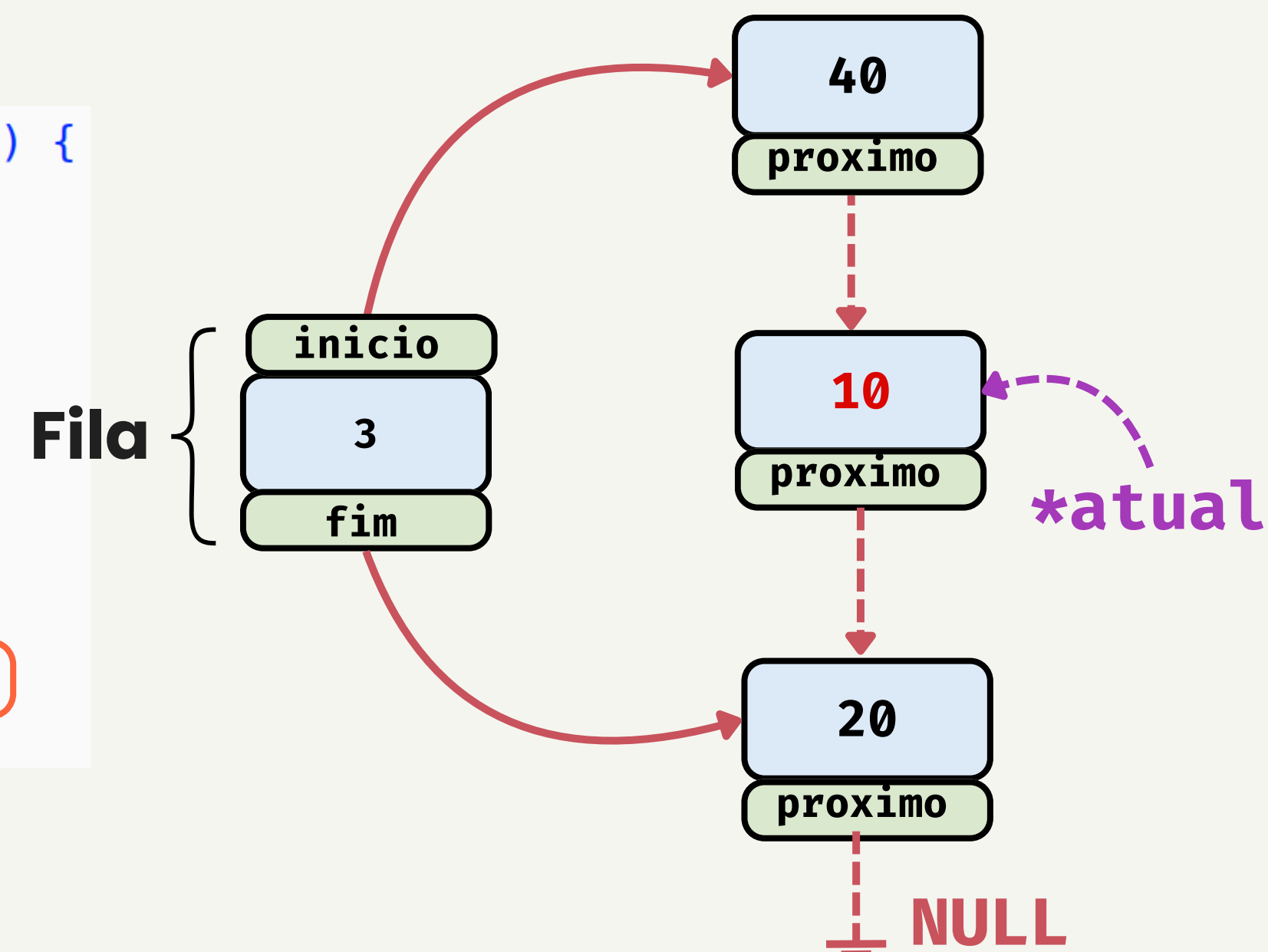
FILAS

- Atualizar valor de um nó

- Valor antigo: 60

- Valor novo: 10

```
void atualizarValor(Fila* fila, int valorAntigo, int valorNovo) {
    NoFila* atual = fila->inicio;
    while (atual != NULL) {
        if (atual->valor == valorAntigo) {
            atual->valor = valorNovo;
            return;
        }
        atual = atual->prox;
    }
    printf("Valor %d não encontrado na fila.\n", valorAntigo);
}
```



FILAS

- Imprimir a fila

```
void imprimirFila(const Fila* fila) {  
    if (filaVazia(fila)) {  
        printf("Fila vazia!\n");  
        return;  
    }  
    NoFila* atual = fila->inicio;  
    printf("Fila: ");  
    while (atual != NULL) {  
        printf("%d -> ", atual->valor);  
        atual = atual->prox;  
    }  
    printf("NULL\n");  
}
```

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Operações obrigatórias

1. **Entrar na fila**: inserir um código de pedido no final da fila.
2. **Chamar próximo**: mover o primeiro da fila para “**em atendimento**”.
3. **Finalizar compra**: remover a pessoa “**em atendimento**” (compra concluída).
4. **Expirar tempo**: remover a pessoa “**em atendimento**” (perdeu a vez).
5. **Imprimir** fila.

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Operações obrigatórias

1. **Entrar na fila**: inserir um código de pedido no final da fila. -----> enfileirar
2. **Chamar próximo**: mover o primeiro da fila para “**em atendimento**”.
3. **Finalizar compra**: remover a pessoa “**em atendimento**” (compra concluída).
4. **Expirar tempo**: remover a pessoa “**em atendimento**” (perdeu a vez).
5. **Imprimir** fila.

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Operações obrigatórias

1. **Entrar na fila**: inserir um código de pedido no final da fila.
2. **Chamar próximo**: mover o primeiro da fila para “**em atendimento**”. ----> desenfileirar + operação com valor desenfileirado
3. **Finalizar compra**: remover a pessoa “**em atendimento**” (compra concluída).
4. **Expirar tempo**: remover a pessoa “**em atendimento**” (perdeu a vez).
5. **Imprimir** fila.

• **OBS: TODA OPERAÇÃO REALIZADA EM UMA TAD DEVE SER FEITA ATRAVÉS DE FUNÇÕES**

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Operações obrigatórias

1. **Entrar na fila**: inserir um código de pedido no final da fila.
2. **Chamar próximo**: mover o primeiro da fila para “**em atendimento**”.
3. **Finalizar compra**: remover a pessoa “**em atendimento**” (compra concluída). -----> operação no main
4. **Expirar tempo**: remover a pessoa “**em atendimento**” (perdeu a vez).
5. **Imprimir** fila.

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Operações obrigatórias

1. **Entrar na fila**: inserir um código de pedido no final da fila.
2. **Chamar próximo**: mover o primeiro da fila para “**em atendimento**”.
3. **Finalizar compra**: remover a pessoa “**em atendimento**” (compra concluída).
4. **Expirar tempo**: remover a pessoa “**em atendimento**” (perdeu a vez). -----> operação no main
5. **Imprimir** fila.

Exercício – Fila Online para Comprar Ingresso de Show

Uma plataforma de venda de ingressos mantém uma **fila online** para organizar quem vai comprar primeiro.

Regras do sistema

- Cada pessoa entra na fila com um **código de pedido** (inteiro).
- Quem entra primeiro é atendido primeiro (**FIFO**).
- Quando chega a vez, a pessoa **sai da fila** e fica com o status “**em atendimento**” (ela tem a chance de comprar).
- Se a pessoa **não finalizar a compra no tempo (simulado)**, ela **perde a vez e sai do atendimento**.

Operações obrigatórias

1. **Entrar na fila**: inserir um código de pedido no final da fila.
2. **Chamar próximo**: mover o primeiro da fila para “**em atendimento**”.
3. **Finalizar compra**: remover a pessoa “**em atendimento**” (compra concluída).
4. **Expirar tempo**: remover a pessoa “**em atendimento**” (perdeu a vez).

5. **Imprimir fila.** -----> imprimirFila

Exercício – Fila Online para Comprar Ingresso de Show

```
int emAtendimento
```

Armazena o **código do cliente** que já saiu da fila e agora **está sendo atendido** pelo sistema

- **Não** faz parte mais da fila.
- O sistema só suporta **um cliente** em atendimento **por vez**
- **-1** → "**ninguém em atendimento**"

Exercício – Fila Online para Comprar Ingresso de Show

Função a ser implementada:

chamarProximo → Chama o próximo para atendimento

Ações:

- **remove** do início da fila (desenfileira)
- coloca em "**emAtendimento**"
- retorna **1** se conseguiu, **0** caso contrário.

Exercício – Fila Online para Comprar Ingresso de Show

Função a ser implementada:

chamarProximo → Chama o próximo para atendimento

Ações:

- **remove** do início da fila (desenfileira)
- coloca em "**emAtendimento**"
- retorna **1** se conseguiu, **0** caso contrário.

```
int chamarProximo(Fila* fila, int* emAtendimento) {  
    if (!fila || filaVazia(fila)) return 0;  
  
    *emAtendimento = obterInicio(fila);  
    desenfileirar(fila);  
    return 1;  
}
```

Exercício – Fila Online para Comprar Ingresso de Show

Tarefa: implementar a seguinte função e adicionar no menu de opção:

desistirDaFila → Remove um código específico de qualquer posição (desistência)

Ações:

- Verifica se o **pedido está em atendimento. Se não:**
- **Procura** o código ao longo da fila (percorrer a fila)
- **Remove** o código se encontrar
- **retorna 1** se removeu, **0** se não encontrou